Deep Learning for Biological Discovery Across Omics Layers: From Phenomics to Mechanistic Reasoning

Comprehensive Exam

The University of Tennessee, Knoxville

Peter Kruse

September 2026

Contents

List of Tables

List of Figures

Nomenclature

MENTOR-EV	MENTOR applied with every gene in a multiplex as a seed, yielding a hierarchy over the full gene universe
MENTOR-IA	MENTOR Interpretation Agent
MENTOR-RL	The reinforcement learning-trained mechanistic reasoning model proposed in Chapter 4
MENTOR	Multiplex Embedding of Networks for Team-Based Omics Research
BPE	Byte-pair encoding
CCC	Concordance correlation coefficient
CIELAB	CIE $L^*a^*b^*$ color space; a^* is the green–red axis and b^* the blue–yellow axis
CNN	Convolutional neural network
DDP	Distributed data parallelism
DPO	Direct preference optimization
EC	Environmental covariate
FSDP	Fully-sharded data parallelism

G2F	Genomes to Fields
GAPIT	Genome Association and Prediction Integrated Tool
GBLUP	Genomic best linear unbiased predictor
GO	Gene Ontology
GRIN	Gene set Refinement through Interacting Networks
GRPO	Group-relative policy optimization
GWAS	Genome-wide association study
G×E	Genotype-by-environment
H^2	Broad-sense heritability
HPC	High-performance computing
HSV	Hue, saturation, value (brightness) color space
ILSVRC	ImageNet Large Scale Visual Recognition Challenge
IoU	Intersection over union
iRF-LOOP	Iterative Random Forest Leave-One-Out Prediction
KEGG	Kyoto Encyclopedia of Genes and Genomes
LD	Linkage disequilibrium
LEO	Leave-environment-out validation
LLM	Large language model
LoRA	Low-rank adaptation
LSTM	Long short-term memory

MAF	Minor allele frequency
MDI	Mean decrease in impurity
MDP	Markov decision process
mIoU	Mean intersection over union
MLM	Mixed linear model
MLMM	Multi-locus mixed model
MLP	Multilayer perceptron
MoE	Mixture of experts
MSE	Mean squared error
OLCF	Oak Ridge Leadership Computing Facility
PCC	Pearson correlation coefficient
PEFT	Parameter-efficient fine-tuning
PEN	Predictive expression network
PPN	Predictive phenotype network
PPO	Proximal policy optimization
RGB	Red, green, blue color space
RL	Reinforcement learning
RLHF	Reinforcement learning with human feedback
RNN	Recurrent neural network
RWR	Random walk with restart

RWR-LOE	Random walk with restart, lines of evidence
SAM	Segment Anything Model
SFT	Supervised fine-tuning
SNP	Single-nucleotide polymorphism
SNV	Single nucleotide variant
TAIR	The Arabidopsis Information Resource
TAV2	Pennycress (*Thlaspi arvense*) version 2 reference genome
TRPO	Trust region policy optimization
ViT	Vision transformer
ZeRO	Zero Redundancy Optimizer

Executive Summary

Motivation

Modern biology has become increasingly data-rich but interpretation-limited. Data collection is now cost-effective in many realms, enabling the construction of large, novel corpora. High-throughput imaging technology has led to population-scale phenotyping datasets, while advanced sequencing methods allow researchers to capture genome-scale variation across populations. Global consortia efforts have standardized environmental metadata across climates and sites, while novel experimental data and computational frameworks have prompted the curation and creation of biological networks down to the single-cell level. These data span fundamentally different structures: images contain spatial, pixel-level information; genotypes comprise high-dimensional, sparse signals; environments exist in a continuous and contextual space; and networks represent relational graph structures. Handling these structures requires systems designed with different inductive biases; no single model class can handle them all.

Thesis Statement

While deep learning has been successfully applied to several domains, biological data are heterogeneous and structurally varied, making the gains from current deep learning methods unequally distributed across biology. Architectures, objectives, and benchmarks must be designed with appropriate inductive biases to match the structure of biological subdomains. This proposal demonstrates the utility of this framework across three linked stages of biological discovery: phenomic *measurement*, genomic and environmental *prediction*, and interactomic *interpretation*.

Chapter 1 Summary

Chapter 1 reviews the history of deep learning and its application to biological problems. We first discuss the origins of deep learning, starting with the formalization of the artificial neuron in the 1940s and tracing the path of development to modern networks with trillions of parameters. Then, we cover two domains that have seen rapid gains from deep learning: computer vision and language modeling. Next, we discuss the scaling laws and distributed training strategies that made these gains possible. We close by exploring the application of neural networks to phenomics, genomics, and interactomics. We find that gains from deep learning have followed from matching an architecture's inductive biases to the structure of its input data, and that biological data satisfy those assumptions unevenly across omics layers.

Chapter 2 Summary

Chapter 2 addresses the *measurement* stage by extracting quantitative phenotypes from raw imagery. No deep learning framework currently exists for phenotyping pennycress (*Thlaspi arvense L.*), an emerging oilseed cover crop. In this work, we developed an automated deep learning pipeline that extracts 52 tissue-resolved phenotypes per seed pod. We benchmarked convolutional, transformer, and foundation-model segmentation architectures using 347 manually annotated pods from 21 scans, including an independent test set of 65 pods. A boundary-down-weighted U-Net achieved 85.58% mean intersection over union while offering the most favorable accuracy–efficiency tradeoff. We then applied the pipeline to 12,253 seed pods from 768 accessions and connected the resulting traits to candidate genetic mechanisms through heritability filtering, genome-wide association, and network-based refinement. We identified three modules of network-supported genes whose candidate mechanisms recover known *Brassicaceae* pod and seed biology.

Chapter 3 Summary

Chapter 3 addresses the *prediction* stage by mapping genotype and environment to phenotype. Accurate maize yield prediction is crucial for global food security, but it remains challenging due to complex genotype-by-environment (G×E) interactions. In this work, we present a unified transformer architecture for maize yield prediction that places 2,224 single nucleotide variant markers and 702 environmental covariates in one sequence, allowing self-attention to learn direct G×E interactions. A sparse mixture-of-experts layer supports token-level specialization, and an environment-conditioned transformation head calibrates absolute yield while preserving within-environment rank. We trained our model on 143,050 pre-2024 Genomes2Fields plot records from 238 environments and retrospectively evaluated it on 9,486 records from 22 environments in the 2024 competition cohort. The selected model achieved a macro environment Pearson correlation coefficient (PCC) of 0.423 and mean squared error of 55.40, corresponding to third place in a retrospective competition comparison, 0.014 PCC below the winning correlation of 0.437.

Chapter 4 Summary

Chapter 4 addresses the *interpretation* stage by training a language model to reason over biological networks and produce evidence-grounded mechanistic hypotheses. We propose MENTOR-RL, an open-weight model trained on an ontology-grounded lattice of context-specific multiplex networks. First, a world-model curriculum teaches the operators that underlie the lattice structure. Then, direct preference optimization and reinforcement learning train a tool-using policy with deterministic, verifiable rewards. We use two flagship tasks to test the resulting model. The first generates a hierarchy for an unseen context, which requires context-specific structure rather than a memorized hierarchy. The second asks the model to project a target gene's neighborhood onto that hierarchy and recover the network connections that the

hierarchy does not represent. Infrastructure experiments demonstrate full fine-tuning of gpt-oss-120b on up to 128 Frontier nodes and an end-to-end trajectory loop. In preliminary mapping experiments, atomic tokenization increased overall accuracy from 45.6% to 85.9% over the base tokenizer on a hard subset of test cases.

Completion Plan and Expected Outputs

Chapter 2 is being prepared for submission to *Plant Phenomics*, and Chapter 3 to *The Plant Genome*, while Chapter 4 defines the proposed remaining research and its staged evaluation plan. Upon publication, we will release the open-source ground truth dataset of hand-annotated seed pod images and segmentation pipeline for Chapter 2; the training and evaluation code for Chapter 3; and the training environment, evaluation harness, and trained checkpoints across pipeline stages for Chapter 4.

Chapter 1

Literature Review

1.1 Introduction

Deep learning has become the dominant approach to processing massive, unstructured datasets. However, progress has been concentrated in domains with data that match the structure of neural architectures. This review traces the history of deep learning and explores where it can be applied to various biological problems. Throughout, we examine the *inductive biases* each architecture draws on to encode data. Because each omics layer has its own unique structure and characteristics, no single architecture serves them all. Tailoring inductive biases to the layer being modeled is what makes deep learning effective across omics layers, and it is the focus of this proposal. This proposal demonstrates the utility of this framework across three linked stages of biological discovery: *measurement*, *prediction*, and *interpretation*. Each stage operates on a different omics layer: measurement extracts phenotypes from images, prediction maps genotype markers and environmental covariates to yield values, and interpretation reasons over interactomic networks to produce mechanistic hypotheses.

We begin in Section 1.2 with the history of neural network research, then examine two modalities that served as proving grounds: computer vision in Section 1.3 and language modeling in Section 1.4. Section 1.5 covers the scaling

laws and distributed training strategies that support models at their current size. Section 1.6 turns to biological applications, covering the three areas this proposal addresses: phenotypic measurement, genomic prediction, and mechanistic interpretation. Finally, Section 1.7 synthesizes these findings.

1.2 Deep Learning History

According to Goodfellow et al., the history of neural network research can be divided into three eras: cybernetics, connectionism, and deep learning [58]. In this section, we explore the characteristics of each era and the innovations took place during them. Figure 1.1 summarizes the milestones discussed in this review within this context.

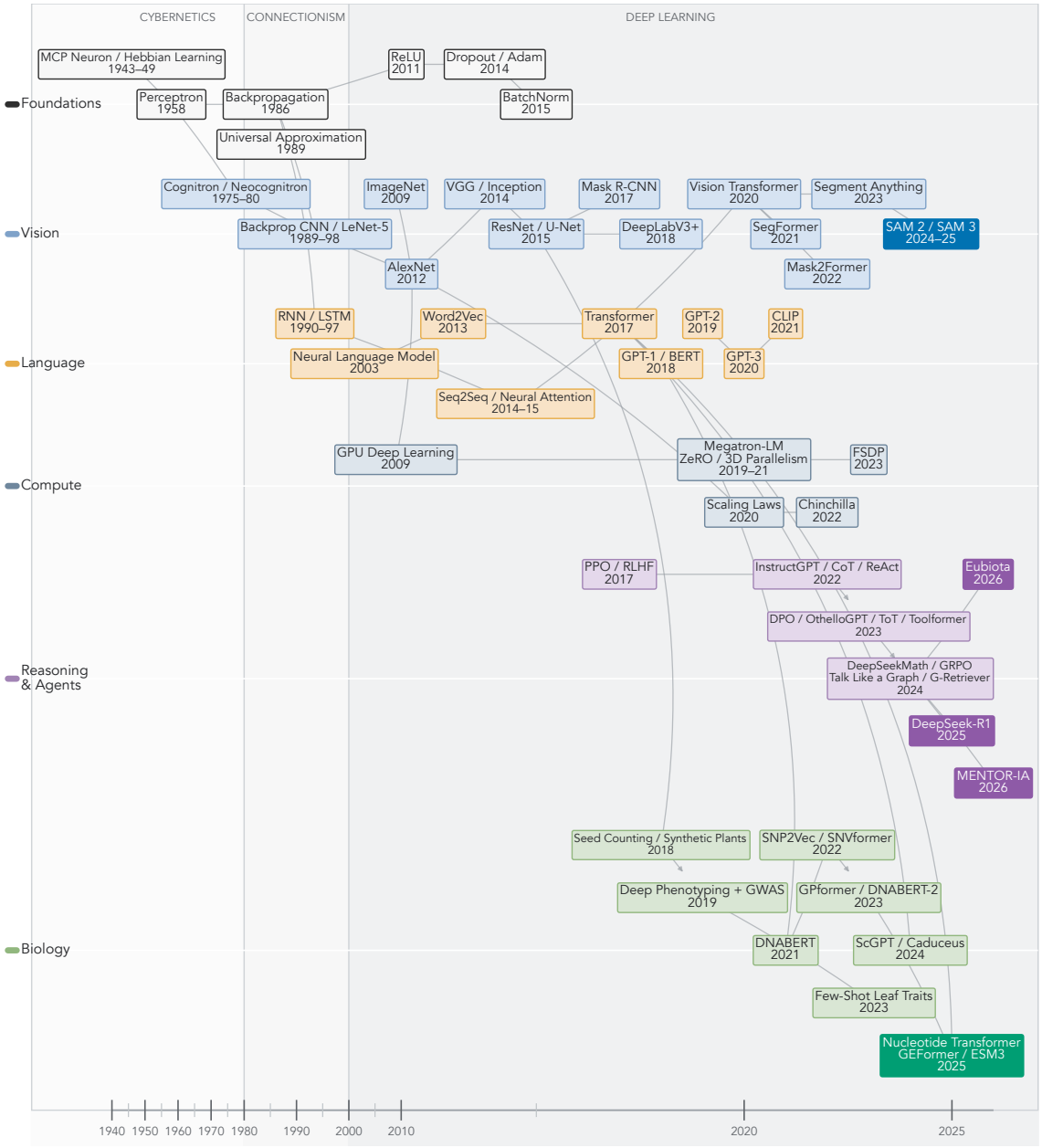

Figure 1.1: Selected milestones in deep learning from 1940 to 2026, grouped by foundations, vision, language, compute, reasoning and agents, and biology. The nonlinear time axis expands the period after 2012. Arrows indicate influence, but not necessarily direct derivation. Related papers are grouped in the same box. Boxes with solid fill indicate recent innovations in a given subdomain.

1.2.1 Cybernetics

The cybernetics era, spanning the 1940s to the 1960s, produced the first mathematical models of biological neural computation. McCulloch and Pitts introduced the artificial neuron, modeling neuronal activity using formal logic [127]. Artificial neurons were framed as binary units within a larger symbolic logic network. They received excitatory or inhibitory signals from connected neurons or external stimuli and produced a signal if these inputs exceeded an activation threshold. These artificial neural networks worked over discrete time steps, so they could represent sequential dynamics, foreshadowing later concepts like recurrence and memory. However, McCulloch and Pitts's formulation lacked a learning mechanism; network structure and parameters were fixed, requiring manual design instead of adapting to data. This static framework stood in contrast to Hebb's work on synaptic plasticity, which postulated that neural pathways are strengthened by repeated co-activation [67].

Rosenblatt's perceptron addressed the learning mechanism directly. In contrast to the symbolic formulation of McCulloch and Pitts, Rosenblatt adopted a probabilistic perspective, treating data as statistically separable and allowing connections to be reinforced or discouraged via positive or negative stimuli from the data [166]. Although trainable neurons and learning from data remain central to modern deep learning, Rosenblatt's formulation was incapable of representing non-linearly separable functions, with the XOR problem serving as a canonical example. This was a consequence of the perceptron's reliance on linear decision boundaries and its restriction to single-layer architectures. These limitations were analyzed by Minsky and Papert [132], whose formal results altered public perception of AI research and contributed to a temporary decline in funding.

1.2.2 Connectionism

The subsequent era of neural network research, commonly referred to as *connectionism*, was defined by the effort to overcome these limits through multi-layer

architectures and the backpropagation algorithm. In 1986, Rumelhart, Hinton, and Williams demonstrated that multi-layer networks could be effectively trained through a general procedure called *backpropagation* [168]. The mathematical foundations of backpropagation had been derived earlier by Linnainmaa [118] and applied to neural networks by Werbos [216], but this work popularized the method within the AI community. Backpropagation relies on the differentiability of both the loss function and the network's activation functions, which allow gradients with respect to all parameters to be computed efficiently via the chain rule. This replaced the manually specified learning rules of the perceptron with an optimization problem, making multi-layer training possible. The same work contributed an explicit learning rate, momentum-based updates, and gradient accumulation, all of which are core to modern neural network optimization.

Rumelhart, Hinton, and Williams also characterized networks as self-organizing systems capable of learning internal representations from data. They theorized that hidden units encode features of increasing complexity as network depth increases, which provided the basis for future work in representation learning. These capabilities were substantiated in 1989, when Cybenko and Hornik et al. independently proved the Universal Approximation Theorem: a feed-forward network with a single hidden layer and a finite number of neurons can approximate any continuous function on a compact set [36, 74]. This confirmed that the limitations Minsky and Papert identified were specific to single-layer architectures, though the theorem addresses representational capacity, not whether such a network can be learned from data.

Connectionist networks nonetheless remained shallow in practice. Training was restricted by the computational resources available at the time; networks were trained on general-purpose central processing units (CPUs), making it impractical to explore deeper architectures or large datasets. Optimization compounded the problem: with activations such as sigmoid and tanh, excessively large or small values fall near the asymptote of these functions, so gradients could become saturated during backpropagation, reducing the learning signal in successive layers (Figure 1.2). These

limitations prevented networks from increasing in depth, and the transition to deep learning was marked by its effort to address them.

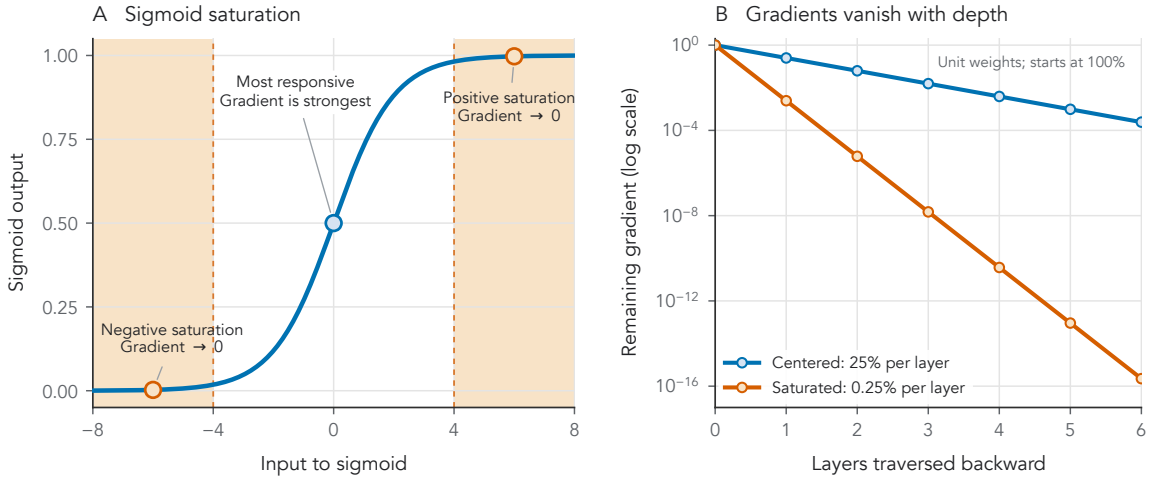

Figure 1.2: Sigmoid saturation and the vanishing gradient problem. **A**, Inputs near either extreme are compressed into the sigmoid's flat tails. In these saturated regions, changing the input barely changes the output, so almost no gradient passes backward. **B**, Backpropagation compounds this loss across layers. In an illustrative network with unit weights, a centered input retains 25% of the incoming gradient per layer, whereas a saturated input retains only about 0.25%. Earlier layers therefore receive an effectively zero learning signal.

1.2.3 Deep Learning

The era of deep learning, a term popularized by Hinton in 2006 [70], stemmed from convergent improvements in hardware, data, and algorithms during the late 1990s and early 2000s. The first of these was the shift from CPUs to Graphics Processing Units (GPUs), whose physical architecture is well suited to homogeneous, parallelizable, and batched operations, specifically the massive matrix multiplications central to neural network training. By storing parameters in global memory, parallelizing homogeneous thread operations in stream processors, and using efficient scheduling to reduce overhead times, Raina et al. achieved up to a 72x speedup over CPU training [157]. This acceleration rendered previously intractable problems computationally feasible, enabling the exploration of deeper architectures and larger datasets.

Such exploration required large, annotated datasets to avoid overfitting and improve generalization. Dataset expansion occurred most notably in computer vision. MNIST [110] served as an early computer vision benchmark but comprised only 10 image classes within a narrow domain, and successors such as Caltech101 [49] and PASCAL VOC [47] were still relatively small and lacked intra-class variability. In 2009, the ImageNet dataset [39] leveraged the increasing amount of data available on search engines and the advent of crowdsourcing platforms to produce 3.2 million annotated images following the hierarchical structure of WordNet [50]. The availability of large-scale datasets encouraged the development of networks with more layers to capture increasingly abstract representations, a shift further catalyzed by the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) [169], which we discuss in Section 1.3.2.

As hardware capabilities improved and massive datasets became available, several algorithmic innovations enabled researchers to train deeper, larger-scale models. One notable breakthrough was the rectified linear unit, or ReLU, which supplanted the sigmoid and hyperbolic tangent (tanh) activation functions that were previously standard [57]. ReLU enforced network sparsity by setting negative activations to zero and mitigated the vanishing gradient problem with strictly linear positive activations, preventing gradient saturation. Weight initialization schemes followed to match each nonlinearity [56, 63]. Dropout addressed generalization challenges posed by these high-capacity models, probabilistically removing neurons during training so that they cannot co-adapt and must instead learn to function with a randomly chosen sample of units [188]. Batch normalization normalized layer activations around the mini-batch mean and variance, accelerating training while acting as a regularizer [86], though Santurkar et al. later argued that its benefit comes from smoothing the optimization landscape, not from reducing internal covariate shift [170]. Finally, the Adaptive Moment Estimation (Adam) optimizer utilized the first and second moments of the gradients to compute adaptive learning rates for each weight, bounding the effective step size and promoting stability when gradients are sparse or noisy [100].

As depth increased, a major paradigm shift occurred: the transition from *feature engineering* to *feature learning* [109]. While prior approaches relied on hand-crafted features to structure data, deeper networks allowed for complex representations to be learned directly in latent space, deriving hierarchical features with backpropagation instead of by hand. With this shift away from manual feature engineering, research focus turned toward architectural and algorithmic design, with inductive biases tailored to specific data modalities. Computer vision assumed translation invariance, while language models assumed sequentiality and contextual dependence. Biological data satisfy these assumptions unevenly, which makes understanding the history and current state of deep learning across modalities crucial for applications in phenomics, genomics, and interactomics.

1.3 Computer Vision

Computer vision served as the primary proving ground for neural network techniques and architectures during the deep learning era. This domain was the first where deep learning achieved widespread consumer adoption [110] and later surpassed human performance [63]. These performance breakthroughs shifted the perception of deep learning from an esoteric field into a rapidly evolving discipline with disruptive applications. This success was not accidental; rather, the alignment between the structure of image data and the inductive biases of neural architectures, specifically convolutional neural networks (CNNs), allowed deep learning models to excel at learning complex representations. Translation invariance, local connectivity, and hierarchy were built into these architectures directly, following the model of the mammalian visual cortex discussed in Section 1.3.1.

The establishment of standardized benchmarks catalyzed the advancement of computer vision, promoting competition and innovation within the community. The PASCAL VOC challenge [47], and subsequently the ILSVRC [169], provided annual venues to evaluate visual recognition algorithms. Distinguished by their massive

scale and public availability, these datasets created a feedback loop that encouraged research participation and drove competition error rates down rapidly (Figure 1.3). The ILSVRC also served as a proving ground, where techniques that succeeded in the image domain became part of the standard toolkit for other deep learning subdomains, including biological modeling.

In this section, we trace the evolution of computer vision, from its biologically inspired origins in Section 1.3.1, to the ILSVRC in Section 1.3.2, and finally to segmentation architectures in Sections 1.3.3 and 1.3.4. Understanding this history is crucial for applying computer vision techniques to specialized domains and datasets, such as biological images.

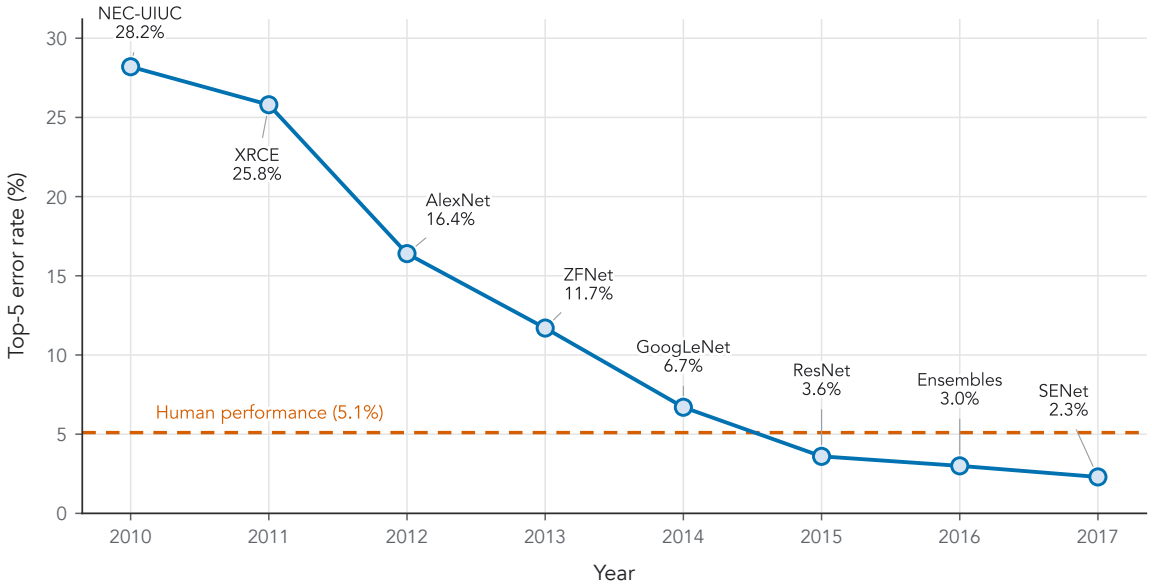

Figure 1.3: Winning top-5 classification error in the ImageNet Large Scale Visual Recognition Challenge (ILSVRC), 2010–2017. Points identify the best-performing entry in each year; the dashed line marks the reported 5.1% human baseline. The sharp decline after AlexNet in 2012 and subsequent gains from GoogLeNet, ResNet, model ensembles, and SENet show the rapid saturation of the benchmark.

1.3.1 Origins

Much like general deep learning (Section 1.2.1), computer vision began as a biologically inspired field grounded in neuroscience principles. In the late 1950s and

early 1960s, Hubel and Wiesel provided the theoretical underpinnings for modern architectures with their work on the mammalian visual cortex [80, 81]. They showed that vision is a hierarchical process built on detecting and combining local features: the visual cortex does not respond to retinal input holistically, but through neurons that respond to stimuli within small, specific regions called *receptive fields*. They further identified two types of cells involved in processing this visual information. While simple cells fire only when a specific feature, like a horizontal or vertical line, appears in a precise location, complex cells activate when the feature appears anywhere in the receptive field. Simple cells connect to complex cells, which connect to hypercomplex cells [79] and onward to higher visual areas, allowing the visual cortex to build a layered representation that abstracts from lines to shapes and eventually to objects (Figure 1.4).

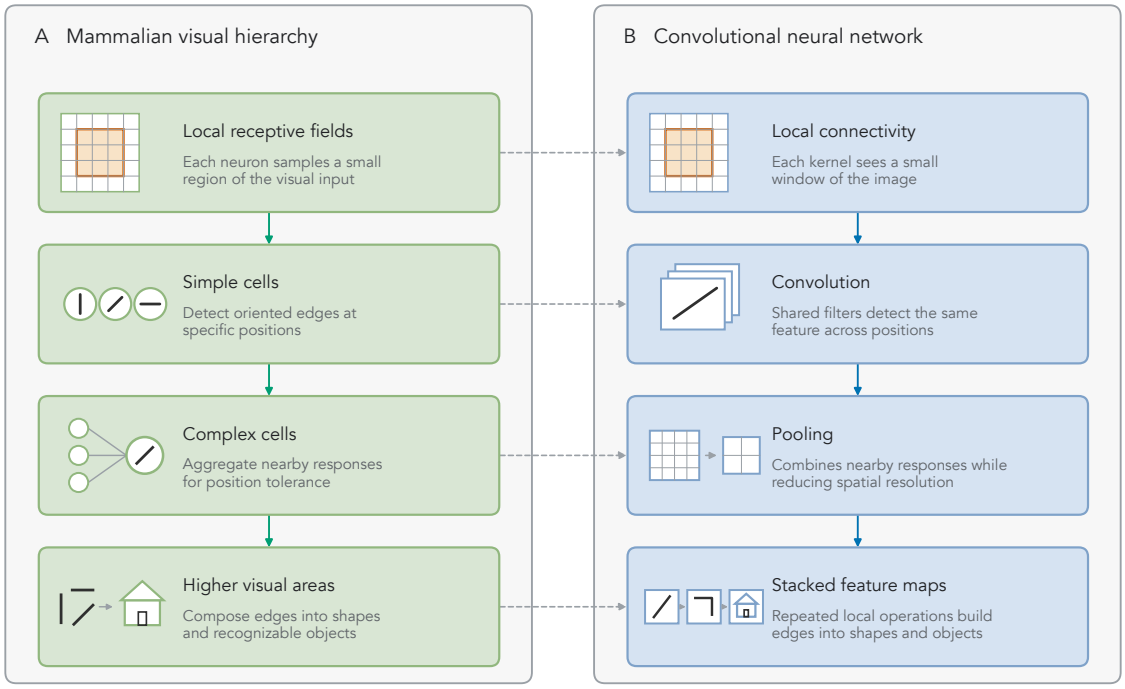

Figure 1.4: Conceptual correspondence between the mammalian visual hierarchy and convolutional neural network (CNN) operations. **A**, Local receptive fields restrict each neuron to part of the visual input; simple cells detect oriented edges at specific positions, complex cells aggregate nearby responses for position tolerance, and higher visual areas compose these signals into shapes and objects. **B**, CNNs encode analogous biases with local kernel windows, shared convolutional filters, pooling, and repeated feature-map transformations that build progressively more abstract representations.

Kunihiko Fukushima built on this work by introducing the cognitron in 1975 and the neocognitron in 1980 [51, 52]. A precursor to modern CNNs, the neocognitron operationalized receptive fields, simple cells, and complex cells directly. Instead of the fully connected layers of the perceptron [166], it featured a shared weight mechanism that simulated simple cells (referred to as "S-cells" by Fukushima) by applying the same filter to each part of the image, with multiple S-cells applied in parallel to detect multiple features. These features were then propagated to complex cells, or C-cells, for transformation-invariant feature detection. S-cells are equivalent to convolution operations and C-cells to pooling operations, so stacking S-C layers

reproduced the hierarchical structure Hubel and Wiesel described. The neocognitron relied on unsupervised competitive learning instead of backpropagation, however, which hindered its standalone success for complex supervised tasks.

In 1989, Yann LeCun combined Fukushima's architectural foundations with the learning rules of Rumelhart et al. [168] to train the first modern CNNs [111], using backpropagation to recognize digits from the United States Postal Service. The model achieved a 1% error rate and 9% rejection rate, demonstrating state-of-the-art performance on a real-world task. Because the model learned hierarchical features, the need for manual preprocessing was minimized, and the shared-weight design both lowered the computational burden relative to fully-connected networks and reduced overfitting by cutting the number of free parameters. LeCun iterated on this approach through LeNet-5 [110], scaling the dataset and model size while comparing against other gradient-based models; LeNet-5 achieved performance superior to most existing methods and comparable to Support Vector Machines (SVMs), while offering significantly faster inference speeds. LeCun noted that "better pattern recognition systems can be built by relying more on automatic learning and less on hand-designed heuristics" [110]. LeCun's work also introduced a modular paradigm for neural network programming, composing forward and backward propagation functions while storing a computational graph. This work is a precursor to modern deep learning frameworks such as Caffe [92], TensorFlow [126], PyTorch [148], and Jax [18].

However, training was still constrained by hardware and data limitations: LeNet-5 took three days to train for 20 epochs, and MNIST contained only around 60,000 images. As discussed in Section 1.2.3, several innovations lifted these constraints, with the ILSVRC [169] serving as the catalyst for rapid progress.

1.3.2 The ImageNet Challenge

Because the ILSVRC served as the field's standardized benchmark, its winning entries mark the major architectural advances of the period. The first was AlexNet [105],

which won the competition in 2012 with a reduction in top-5 error rate of nearly 10 percentage points over the previous winner, producing "by far the best results ever reported on these datasets" [105] at the time. The authors used model parallelism across two GPUs, partitioning convolutional filters between them to distribute processing; despite being one of the largest CNNs of its time, training took a relatively modest five to six days. AlexNet also combined dropout, ReLU, and data augmentation at scale. The adoption of these techniques, combined with the competition-winning performance they yielded, solidified AlexNet as a foundational work and exemplified the superiority of deep learning for image recognition over other techniques, like SVMs, which were popular at the time. AlexNet's success also foreshadowed modern scaling laws (Section 1.5.1); the authors demonstrated that using more compute and data had a direct relationship with performance, a trend that continued to be explored across deep learning subdomains in the following years.

In 2014, two contrasting architectures achieved first and second place in the ILSVRC. GoogLeNet [194] won with its Inception architecture, which enabled the training of even deeper CNNs while using fewer parameters. Each inception block applied 1×1, 3×3, and 5×5 convolutional kernels in parallel within a single layer, giving the block an explicit range of receptive-field sizes while balancing broad context and fine localization through training. To reduce the number of parameters in the network, and thus overfitting, GoogLeNet used global average pooling to transform the inception outputs for classification rather than the dense layers that previous architectures had relied on for reshaping outputs [105]. It also leveraged auxiliary classification, adding outputs at intermediate layers so that backpropagation from these additional objectives maintained a strong gradient signal throughout an increasingly deep network. VGGNet took the opposite approach, demonstrating that a homogeneous network with many layers of small convolutions could yield comparable performance [186]. The authors tested CNNs of various depth while exclusively using 3×3 convolutional filters, arguing that stacks of smaller filters were equivalent to a single, larger filter while fitting a more discriminative function

with fewer parameters. Where Inception showed that a complex, highly engineered CNN could reach state of the art, VGGNet showed that a simple, uniformly stacked one could as well, and its scalability and simplicity led to its adoption across multiple other subdomains.

ResNet [62] won the ILSVRC in 2015 by addressing the vanishing gradient problem, which, despite being partially treated by innovations such as ReLU [57], was still prevalent as network depth exceeded 30 layers. The authors introduced residual learning, where instead of fitting a function, $h(x)$, its residual, $h(x) - x = f(x)$, was learned. Consequently, layer mappings could be reformulated as learning $f(x) + x$, including a direct connection from the input to the output of a layer. This directly addressed gradient saturation by providing a residual stream without exposure to a nonlinearity, stabilizing gradient flow and allowing the authors to train networks with a depth of up to 152 layers. The innovations introduced in ResNet have been adopted by numerous state-of-the-art architectures across subdomains, showing the impact of this work.

Over its life cycle from a difficult, open research competition to a saturated benchmark, the ILSVRC yielded many innovative approaches to image classification. Model parallelism [105], dropout [71], ReLU [57], homogeneous block architectures [186], and residual connections [62] are all used in various architectures across subdomains today. In the next section, we discuss how these methods were incorporated to address the challenges of semantic segmentation and object detection.

1.3.3 Semantic Segmentation

As performance on the ILSVRC improved, the benchmark became saturated; state-of-the-art models reached an accuracy plateau where meaningful improvements became marginal. Due to this saturation, focus shifted to more complex tasks such as semantic segmentation, where specificity and localization carried greater weight.

While semantic segmentation was historically tied to natural-image benchmarks such as PASCAL VOC [47], one of the first state-of-the-art segmentation models was developed for biomedical image segmentation. U-Net [165], developed in 2015, is a fully convolutional architecture that produces pixel level classifications to create image segmentation maps. The architecture features a contractive path to capture hierarchical and contextual features followed by a symmetric expansive path to provide specific localizations within the image, with the two paths linked via skip connections. Functionally similar to residual connections [62], skip connections improve gradient flow in the model while also enabling the combination of hierarchical features with localized features via concatenation in the expansive feature map. U-Net achieved state-of-the-art performance on both *Drosophila* nerve membrane segmentation and cellular segmentation, solidifying its role as a biologically useful deep learning architecture.

In 2017, He et al. introduced Mask R-CNN, an alternative architecture that unified object detection, instance segmentation, and classification into a multi-task objective [64]. Expanding on the work of the Fast R-CNN [55] and Faster R-CNN [162], which explored only classification and object detection, the Mask R-CNN added a semantic segmentation path that was trained simultaneously with the classification and bounding box objectives. Unlike semantic segmentation, which labels all pixels of a class identically, this approach enabled instance segmentation, distinguishing between individual objects of the same class. This architecture achieved state-of-the-art performance on the common objects in context (COCO) dataset [117] and human pose estimation, showing the transferability and generality of the Mask R-CNN method.

A parallel line of convolutional work pursued dense prediction without U-Net's symmetric contraction and expansion. DeepLabV3+ [23] used atrous, or dilated, convolutions to enlarge the receptive field without reducing spatial resolution or increasing the parameter count, addressing the loss of localization that is introduced by repeated downsampling. Atrous spatial pyramid pooling applies these convolutions

at several dilation rates in parallel, capturing context at multiple scales within a single layer, and a lightweight decoder then recovers object boundaries. DeepLabV3+ remains a strong convolutional baseline for semantic segmentation and is frequently used as a point of comparison for later architectures.

1.3.4 Transformer-Based Segmentation

Despite the innovations of Mask R-CNN and U-Net, the explosive success of transformer architectures in the language modeling domain (detailed in Section 1.4) inspired novel approaches to computer vision. Dosovitskiy et al. introduced the vision transformer (ViT) in 2020 [43]; instead of relying on convolutional layers, which were the standard for computer vision architectures, the authors leveraged the attention mechanism. By dividing images into 16×16 pixel patches and encoding these patches into tokens with a linear layer, the ViT architecture allowed images to be processed as sequences. The original ViT was supervised and required very large pretraining corpora to outperform CNNs, but it established the architectural framework that later enabled self-supervised masked image modeling [65], reducing reliance on expensive labeled data. Although not strictly a segmentation model, the ViT created a framework that enabled transformer-powered segmentation.

Applying the ViT to dense prediction required adapting it to produce per-pixel output instead of a single class label. SegFormer [220] paired a hierarchical transformer encoder, which produces feature maps at multiple resolutions in the manner of a convolutional backbone, with a lightweight all-MLP decoder. Because the encoder aggregates context across scales, SegFormer needs no explicit positional encodings, which degrade when inference resolution differs from training resolution. Mask2Former [28] took a different route, building on the mask-classification formulation [27] rather than per-pixel classification: instead of labeling each pixel independently, the model predicts a set of binary masks and assigns a class to each. Masked attention restricts each query's attention to the region of its own

predicted mask, which speeds convergence and sharpens boundaries. Because semantic, instance, and panoptic segmentation differ only in how the predicted masks are grouped, this formulation unified tasks that had previously required separate architectures.

Building on the ideas introduced by the ViT, researchers at Meta released the Segment Anything Model (SAM) for semantic and instance segmentation [102]. SAM is a foundation model built with a ViT architecture, trained on over 11 million images and 1 billion segmentation masks with a model-in-the-loop paradigm. Newer versions, SAM 2 [160] and SAM 3 [22], were released in 2024 and 2025 to enable video segmentation, improve inference speed, and enhance text prompting.

These architectures are pretrained on large natural-image corpora and operate natively on scenes at high resolution. Biological imaging frequently departs from that regime, presenting narrow-domain subjects, limited annotation, and images large enough that they must be tiled into small patches before they fit in memory. Whether the advantages these models demonstrate on natural-image benchmarks survive this domain shift is an empirical question, and one we return to in Section 1.6.1.

1.4 Language Modeling

Like computer vision, language modeling has been a proving ground for deep learning, with the two subdomains evolving in parallel. As in other deep learning domains, language model architectures are shaped by the properties of data through inductive biases. Modeling language assumes that word order carries meaning, that a word's meaning changes with the words around it, and that sentences can be of arbitrary length. These assumptions, also known as *sequentiality*, *contextual dependence*, and *variable length*, shape the architectures that follow. Language modeling also comprises a set of related tasks: next-token prediction of a discrete vocabulary for generation,

masked token prediction for representation learning and classification, and sequence-to-sequence modeling for translation, among others. Most language modeling tasks are self-supervised, and advances in one task often inform the development of others.

In this section, we discuss the evolution of language modeling, beginning with the origins of the field in Section 1.4.1, followed by the introduction of the transformer architecture in Section 1.4.2. We conclude with the reinforcement learning methods used to steer model behavior after pre-training in Section 1.4.3.

1.4.1 From Counting to Recurrence

The statistical treatment of language was formalized by Claude Shannon in 1948. Shannon formulated text as a stochastic process, establishing natural language as having a measurable statistical structure; this structure could be exploited to predict the next word in a sequence, which is the fundamental objective of language modeling [178]. Specifically, modeling the probability of a sequence of words occurring, $P(A, B, C)$, can be decomposed into a product of conditional probabilities:

$$P(A, B, C) = P(A)\, P(B|A)\, P(C|A, B). \tag{1.1}$$

Early language models were based on this principle, applying a Markov assumption of order $n-1$ to predict the next word from the previous $n-1$ words [25]. However, these *n-gram* models were limited by the curse of dimensionality [12]: as n increased, the number of possible sequences grew exponentially, so modeling 10 consecutive words with a vocabulary of 100,000 words would require estimating $100{,}000^{10} = 10^{50}$ probabilities. N-gram models were therefore limited to small values of n, typically bigrams or trigrams, and remained subject to a second failure case: a sequence never observed in the training data received a probability of zero, which drives cross-entropy to infinity [25]. Smoothing techniques redistributed probability mass to unseen sequences, but the deeper limitation was the representation itself. By modeling words as discrete tokens, n-gram models could not capture semantic relationships between

words, such as synonyms or antonyms [12], which motivated learning distributed representations and catalyzed research in neural language modeling.

Bengio et al. first trained distributed representations of individual words with a neural language model in 2003 [12], in the form known today as word embeddings. Recognizing the difficulty of discrete language modeling, the authors proposed a neural architecture whose initial layer was a lookup table that mapped each token in the vocabulary to a continuous vector, allowing the discrete representation to be distributed continuously in vector space and the model to capture semantic relationships between words. Mikolov et al. extended this with the word2vec framework in 2013 [131], learning word embeddings by predicting the context of a word in a sentence. Training on a significantly larger corpus than previous work, the authors demonstrated that the learned embeddings captured semantic relationships well enough to support vector arithmetic. For example, the operation $v(\text{king}) - v(\text{man}) + v(\text{woman})$ produces a vector that is closest to $v(\text{queen})$, indicating that vector dimensions could correspond to concepts like royalty and gender. Together these works established that neural architectures could learn meaningful representations of words from large corpora of text.

However, utilizing learned representations in a model also required architectural innovation. The first widely-used neural architecture for language modeling was the recurrent neural network (RNN), introduced by Elman in 1990 [45]. The RNN consists of a hidden state, h_t, that is updated at each time step based on the current input, x_t, and the previous hidden state, h_{t-1}, allowing the network to maintain a memory of previous inputs. Context length was limited by the vanishing gradient problem, however: as sequences grew longer, the gradient signal that propagated through the RNN's hidden states would either vanish or explode, making training on long sequences unstable. Hochreiter and Schmidhuber addressed this with the long short-term memory (LSTM) architecture in 1997 [72], which introduced a cell state, c_t, updated at each time step through a series of gates that control the flow of information. Because the cell state is updated without nonlinearities, the gradient

signal can propagate through the network without vanishing or exploding, allowing the LSTM to maintain a memory of previous inputs over long sequences and making it the dominant architecture for language modeling for nearly two decades.

Neural methods began to displace count-based language modeling in 2014, when Sutskever et al. employed LSTMs to achieve state-of-the-art performance on machine translation [192]. They used two LSTMs: an encoder to represent the source sequence, and a decoder to generate the target sequence, with the encoder's hidden state used to initialize the decoder's and thereby transfer information from the source to the target. Bahdanau et al. argued that this sequence-to-sequence (seq2seq) architecture was limited by its fixed-length representation of the source sequence, which was a bottleneck for long sequences, and proposed an attention mechanism that allowed the decoder to compare its current state to the encoder's hidden states and gather information from the most relevant parts of the source [6]. The introduction of attention laid the foundation for the transformer architecture, which would revolutionize language modeling.

1.4.2 The Transformer

The transformer architecture was introduced by Vaswani et al. in 2017 [206], with the original work once again being applied to machine translation. The architecture consisted of an encoder and decoder, each composed of multiple transformer blocks that featured multi-head self-attention and feedforward layers; as in seq2seq, the encoder generates a representation of the input sequence, which is then used by the decoder to generate the output sequence. Because self-attention is permutation-invariant, positional information must be injected explicitly, so the original work added sinusoidal positional encodings to the input embeddings. The attention mechanism allows the transformer to excel at capturing long-range dependencies, and the model's lack of a hidden state makes it parallelizable on the time dimension.

These properties allowed the transformer to scale to larger datasets and model longer sequences than previous architectures.

The Generative Pre-trained Transformer (GPT) series demonstrated the gains from scalability. GPT-1 [153] was a decoder-only architecture that, instead of producing a target sequence from an input sequence, predicted the next token in a sequence given previous tokens. Notably, this is the same modeling approach taken by n-gram models. GPT-1 was pre-trained on a large corpus of text using a negative log-likelihood objective and then fine-tuned on specific tasks using labeled data, a paradigm that allowed general language representations to be adapted to various domains. GPT-2 [154] showed that fine-tuning was not always necessary, achieving state-of-the-art performance on several tasks using only prompted examples at inference time; this demonstrated that models could show emergent behavior when scaled to larger sizes, and began to popularize foundation modeling, where a single model is pre-trained on a large corpus and then adapted to specific tasks. GPT-3 [20] scaled to 175 billion parameters, two orders of magnitude larger than GPT-2, and introduced in-context few-shot learning, where merely supplying a few examples of the task during prompting was sufficient to achieve state-of-the-art performance. Across the GPT series, large language models (LLMs) generalized to new tasks without task-specific training, further solidifying the foundation modeling paradigm.

The transformer has also been applied well beyond autoregressive generation. BERT [40] uses a masked language modeling objective to learn representations of text, the ViT of Section 1.3.4 uses a transformer architecture to process images, and CLIP [155] learns joint representations of images and text. These models have achieved state-of-the-art performance on a variety of tasks, demonstrating the versatility and power of the transformer architecture.

Furthermore, several architectural innovations have been made to improve the efficiency and scalability of transformer models. Most notably, the feed-forward blocks in modern transformers are often replaced with sparse mixture-of-experts (MoE) layers [180] which allow more parameters to be used without increasing

the computational cost of inference. Rotary positional embeddings (RoPE) [189] and context extension methods such as YaRN [149] have allowed models to handle longer contexts, and parameter-efficient fine-tuning (PEFT) methods such as low-rank adaptation (LoRA) [75] allow models to be adapted to new tasks without retraining every parameter.

1.4.3 Reinforcement Learning and Post-Training

While transformer models trained for next token prediction demonstrated strong performance on a variety of tasks, pre-training imbues the model with the *capability* to generalize to many tasks without guaranteeing the model's *behavior*. In many cases, labeled datasets do not exist to train model behavior through imitation learning, so reinforcement learning (RL) is used instead to steer that behavior through comparative judgements and verifiable rewards. A core challenge of RL is credit assignment; in the case of language model post-training, how does one determine which tokens in a sequence were responsible for correct or incorrect outcomes? Even if relevant tokens can be identified, assigning credit to them through backpropagation is non-trivial. The lineage of RL methods developed to train LLMs can be viewed as successive answers to the credit assignment problem.

Reinforcement learning is formulated as a Markov decision process (MDP) with an agent, environment, state, and reward: the agent takes actions in the environment, transitioning between states and receiving rewards based on those actions, with the goal of learning a policy that maximizes the expected cumulative reward over time. Sutton's work on temporal-difference learning [193] laid much of the foundation for RL research, which has since been applied to a variety of domains. Applying policy gradients to large models required constraining how far a single update could move the policy. Trust region policy optimization (TRPO) [175] constrained the update to a trust region with Kullback-Leibler (KL) divergence, ensuring that the new policy is not too far from the old policy, but it is complex to implement and requires

second-order optimization, which can be computationally expensive. Proximal policy optimization (PPO) [174] introduced a clipped importance ratio instead, simplifying the optimization process while improving performance. The clipped objective ensures that a single update does not change the policy too much in the direction of positive-reward actions, while still allowing the penalty for negative-reward actions to be unbounded. PPO has become a standard method for RL in language modeling.

Steering behavior also requires a reward, but language, like many other domains, is difficult to quantify. Reinforcement learning with human feedback (RLHF) [29] replaced numerical rewards with human preferences, training a reward model on a dataset of human-labeled comparisons between different outputs so that the agent could learn a policy aligned with those preferences. In 2022, OpenAI unified PPO and RLHF into a single framework with the release of InstructGPT [146], replacing the numerical reward function in PPO with a learned reward model and yielding a model that was well aligned with human preferences despite only being trained with preference comparisons. This result set a new standard for LLM training, but RLHF can be unstable, requiring careful tuning of the reward model and the policy during training, and its dual optimization is computationally expensive. Direct preference optimization (DPO) [156] was introduced as an alternative, reformulating the RLHF objective as an offline one in which the model is directly optimized against recorded human preference comparisons, simplifying the optimization process and reducing the computational cost of training while still achieving strong performance on preference alignment tasks.

While the above methods all address preference alignment within language models, other approaches have been introduced for reinforcement learning through verifiable rewards. Verifiable domains, such as coding and math, have clear correctness criteria that can be used to update the model's policy. DeepSeekMath [179] introduced group-relative policy optimization (GRPO), which samples from a group of rollouts and calculates the advantage as the rollout's divergence from the group mean. Because rewards in these domains are cheap to calculate, no value model is required, roughly

halving the trainable parameters compared to PPO. DeepSeek then applied GRPO at scale in a pair of models: DeepSeek-R1 and DeepSeek-R1-Zero [60]. DeepSeek-R1-Zero was post-trained using GRPO alone, demonstrating that reasoning could be induced solely through post-training with verifiable rewards. Additionally, the authors observed emergent behaviors like backtracking and self-correction. DeepSeek-R1 added a supervised fine-tuning stage prior to the reinforcement learning phase, improving readability and achieving performance comparable to closed-source frontier models at the time. These results suggest that reinforcement learning can be used to induce long-form, multi-step reasoning, making it a crucial component of the standard training pipeline for LLMs.

The assumptions of language modeling, such as sequentiality, contextual dependence, and variable length, make text a natural domain to model with mechanisms such as attention. However, in domains such as biology, these assumptions are satisfied unevenly. Genotypes are sequentially ordered along a chromosome, but markers are encoded as dosages with a largely additive signal, so ordering matters less. This means that typical language modeling frameworks must be altered in order to be effective at modeling genomic data; we discuss techniques for this in Section 1.6.2. Biological networks depart even further from natural language, featuring network topologies that are not sequential and too large to fit in a single context window. However, in this domain, the post-training machinery developed for language can be applied to steer model behavior. Fine-tuning, reinforcement learning, and tool use can be used to align model behavior with biological knowledge, and we discuss the potential for this transfer in Section 1.6.3.

1.5 High-Performance Computing for Deep Learning

Progress in vision and language has depended on high performance computing (HPC), a critical enabler of new architectures and research. As the centers of innovation shifted from academia to industry, the scale of research has expanded, and with it the computational resources required to train state-of-the-art models. In this section, we discuss the scaling laws that motivated HPC use for deep learning, and then the strategies employed to train large models across multiple GPUs and nodes.

1.5.1 Scaling Laws

A central theme in deep learning is that larger models trained on more data with more compute tend to perform better. This was seen in the ImageNet competition, where AlexNet [105] achieved state-of-the-art performance by training a larger model on more data than its competitors, and in language modeling, where GPT-3 [20] demonstrated that scaling to 175 billion parameters yielded emergent capabilities. However, for most of the history of deep learning, this relationship was anecdotal.

In 2020, Kaplan et al. formalized the relationship between model size, dataset size, and compute budget in a series of scaling laws [95]. The authors first demonstrated that scaling is architecture insensitive, implying that these laws are generalizable across deep learning subdomains. They also demonstrated the relationship that each of these three factors has with performance, showing that model loss decreases according to a power law with increasing model size, dataset size, and compute budget. This relationship, along with the finding that small-scale experiments can forecast large-scale results, was a motivating factor for the development of LLMs, which today can be trained with trillions of parameters on trillions of tokens.

However, some details in the original scaling laws were contested with the release of Chinchilla [73]. While Kaplan et al. implied that model size should grow faster than

dataset size, Hoffmann et al. showed that most models were badly undertrained, and parameters and tokens should scale at a 1:1 ratio. While this result did not challenge the notion that larger models trained on more data with more compute perform better, it did frame a concrete question for future work: given a fixed FLOPs budget, how should one trade off model size against training tokens?

The observation of these concrete scaling laws has motivated the development of larger and larger models. However, as model size increases, it becomes impossible to train models on a single GPU, or even a single node. Therefore, the remainder of this section will discuss the distributed training strategies that have been developed to train large models at the billion- and trillion-parameter scale.

1.5.2 Distributed Training Strategies

A variety of distributed training strategies have been developed to meet the scaling needs of deep learning, which can be bound by compute or memory depending on the regime. Data parallelism is a common strategy for improving model throughput; specifically, distributed data parallelism (DDP) [115] replicates the model weights across multiple GPUs, and each GPU is responsible for computing forward and backward passes on a subset of the training data. After each GPU computes its gradients, they are all-reduced to synchronize model weights. This approach is effective for models that fit within a single GPU's memory, but as model size increases, it becomes infeasible to replicate the model across multiple accelerators.

Therefore, the rest of this section covers model parallelism, which divides the model across multiple GPUs to enable training of models that are memory-bound. In order to shard a model across accelerators, different states must be partitioned across GPUs. DeepSpeed ZeRO and fully-sharded data parallelism (FSDP) [158, 227] are two popular approaches for handling model states. DeepSpeed ZeRO partitions the optimizer states, gradients, and parameters of a model across GPUs in separate stages: ZeRO-1 partitions only optimizer states, ZeRO-2 partitions optimizer states and

gradients, and ZeRO-3 partitions optimizer states, gradients, and parameters. FSDP partitions all model states across GPUs, but uses a PyTorch-specific implementation.

In addition to sharded model parallelism, tensor parallelism [184] is a strategy for partitioning the computation of a single layer across devices, separating row- and column-wise operations for different parts of the layer. Pipeline parallelism [78] shards different layers of the model across devices, allowing for concurrent execution of different layers on different GPUs. Expert parallelism [112] moves different experts in an MoE layer to different GPUs, and context/sequence parallelism [89] divides the input sequence across GPUs, allowing for longer contexts to be processed. These strategies can be combined into multi-dimensional parallelism [141], which allows for the training of models with trillions of parameters on thousands of GPUs. Overall, the combination of these distributed training strategies has enabled the scaling of LLMs to sizes that were previously infeasible, allowing for the exploration of emergent behaviors and capabilities.

HPC strategies enable the exploration of the biological questions at the heart of this proposal. At each stage of the proposal, complex interactions increase, demanding computational techniques that scale accordingly. The segmentation models in Chapter 2 can be trained with a single node, while the transformer models in Chapter 3 required distributed data parallelism across multiple nodes. The computational requirements of Chapter 4 are even greater, requiring multi-dimensional parallelism to train models with hundreds of billions of parameters through an entire post-training pipeline. The ability to scale models to these sizes is critical to the success of this proposal, and the HPC strategies discussed in this section are essential to supporting scaling.

1.6 Deep Learning for Biology

While deep learning has achieved notable success in domains such as computer vision and natural language processing, its application to biological data remains

comparatively less mature. The preceding sections traced how each architecture encodes an inductive bias about its data: convolution assumes translation invariance and local structure, while attention assumes sequentiality and contextual dependence. Biological data satisfy these assumptions unevenly. At the phenomic layer, the vision architectures of Section 1.3 transfer plant images directly, but annotated label data is often expensive or tedious to collect. At the genomic layer, genotypes are ordered along a chromosome, but their predictive signal is largely additive and structurally sparse, offering a generic sequence model less to exploit than word order offers in text. At the systems level, biological networks are relational rather than sequential, and are far too large to fit within a context window at all. Unlike image or text modalities, biological data also exhibit hierarchical structure and long-range dependencies with limited or noisy supervision. These characteristics challenge many of the assumptions that underlie standard deep learning architectures and training paradigms, and the gains from deep learning are distributed unequally across omics layers.

The uneven transfer of deep learning across omics layers, summarized in Figure 1.5, organizes the remainder of this review. Section 1.6.1 covers vision-based phenotyping, where the alignment between data and architecture is strongest and the task is one of measurement. Section 1.6.2 covers genomic prediction, where linear models remain competitive and progress depends on shaping architectures to reflect the underlying genetic architecture. Section 1.6.3 covers network-based and language model reasoning over biological networks, where the structure of the data departs furthest from the modalities these methods were developed on. Each corresponds to one of the three research chapters that follow.

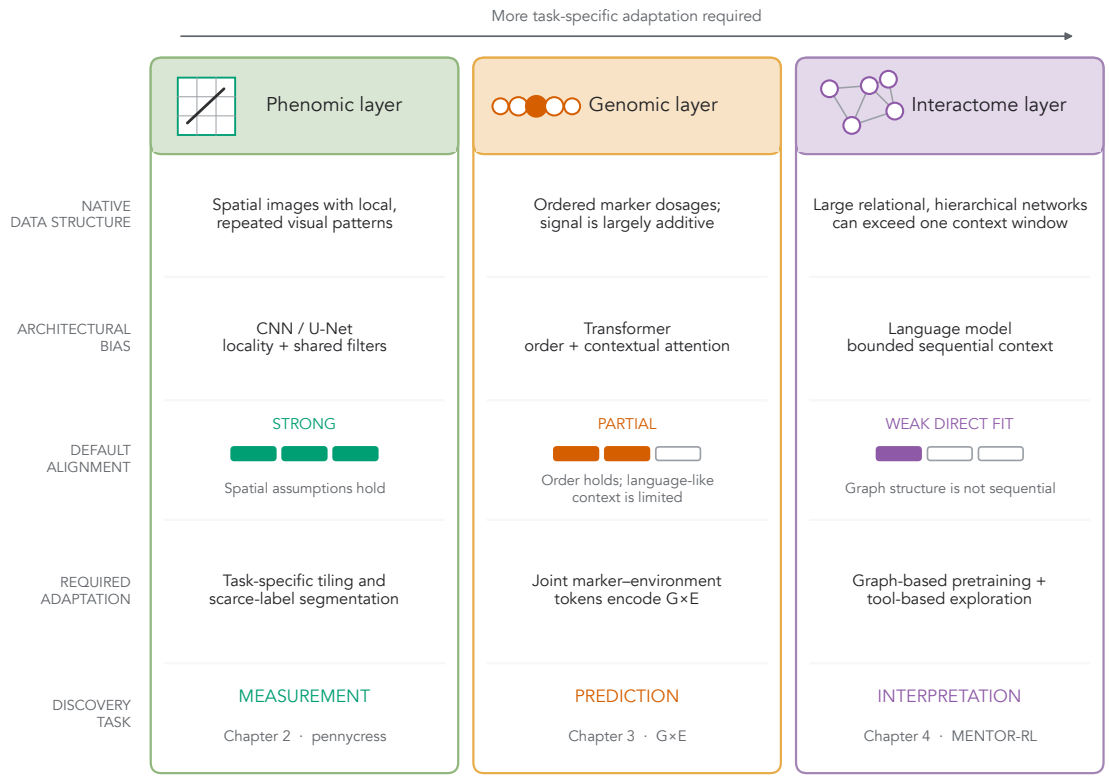

Figure 1.5: Alignment between biological data structure and architectural inductive bias across the three omics layers addressed in this proposal. The alignment row describes how well each layer satisfies an architecture's default assumptions before task-specific adaptation. Phenomic images strongly match the locality and shared-filter biases of CNNs. Ordered marker data partially match transformer sequence assumptions, motivating a joint marker–environment representation that exposes G×E interactions. Interactomes are relational and hierarchical, and large interactomes can exceed practical context windows, motivating graph-based pretraining and tool-based exploration. The columns connect these adaptations to measurement, prediction, and interpretation in Chapters 2–4.

1.6.1 Vision-Based Phenotyping

Phenotyping, or measuring an organism's observable traits, is a critical step in understanding biological mechanisms. In plant research, phenotyping enables large-scale breeding studies that can connect phenotypic traits to their underlying genetic drivers. As technology has advanced, it has become possible to collect phenotypic

data at scale, often with high-throughput imaging [114]. However, despite a wealth of raw image data, extracting ground truth labels, such as pixel-level measurements, has stayed a bottleneck for phenotyping pipelines due to its labor-intensive and time-consuming nature. Overcoming these challenges is crucial to unlocking the potential of high-throughput phenotyping and enabling more efficient and effective breeding programs.

Several early approaches used classical image analysis techniques to extract phenotypic traits from images. Tanabata et al. [196] developed SmartGrain, which binarized scanned seed images, traced the outline of each seed, and derived shape descriptors such as length, width, area, and perimeter from the extracted boundaries. Clohessy et al. [31] paired a low-cost imaging apparatus with an automated processing routine to measure the size and shape of single oat seeds as they passed the camera, raising throughput without raising labeling cost. However, these methods are limited by their reliance on hand-crafted features and collection under controlled imaging conditions, which may not generalize well to diverse datasets or collection scenarios. This presents an opportunity to learn feature representations, which may be more robust to natural variation and field settings.

Deep learning has been applied successfully to several phenotyping tasks, setting a precedent for its application to plant measurement. Mohanty et al. [133] treated disease diagnosis as an image classification problem, fine-tuning the ILSVRC architectures of Section 1.3.2 on 54,306 leaf images to distinguish 26 diseases across 14 crop species. Counting tasks require resolving individual organs rather than labeling the image as a whole: Uzal et al. [203] trained a CNN to estimate seeds per pod in soybean directly from pod images, and Ubbens et al. [201] addressed the cost of annotation itself, rendering synthetic rosettes from a procedural plant model and training on them to show that generated data could substitute for scarce labeled examples.

Several works have also connected high-throughput deep learning pipelines with downstream trait extraction. Lagergren et al. [106] used few-shot deep learning to

extract leaf phenotypes from images while training on a small number of labeled examples; extracted phenotypes were then connected to their genetic mechanism through a combination of GWAS and network-based methods, an approach we return to in Section 1.6.3. Wang et al. [211] scored wheat flowering time from repeated field imagery and carried the resulting measurements into a GWAS, recovering known flowering loci and showing that a deep learning measurement could stand in for a manually scored trait in genetic analysis.

The success of deep learning in phenotyping tasks has motivated its application to other species, all of which face similar obstacles. Annotation scarcity, domain shift, and evaluation protocols are all critical challenges that require a model's inductive biases to be tailored to the task at hand. Small annotated corpora and the small images tiles favor architectures with built in locality and weight sharing. In Chapter 2, we discuss our work building a high-throughput phenotyping pipeline for pennycress (*Thlaspi arvense*), a non-model oilseed crop, and the design decisions that are necessary to adapt deep learning methods to this species.

1.6.2 Deep Learning-Based Genotype-by-Environment Prediction

Genomic prediction is the task of estimating an individual's breeding value from its genotype before the trait has been observed [34]. Genomic prediction shortens the breeding cycle because it does not require measuring every candidate: once a model has been trained on a reference population that is both genotyped and phenotyped, untested individuals can be ranked from marker data alone. The traits that matter most, such as yield, are quantitative and polygenic, arising from many loci of small effect. Prediction therefore operates over dense marker panels rather than individual causal variants. The expression of phenotypic traits is not solely dependent on genotype, either; traits vary based on how genetics change under different environmental conditions. Modeling this *genotype-by-environment* (G×E)

interaction is therefore central to the task, and it is the aspect of genomic prediction this proposal addresses.

Genomic prediction has traditionally relied on statistical models to estimate breeding values from genotypic data. Meuwissen et al. introduced genomic selection [130], fitting linear models with shrinkage to dense single nucleotide polymorphism (SNP) marker data to predict breeding values. VanRaden later formalized the genomic best linear unbiased predictor (GBLUP) by deriving a genomic relationship matrix from marker data and substituting it for the pedigree relationship matrix in the mixed model equations [204], improving predictive accuracy. However, these methods are limited by their linearity assumptions and inability to capture complex interactions between genetic variants.

Jarquin et al. introduced the reaction norm model [90] in 2014, which extended the GBLUP framework to account for G×E interactions using genomic, environment, and G×E kernels. This model improved predictive accuracy in multi-environment trials, but the interaction structure still had to be specified a priori through the choice of kernel.

Deep learning offers a contrasting approach. Because the relationships between genetic variants, environmental variables, and phenotypic traits are nonlinear, deep learning should be well suited to genomic prediction. Early work applied MLPs and CNNs to the task, such as Bellot et al. [11] and Khaki and Wang [97]. However, these methods have yet to consistently outperform linear models [136, 4]. One explanation is that linear models such as GBLUP are already well matched to the largely additive genetic architecture of many complex traits [69], leaving deep learning models with few opportunities to improve performance. Additionally, marker inputs are typically encoded as allele dosages at each locus, offering little for a deep model to exploit beyond the additive signal a linear model already captures [11]. Recent work, however, has shown that shaping the inductive biases of deep learning models to better reflect the underlying biology can improve performance.

Inspired by the success of sequence models for language modeling, several recent works have applied transformer architectures to genomic modeling. Broadly, these works fall into two categories: reference-sequence foundation models and genotype-matrix models over individuals. Reference-sequence foundation models pretrain on genomic sequences, learning general representations of genomic data, and are then evaluated on functional or regulatory tasks. DNABERT [91] and DNABERT-2 [228] each trained a bidirectional transformer to learn genomic sequence representations through masked language modeling. The Nucleotide Transformer [37] scaled this approach across human and multi-species genomes, and Caduceus [172] replaced attention with a bidirectional state-space backbone that respects reverse-complement symmetry, extending the sequence context these models can process. These models therefore learn general sequence representations instead of predicting phenotypes across a population of individuals.

The second category consists of genotype-matrix models, which operate over the variant genotypes of many individuals and therefore align more directly with the genomic prediction task. SNP2Vec [21] adapted subword tokenization to diploid variant sequences, pretraining representations of genomic variation for downstream association tasks, while SNVformer [46] applied an efficient attention variant to single nucleotide variant data, treating association analysis as a supervised sequence problem. GPformer [218] embedded markers with a convolutional block before a transformer encoder and regression head, coupling the architecture with a knowledge-guided module to predict quantitative phenotypes. Most relevant to this proposal, GEFormer [223] targeted G×E prediction directly, encoding markers with a gated multilayer perceptron and daily environmental measurements with dynamic convolution and linear attention, then combining the two representations through a gating mechanism. This branch-based design is characteristic of multimodal genomic prediction models, in which marker–environment interactions are typically fused after each modality has been encoded independently.

In recent years, several efforts have standardized genomic prediction and selection research under unified benchmarks. Notably, in the plant research community, the Genomes to Fields (G2F) Maize Prediction Competition [24] has provided a benchmark for evaluating G×E prediction methods, including deep learning approaches. This large, multi-modal dataset and competition structure provide a testbed that is analogous to the role of the ILSVRC in computer vision (Section 1.3.2). However, unlike the ILSVRC, the G2F competition has not yet produced a breakthrough result. In Chapter 3, we discuss our efforts using deep learning for G×E prediction on this dataset.

1.6.3 Mechanistic Interpretation of Biological Networks

Mechanistic interpretation is the process of explaining how biological function arises from the interactions among molecular entities, rather than cataloging those entities individually [61, 8]. The object of interpretation is therefore not a single sequence or measurement, but the *interactome*: the set of physical, regulatory, and functional relationships that connect genes, proteins, and metabolites across an organism. These relationships are naturally represented as networks, which are hierarchically structured. Individual interactions aggregate into functional modules, modules participate in pathways, and pathways combine into the higher-order processes that produce a phenotype [7, 128]. Furthermore, interactions can scale across cell types, tissue regions, and entire tissues. Interpretation operates across these scales, since the meaning of a single edge depends on the module and pathway context around it [84]. These structures are far larger than any researcher can examine exhaustively, which motivates computational approaches to interpretation itself.

Efforts to connect measured phenotypes to mechanism have traditionally started with GWAS, which statistically tests loci across the genome for significant associations with a trait [202]. However, its utility for mechanistic interpretation is limited in three ways. GWAS returns significant loci, not genes, so an intermediate step

must be performed linking the two. Additionally, linkage disequilibrium inflates each associated interval, so significant markers may not be causal. Finally, the resulting candidates returned by GWAS are unranked, making it hard to prioritize them. Connecting phenotypic traits to mechanisms therefore requires additional methods.

Using network-based tools can help close the gap between observation and interpretation. Networks make modularity explicit and allow multiple lines of evidence to be collated in a single multiplex structure. Furthermore, predictive expression networks (PENs) and predictive phenotype networks (PPNs) can be constructed from data using methods such as iRF-LOOP [30, 9], which applies iterative random forest in a leave-one-out scheme to build predictive relationships between variables.

Given such a network, propagation methods can convert a set of seed genes into a ranked neighborhood. Random walk with restart (RWR) [199] was first applied to candidate disease gene prioritization [103, 205] and has since been characterized as a general amplifier of weak genetic association signal [33], addressing the deficit of GWAS. RWRtoolkit extends diffusion methods to multiplex networks in any species [93]. Propagation alone, however, returns a ranked list with no measure of statistical significance. GRIN adds a null model, built from randomly sampled seed sets of equal size, and refines seed gene sets by retaining only those whose network proximity is unlikely to have arisen by chance [190].

While diffusion-based methods are effective for exploring networks, specifically at the level of local topology, they fail to provide a top-down, hierarchical representation of network structure. To account for this limitation, tools such as MENTOR [191] convert RWR embeddings into a dendrogram of coherent modules through dissimilarity scoring and agglomerative clustering. These modules improve the interpretability of results by incorporating hierarchical relationships sourced from multiple layers of data. However, this methodology presents a new bottleneck further down the interpretation pipeline. While modules are connected through a deterministic and traceable process, mechanistic discovery still requires reading

supporting literature, weighing conflicting evidence, and synthesizing it to explain why a module coheres. While this work can be accomplished through human collaboration, language models present an opportunity to assist humans in large-scale interpretation tasks through mechanistic reasoning.

Before a language model can reason over a network, however, it must be able to represent graph structure at all. Evidence that transformers can internalize structured state comes from OthelloGPT [113], in which a model trained only to predict legal moves in a board game developed an internal representation of the board, despite never observing one directly. This result suggests that supervised training on sequences generated by a structured process can capture a representation of the underlying structure. For graphs specifically, Fatemi et al. [48] showed that language model performance on graph reasoning tasks depends substantially on how the graph is serialized into text, and that the encoding choice can matter as much as model scale. Together, these results motivate a supervised fine-tuning stage in which a model learns graph rules and operations directly from serialized structure.

Mobilizing learned representations for interpretation, however, requires effective use of test time compute. Chain-of-thought prompting [215] showed that eliciting intermediate reasoning steps substantially improves performance on multi-step problems, establishing that useful computation can be externalized at test time. Tree-of-thoughts [222] generalized this from a single linear trace to a search over a tree of candidate thoughts, with explicit branching, evaluation, and backtracking. This formulation is directly relevant to network exploration, where several starting points may be plausible and unproductive branches should be abandoned early. ReAct [221] extended reasoning traces to include actions, allowing a model to query external sources and condition subsequent reasoning on what it retrieved, while Toolformer [171] demonstrated that models could learn when to invoke external tools in a self-supervised manner. Such policies are trained with the post-training methods described in Section 1.4.3. This framing is well suited to mechanistic interpretation, where proposed mechanisms are only useful if they can be traced to a source. A

model that retrieves evidence from a network can supply provenance, while one that relies purely on recall cannot.

Several challenges remain when language models are applied to biological networks. Multiplex networks are typically too large to fit within a model's context window, so exploration requires multiple steps. Because of this multi-hop reasoning process, errors compound across a trajectory, and an incorrect early retrieval can misdirect every step that follows. Unlike mathematics and code, where correctness is cheaply verifiable, there is no ground truth for a correct mechanistic narrative, so reward must be constructed from structural properties of the exploration rather than from a single correct answer.

These constraints also shape how the graph should reach the model. One approach encodes graph structure with a learned graph neural network and injects the resulting representation into the model's context [66]. Applied to biological networks, this approach faces two practical obstacles. First, it requires trainable graph data, which is considerably harder to obtain than text; curated interaction data is expensive to produce and covers only a small fraction of the interactome. Second, graph neural networks [101, 207] are difficult to scale relative to transformers, because neighborhood aggregation is irregular and does not permit the dense, uniform parallelism that the training strategies in Section 1.5.2 depend on. A learned graph encoder also compresses structure into a representation that cannot be traced, which is a liability when the output is a mechanistic claim.

Several strategies exist that circumvent the limitations of neural graph encoders. Supervised fine-tuning can teach the model general graph structure and operations, unifying graph representations with language space so that the model's reasoning about the network remains expressible in text, and therefore interpretable. Reinforcement learning methods [156, 179] can post-train the model on reasoning and exploration tasks, teaching multi-hop search in a graph environment. Throughout, the graph is reached with the same deterministic operators described above: RWR, MENTOR, shortest-path search, and others. These operations return provenance

alongside each result, so claims can be traced back to the graph evidence that produced them.

Recent systems have begun to incorporate these strategies. Eubiota [123] and MENTOR-IA [208] demonstrated that provenance-preserving retrieval and tool-mediated synthesis can produce high-fidelity mechanistic narratives over biological networks, building on the MENTOR module structures described above. However, both systems depend on frontier model inference, which limits deployment at the exploration depths that large networks demand, and neither is optimized as a trainable policy. In Chapter 4, we discuss our efforts to train an open-source reasoning agent for mechanistic exploration of biological networks with reinforcement learning.

1.7 Conclusion

The preceding sections follow a consistent pattern: progress in deep learning has followed from matching an architecture's inductive biases to the structure of its data. Convolution succeeded on images because translation invariance and local connectivity hold for images, and attention succeeded on language because sequentiality and contextual dependence hold for text. In vision, a standardized benchmark concentrated effort on a common task, and the scaling behavior described in Section 1.5.1 converted architectural fit into performance once sufficient compute and data were available.

Biological data do not present a single structure, so this pattern does not transfer uniformly, and the three omics layers this proposal addresses sit at different points along the gradient summarized in Figure 1.5. The phenomic layer inherits the alignment directly; plant images are images, and the segmentation architectures of Sections 1.3.3 and 1.3.4 apply to them. Which of those architectures to choose remains a question of inductive bias, however, because scarce annotation and small image tiles reward a strong spatial prior over one learned from data at scale. The genomic layer sits further away. Markers are ordered along a chromosome but encoded as allele

dosages, and the signal they carry is largely additive, which is why linear models remain competitive and why architectural gains depend on building the structure of the problem, particularly genotype-by-environment interaction, into the model rather than expecting a generic sequence model to discover it. The systems layer, over which mechanistic interpretation operates, sits furthest away still. The data are relational, not sequential, exceed any context window, and admit no single correct answer against which a model can be scored.

These three layers correspond to the three stages of biological discovery this proposal addresses. Chapter 2 addresses *measurement*, extracting quantitative phenotypes from raw imagery. Chapter 3 addresses *prediction*, mapping genotype and environment to phenotype. Chapter 4 addresses *interpretation*, reasoning over biological networks to produce evidence-grounded mechanistic hypotheses. In each case the contribution is not a general advance in deep learning, but a system whose architecture, objective, and benchmark are shaped to the structure of the omics layer it addresses.

Chapter 2

A Deep Learning Pipeline for Pennycress Seed Pod

2.1 Introduction

Pennycress (*Thlaspi arvense L.*) is a winter annual in the *Brassicaceae* family that has shown potential as a bioenergy feedstock and cover crop due to its winter-annual habit, growth in the fallow period, and amenability to genetic modification [137, 176]. Its short generation time, established transformation protocols, sequenced reference genome [143] and species-wide pangenome [16] make pennycress a viable candidate for breeding and domestication [176]. However, domesticating pennycress requires improving yield, seed retention, and cover-crop/feedstock traits [176]. In *Brassicaceae*, silicle morphology is directly tied to seed yield and pod shattering [142]. Pods play an active role in development and resource allocation, not merely containing seeds [13]. Pod walls are photosynthetically active and supply assimilate to the developing seed [76, 229]. Seeds per pod is one of three multiplicative components of seed yield, alongside pods per plant and seed weight, and in rapeseed its natural variation is under maternal and embryonic genetic control [230]. Therefore, tissue-specific pod

traits such as wing area, envelope area, seed count, and tissue-to-seed ratios are biologically meaningful targets for breeding and phenotyping.

However, realizing the potential of pennycress as a biofuel feedstock requires population-scale studies that characterize variation in feedstock-relevant traits, which are currently limited by phenotyping bottlenecks, most notably the manual collection of pod-level traits [114, 203]. Although high-throughput methods exist for measuring seed composition [200], single-seed weight [31], and seed shape [196], each operates on individual seeds and extracts a limited set of traits. These techniques do not resolve distinct pod tissues (wing, envelope, and seed) or support the depth of morphological and inter-tissue measurements that pod-level phenotyping requires. Despite their relevance to pennycress improvement, pod traits have not been measured at tissue resolution across large pennycress panels.

The central bottleneck to measuring pod traits is tissue segmentation, which involves separating wing, envelope, and seed regions, and counting seeds in each pod [203]. This process is labor-intensive because each pod requires boundary tracing, object separation, and visual inspection [114]. Consequently, manual measurement limits both sample size and trait depth, forcing researchers to measure either fewer accessions or fewer traits [203]. As a result, detailed pod traits are systematically omitted from population-scale studies. Automated segmentation addresses this bottleneck by converting raw images into tissue-specific masks that can be used for scalable trait extraction [114, 106].

In other domains, image-based phenotyping has enabled measurements at scales and resolutions that are infeasible by hand, from cell-level microscopy to satellite imagery [114, 106]. A single imaging modality can yield many traits simultaneously, allowing researchers to extract complex topological and morphological features that are inaccessible with manual measurement [114]. For pod-level phenotyping, the operative task is segmentation, which separates wing, envelope, and seed regions within each pod. Classical segmentation approaches require controlled imaging

conditions and manually tuned thresholds, which do not generalize across natural variation in raw scans.

Unlike classical segmentation, deep learning models operate on raw images directly, learning relevant features end-to-end; deep learning has achieved state-of-the-art performance across computer vision tasks [105, 62]. In the plant phenotyping domain, deep learning models have been applied to disease detection [133], organ counting [201], seed-per-pod estimation [203], and population-scale trait extraction [211, 106]. In particular, convolutional neural networks (CNNs) learn hierarchical filters over local image neighborhoods [109], giving them a spatial inductive bias well suited to segmentation. U-Net is a CNN-based segmentation architecture that shows strong performance on small, specialized biological and biomedical segmentation tasks, making it a natural baseline for tissue-level pod segmentation [165]. More recent architectures can improve performance in some settings but may require larger datasets, different preprocessing, or task-specific adaptation. Because pennycress pod segmentation is a small, specialized biological task with fine seed/envelope boundaries, benchmarking modern architectures against U-Net is necessary.

Once tissue-resolved pod traits can be measured at scale, the next challenge is linking trait variation to candidate genetic mechanisms. High-throughput phenotyping can be used in conjunction with genome-wide association studies (GWAS) [202] to reveal the genetic architecture of phenotypes, providing candidate genes and markers for breeding. A network-based framework that leverages multiple lines of evidence in pennycress or related species utilizes GRIN (Gene set Refinement through Interacting Networks) [190], MENTOR (Multiplex Embedding of Networks for Team-Based Omics Research) [191], and MENTOR-IA (MENTOR Interpretation Agent) [208] to prioritize these candidates and resolve them into interpretable modules, yielding candidate mechanisms. This framework has been applied to other pennycress traits [32], but not to intensive, tissue-resolved pod phenotypes in pennycress.

In this work, we developed a deep learning pipeline for pennycress seed pod segmentation, enabling deep, tissue-resolved phenotyping at population scale. We

benchmarked a suite of modern computer vision models, spanning CNNs, vision transformers, and foundation models. We found that a custom U-Net provided the most efficient pod tissue segmentation performance with comparable accuracy to modern models and a 30- to 45-fold speed increase over manual phenotyping. We applied this pipeline at population scale to over 12,000 pods from 768 accessions and used it to identify candidate loci, genes, and mechanisms underlying pod traits.

2.2 Materials and Methods

2.2.1 Experimental Design

We designed this study to explore whether it was possible to use deep learning to accelerate image-based pennycress pod phenotyping and connect the extracted phenotypes to candidate genetic mechanisms for pennycress improvement. Following this objective, our study contains two main components. In the first stage, we trained and benchmarked a U-Net segmentation model to extract 52 per-pod phenotypes, applied the model at population scale to 12,253 pods, and examined the resulting trait matrix for biological coherence with a predictive phenotype network. In the second stage, we filtered traits by heritability ($H^2 \geq 0.05$), connected them to loci via GWAS (significant at $p < 1 \times 10^{-6}$), and mapped loci to candidate genes. We then pruned the candidates with GRIN and resolved the survivors into modules with MENTOR. With the assistance of MENTOR-IA, we interpreted the modules to generate exploratory mechanistic hypotheses for genetic targets. Our data were sourced from a 768-accession panel of pennycress pod images, collected from field sites in the midwestern United States. Broad-sense heritability was estimated on $n = 25$ replicated genotypes, and GWAS was performed on a subset of 189 genotyped accessions from dryland environment treatments.

2.2.2 Dataset

2.2.2.1 Data Collection

Our raw data consisted of 768 scanned images of pennycress (*Thlaspi arvense L.*) seed pods collected from field sites in the Midwest, including those at Western Illinois University (WIU), Illinois State University (ISU), and University of Minnesota (UM). After collecting seed pods, we imaged them using an Epson V39 Perfection scanner. Each scanned image contained multiple seed pods due to their size. Figure 2.1A displays an example raw image scan. The 768 scans represented 768 pennycress accessions, with each scan typically containing approximately 10–20 seed pods.

2.2.2.2 Data Annotation

For supervised model training, we created pixel-level annotations using GNU Image Manipulation Program (GIMP). We labeled three areas of each pennycress seed pod: the wing was annotated with red pixels, the envelope was annotated with green pixels, and the seeds were annotated with blue pixels. We annotated these areas due to the biological significance of their phenotypes. Figure 2.1B and C show an example of a separated pennycress seed pod and its associated segmentation. We manually annotated 21 scanned images, amounting to 347 annotated seed pods. We assigned 16 scans containing 282 pod crops to the development set and reserved 5 scans containing 65 pod crops as an independent test set used only for final benchmarking. Within the development set, we randomly allocated the 282 pod crops in an 80:20 ratio to training and validation subsets, respectively. The validation subset was used for checkpoint selection and early stopping; the independent test set was not used during training or checkpoint selection and was evaluated only for the final architecture benchmark.

2.2.2.3 Data Preprocessing

Before model input, we separated each pennycress seed pod in the raw image. We created rectangular bounding boxes around each seed pod object in Python, and we

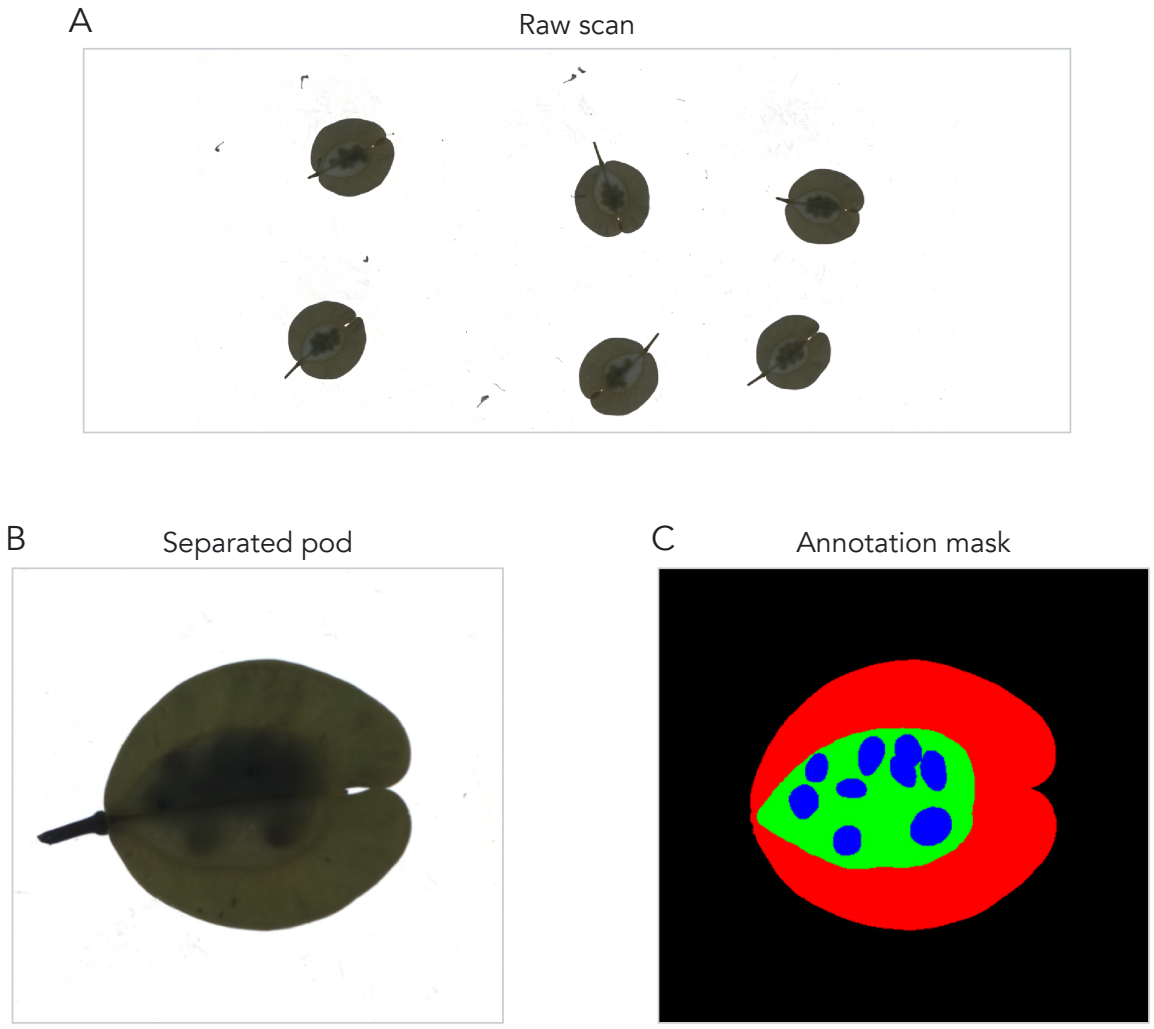

Figure 2.1: Raw pennycress seed-pod scan, separated pod crop, and pixel-level annotation. (A) Flatbed scan excerpt showing multiple *Thlaspi arvense* seed pods on a white scanner background before object separation, annotation, and segmentation preprocessing. (B) Separated *T. arvense* seed pod extracted from a raw scan. (C) Corresponding semantic annotation mask, with wing tissue in red, envelope tissue in green, seeds in blue, and background in black.

saved each bounding box as a separate image, yielding a dataset of 12,253 individual pods across the 768 scans. We used these separated images for model input.

2.2.2.4 Image Tile Generation

We implemented a tile generation scheme to increase the amount of training data available and build an invariance to common imperfections that occur during the image collection process. Tiling is a form of image augmentation that divides each image into small, fixed-size patches that serve as training examples.

Prior to tiling, we padded each separated image with 128 pixels in all directions. Image padding enables us to generate tiles near the edge of images, allowing the model to see a greater context. The red box in the leftmost image in Figure 2.2 highlights the padding boundary, and the remaining panels show the corresponding semantic and class-specific masks used during tile sampling. While we also used a tile size of 128 pixels, the padding and tile size are independent parameters.

After padding, we selected a tile size of 128 pixels. Pixels on the seed pod were randomly initialized as tile centers. After the algorithm selected center pixels, it used the surrounding 128 $\times$ 128 pixels to create each tile.

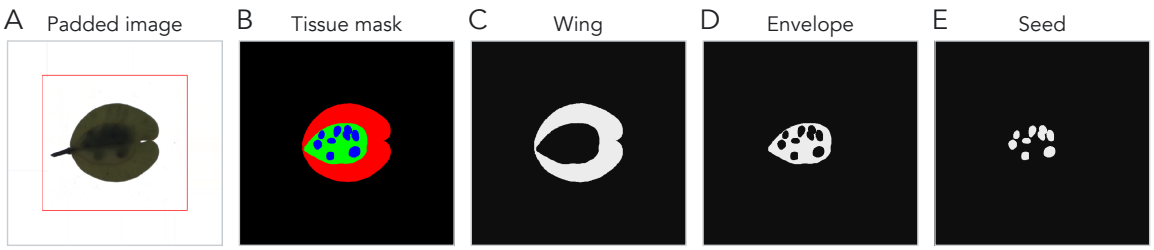

Figure 2.2: Preprocessing outputs for model training. (A) Padded separated pod image; the red rectangle marks the original crop boundary after adding 128 pixels of context on each side. (B) Full semantic tissue mask. (C–E) Binary masks for wing, envelope, and seed classes, respectively.

During validation, we left tiles unchanged to prompt the most accurate predictions possible. However, during training, we applied various image augmentations and transformations to build an invariance to common data collection issues. We applied rotation, blur, saturation, brightness, flips, and transposes with a non-exclusive 50% chance of occurrence each, greatly increasing the number of unique types of tiles that could be generated.

We selected each type of transformation to combat certain issues that occur during the scanning process. For example, blur transformations simulate potential smudges on the scanner screen. Rotations model the differences in angle between scanned seed pods, and brightness transformations capture the potential for shadows underneath folds in an image. By using these transformations to simulate real imaging issues, the model learns to become invariant to such features and instead focus on biologically relevant information. Therefore, the tile generation scheme serves two purposes. It increases the amount of available training data, and it builds an invariance to common scanning issues. We show examples of generated training tiles, their corresponding labels, and representative augmentation outputs in Figure 2.3.

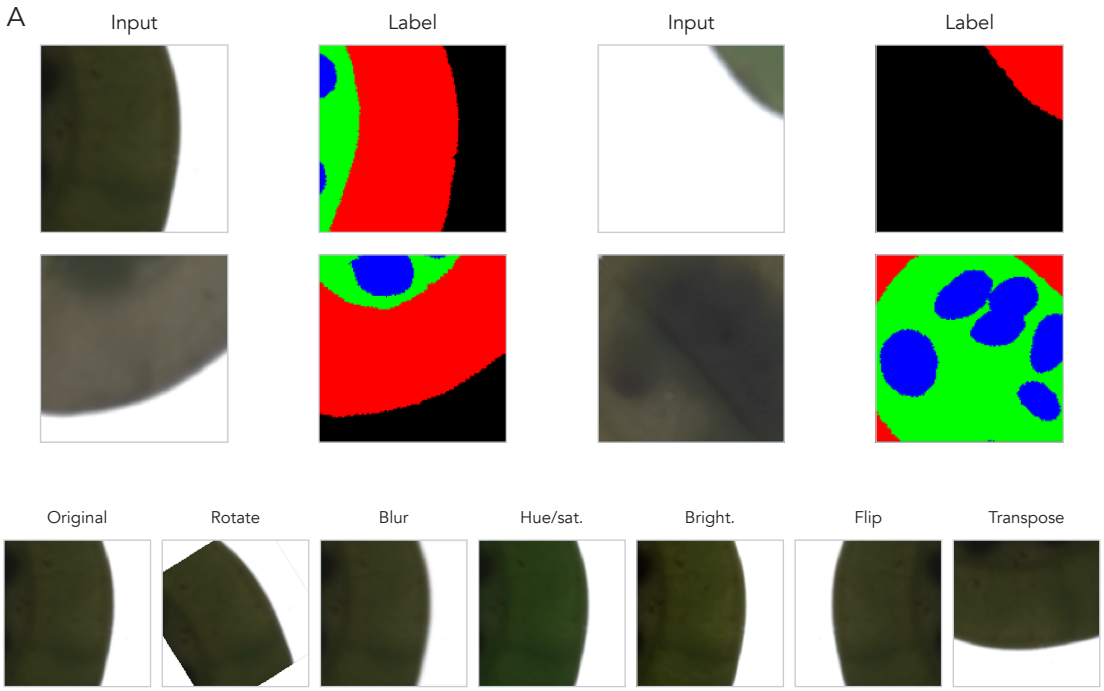

Figure 2.3: Example training tiles and augmentation outputs. (A) Input and label pairs show 128×128 tiles sampled from padded seed-pod images and their corresponding tissue-label masks. Tiles include background, pod-boundary, tissue, and seed contexts used for model training. (B) Representative image-tile augmentations show the transformation families applied during training, including affine rotation, blur, hue/saturation shift, brightness/contrast adjustment, flip, and transpose operations.

2.2.3 U-Net Baseline

2.2.3.1 Architecture

We used U-Net [165], a fully convolutional encoder-decoder architecture in which a contractive path captures hierarchical context and a symmetric, expansive path recovers spatial localization, as a segmentation baseline. We adapted the original architecture with a modified block structure, SELU activations, and tuned hyperparameters for tissue-level pod segmentation.

Because raw seed pod images were too large and computationally expensive to fit into model memory while training, we used the generated tiles created in Section 2.2.2.4 for training. The model's input is a 3-channel RGB tile of size 128 × 128, and its output is also 128 × 128 with four channels corresponding to background, wing, envelope, and seed classes. We applied a softmax activation function to create a pixel-wise segmentation map, where the model gives each pixel a class label. We then compared segmentation maps to the corresponding hand-annotated segmentations detailed in Section 2.2.2.2 and minimized the weighted cross-entropy loss between them to train the model.

The main component in the U-Net architecture is a convolutional block. Each block contains three convolutional layers with 3 × 3 kernels; each convolution is followed by batch normalization, SELU activation, and dropout with a rate of 0.1. The network comprises nine convolutional blocks (Figure 2.4), beginning with a 32-kernel block at the full 128×128 tile resolution. The four contractive blocks are separated by 2×2 max-pooling that halves spatial resolution while increasing the channel depth from 32 to 256. A 512-kernel bottleneck operates at 8×8 resolution. The four expansive blocks reverse this with upsampling-convolution steps that restore spatial resolution and reduce channel depth, returning to 128×128. Skip connections concatenate contractive and expansive feature maps, improving gradient flow and fusing features across scales. A final 1×1 convolution produces the four-channel segmentation map.

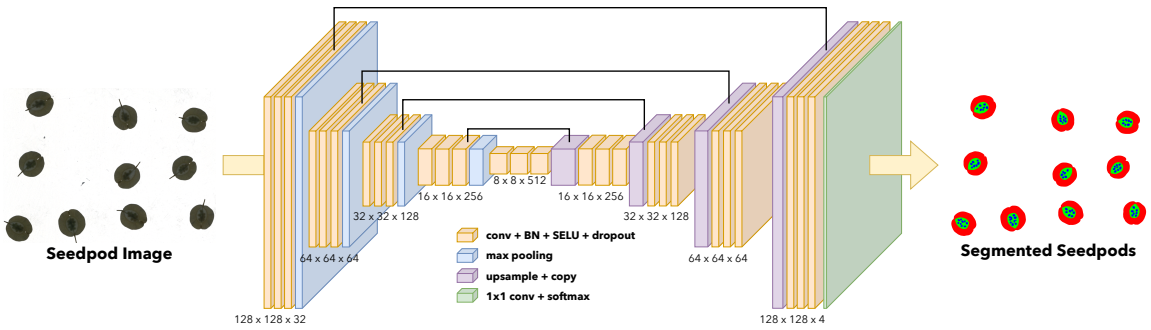

Figure 2.4: Pennycress U-Net architecture. The model takes a 128×128 RGB seed pod tile as input and returns a 128×128 four-channel semantic segmentation map for background, wing, envelope, and seed classes. Orange blocks denote 3×3 convolution, batch normalization (BN), SELU activation, and dropout; blue blocks denote max pooling; purple blocks denote upsampling with skip-copy connections; and the green block denotes the final 1×1 convolution with softmax output. Feature-map labels are shown as height × width × channels.

2.2.3.2 Boundary-Weighted Cross Entropy Loss

To account for uncertainty in the exact boundaries of our ground-truth labels, we implemented a weighted cross-entropy loss that scales the per-pixel loss by a boundary-aware weight map.

We first compute a raw weight map from an eroded seed mask. We binarize the seed channel, subtract its binary erosion to isolate the boundary ring between envelope and seed, and then calculate the Euclidean distance from each pixel to the closest border pixel to produce a distance map. We clip distances at a maximum of $b = 5$ pixels and normalize to the range $[0, 1]$. We evaluated two configurations parameterized by a border weight w_b.

In the **boundary down-weighted** configuration ($w_b < 1$), we min-max normalize the raw weight map so that pixel values $W_{i,j} \in [0, 1]$, where i and j index the row and column of a pixel within a tile. We then invert the weight map so that pixels with a smaller distance to the boundary have a lower weight value. Finally, we scale the weight map so that its range is $W_{i,j} \in [w_b, 1]$. This configuration treats boundary annotations as the noisiest signal and lets the model rely more on confidently-labeled interior pixels.

In the **boundary up-weighted** configuration ($w_b > 1$), we instead clip the raw weight at a minimum of 1, yielding $W_{i,j} \in [1, w_b]$ so that distant pixels carry weight 1 and boundary pixels carry weight up to w_b. This configuration treats boundaries as the most informative regions to learn.

We then multiply the weight map pixelwise with cross-entropy loss and average over pixels:

$$L_{WCE}(t, p(\theta)) = -\frac{1}{\mid I \mid\mid J \mid} \sum_{j}^{J} \sum_{i}^{I} \sum_{k}^{K} W_{i,j}(t_{i,j,k} \log(p(\theta)_{i,j,k})), \tag{2.1}$$

where t is the ground truth segmentation map and $p(\theta)$ is the predicted segmentation map as a function of U-Net parameters. The indices i and j run over the I rows and J columns of a tile, so $\mid I \mid\mid J \mid$ is the number of pixels the loss is averaged over, and k runs over the $K = 4$ classes (background, wing, envelope, and seed). For a given pixel, $t_{i,j,k}$ is 1 if that pixel is labeled class k in the ground truth and 0 otherwise, while $p(\theta)_{i,j,k}$ is the softmax probability the model assigns to class k at that pixel. The inner sum over k therefore reduces to the log-probability the model placed on the correct class, and $W_{i,j}$ rescales that pixel's contribution according to its distance from a seed boundary.

2.2.3.3 Model Training

We trained the U-Net for up to 150,000 mini-batch updates using the Adam optimizer with a batch size of 32. Rather than partitioning training into fixed epochs over the tile dataset, we sampled tiles continuously from the train split (Section 2.2.2) and tracked progress by iteration count. We evaluated validation loss every 1,000 batches and applied an early stopping criterion of 50 evaluation intervals. If the best validation loss did not improve over 50 consecutive checkpoints (50,000 batches), training was terminated.

We used a learning rate schedule with linear warmup and cosine decay. The learning rate ramped from 0 to 1×10^{-3} over the first 1,000 batches to stabilize early gradients, then decayed via a cosine schedule to 1×10^{-5} over the subsequent 90,000 iterations and held constant thereafter.

2.2.4 Phenotype Extraction

We developed functions to extract 52 features from each seed pod segmentation map, grouped into three categories: absolute, ratio, or color.

2.2.4.1 Absolute Features

We calculated seven absolute features from each segmentation map: wing area and perimeter, envelope area and perimeter, seed area and perimeter, and seed count. Areas for each tissue were calculated by summing the pixels assigned to that class. Perimeters were calculated using Scikit-Image [209] on nested binary masks. Wing perimeter was calculated from the union of wing, envelope, and seed pixels, envelope perimeter was calculated from the union of envelope and seed pixels, and seed perimeter was calculated on the raw (pre-watershed) seed mask, giving the total boundary length of all seed pixels in the pod. Pixel counts were converted to physical area by dividing by DPI^2 ($600^2 = 360,000$) and multiplying by 6.4516 cm^2/in^2.

In addition to measuring area and perimeter features, we utilized the watershed algorithm to count the number of seeds in each segmented image. First, we isolated the seed channel and applied a morphological opening to suppress noise. Then, we calculated the Euclidean distance from each seed pixel to the nearest non-seed pixel. We applied a threshold to this distance map, setting any pixel that was less than 60% of the maximum distance value in the map to 0. The thresholded foreground was labeled into connected components and passed as markers to the OpenCV [19] watershed algorithm, which returned a per-seed segmentation; seed count is defined

as the number of unique markers in the map. The watershed process is visualized in Figure 2.5.

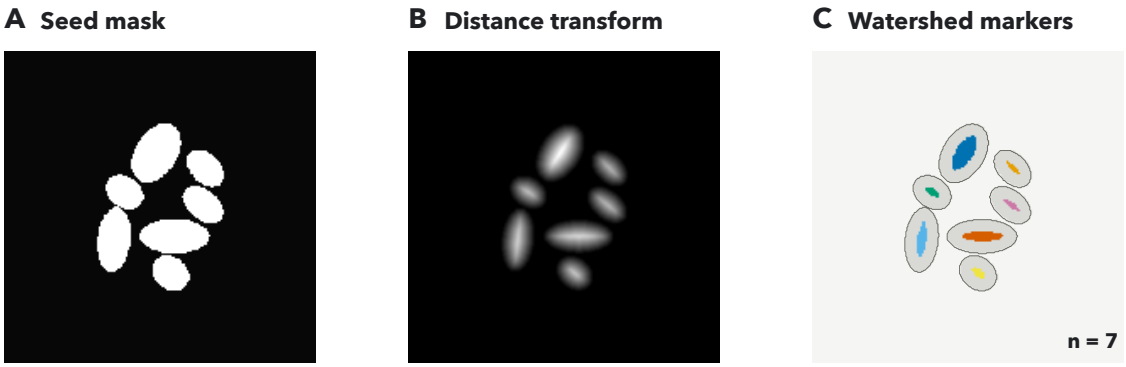

Figure 2.5: Seed-count extraction by watershed segmentation. (A) Binary seed mask isolated from the U-Net segmentation map. (B) Euclidean distance transform used to identify seed interiors. (C) Connected-component markers after thresholding the distance map at 60% of its maximum value; colors denote marker identities used to initialize watershed segmentation, and the resulting marker count defines seed count.

2.2.4.2 Ratio Features

We calculated 18 ratio features from the segmented seed pod masks, with six to-total ratios and twelve between-tissue ratios, evenly split between area and perimeter-based variants. The six to-total ratios divide each tissue's area or perimeter by the total area or perimeter, yielding wing-to-total, envelope-to-total, and seed-to-total ratios for both measurement types.

The 12 between-tissue ratios cover all pairs of distinct tissues for both area and perimeter. These include ratios into seed (envelope/seed, wing/seed), into envelope (wing/envelope, seed/envelope), and into wing (seed/wing, envelope/wing), computed for both measurement types. Reciprocal ratios are retained (e.g., seed/envelope and envelope/seed) so that downstream analysis is not restricted.

2.2.4.3 Color Features

We extracted nine color features from each tissue: red, green, blue (RGB); hue, saturation, brightness (HSV); and lightness, a^* (green-red), and b^* (blue-yellow) from CIELAB. We converted each input image into all three color spaces using OpenCV [19]. We then created a tensor of these features with size $\texttt{height} \times \texttt{width} \times 9$. We used the segmentation masks to subset this tensor by tissue, yielding three sub-tensors covering wing, envelope, and seed features separately.

We aggregated each color channel across spatial dimensions by taking the mean over the relevant tissue's pixels, producing a nine-dimensional color vector per tissue. Across the three tissues, this yielded 27 features in total.

2.2.5 Segmentation Architecture Benchmarking

To ensure that our pipeline extracts the most accurate downstream phenotypes possible, we identified the most effective segmentation architecture. We tested six architectures across three families: CNNs, transformers, and foundation models.

We compared two CNN architectures to the baseline U-Net: nnU-Net [87] and DeepLabV3+ [23]. nnU-Net is an approach that configures a U-Net systematically through a three-stage pipeline. The selection process includes fixed parameters that generalize across many datasets; rule-based parameters that are determined by unique dataset and pipeline characteristics; and empirical parameters that are learned through trial and error. Because nnU-Net derives its own patching, batch size, and augmentation scheme through fingerprinting and rule-based selection, the benchmarking for this architecture did not include our custom tile generation scheme (Section 2.2.2.4). DeepLabV3+ combines an encoder-decoder architecture with atrous depthwise separable convolutions, balancing the use of long-range spatial context with fine-grained boundary detection.

We selected SegFormer [220] and Mask2Former [28] as exemplar transformer architectures for benchmarking. SegFormer uses a hierarchical transformer encoder backbone that generates both high-resolution, fine-grained features and low-resolution, coarse features, and a lightweight MLP decoder to fuse these features for the final semantic segmentation task. Unlike SegFormer, which optimizes a per-pixel classification objective, Mask2Former adopts an alternate paradigm based on set prediction. Mask2Former uses a multi-scale encoder backbone to extract features, which are then upsampled to high-resolution per-pixel embeddings with a pixel decoder. Additionally, Mask2Former utilizes a transformer decoder with learnable query inputs to predict mask candidates. These queries are refined through cross-attention with the multi-scale features, and the transformer decoder output and pixel decoder output are combined via dot product to produce region predictions. Although Mask2Former is designed to be a general segmentation architecture, supporting panoptic, instance, and semantic segmentation, we trained only for a semantic segmentation objective due to our task formulation.

We evaluated SAM 3 [22] as a candidate foundation model for benchmarking. SAM 3 is a foundation model designed for promptable concept segmentation, taking input as either text or image exemplars. SAM 3 was trained with a model-in-the-loop data engine on over 1 billion images, generating segmentation examples using human input while shifting towards more model reliance during the annotation process. Although SAM 3 is capable of segmenting video and using concept prompts, we focused on the benefits that its pretrained encoder provides, adopting an approach similar to ClassWise-SAM-Adapter [150] by using a lightweight convolutional mask decoder for classwise semantic segmentation. Furthermore, we applied Low-Rank Adaptation (LoRA) [75] to the image encoder to enable parameter-efficient fine-tuning and limit drift from pretrained representations.

We trained DeepLabV3+, SegFormer, and Mask2Former from default pretrained backbones with the same tile generation and preprocessing scheme that we used for our custom U-Net to ensure consistency. Since nnU-Net's data-aware preprocessing

approach is emphasized as one of its strengths, we did not apply our custom preprocessing pipeline to it, instead using nnU-Net's adaptive fingerprinting setup for preprocessing and model training. As a consequence, nnU-Net scored predictions under its own pipeline at the level of individual seed pod crops (65 crops), whereas all other architectures were evaluated on the five full images from the same test split. For comparability, we grouped nnU-Net crop-level predictions by their source full image before computing the per-image metrics reported below.

For all models except nnU-Net and Mask2Former, we trained variants with unweighted, up-weighted, and down-weighted boundary loss. Due to nnU-Net's adaptive pipeline and Mask2Former's mask-classification objective, boundary weighting was not compatible with these architectures. These model families provide coverage across architectures (CNN and transformer), training objectives (per-pixel prediction and set prediction), and learning scenarios (supervised pretraining and foundation model pretraining).

We used intersection over union (IoU), also known as the Jaccard index, as the primary evaluation metric for this project, following its established use as the standard measure for semantic segmentation benchmarks [47]. Given a ground truth pixel-wise segmentation set A and a predicted pixel-wise segmentation set B for a single class on a single image, the IoU is defined as:

$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}. \tag{2.2}$$

IoU lies in $[0, 1]$, with a value of 1 indicating identical sets and 0 indicating disjoint sets. We report class-wise IoU for each tissue class (wing, envelope, seed), reflecting the differing biological importance and segmentation difficulty of each class. We additionally report the mean IoU (mIoU) as a single-number summary used to rank models during benchmarking. We compute IoU separately for each test image and summarize it as a mean and standard deviation across images, rather than pooling pixels over the whole test set, because per-image scores are more informative than

a single dataset-level score for segmentation evaluation [35]. For consistency, we compute IoU identically across all architectures.

2.2.6 Predictive Phenotype Networks

To explore the relationships between measured phenotypes, we produced predictive phenotype networks (PPNs) using iRF-LOOP [30]. Given an input set of extracted phenotypes for each pod, we computed the average value for each accession, which we used as input to the model. iRF-LOOP uses iterative random forest [9] to predict a single held out phenotype from all others. Iterative random forest modifies the probability that each feature is sampled to create splits in the decision trees, weighting it by the feature's importance in the previous iteration. Because all iRF-LOOP tasks in our phenotype set are regression tasks, importance is measured by Mean Decrease in Impurity (MDI), which captures the normalized total reduction in mean squared error attributable to each feature across all trees, normalized to sum to one. Using this process, informative features are selected more frequently as the model converges. This process is repeated by holding out each phenotype, with the entire model output yielding pairwise importance values between phenotypes.

Using the iRF-LOOP output, we constructed PPNs, which take the form of a directed graph, where nodes represent phenotypes and edges represent predictive relationships. An edge from phenotype A to phenotype B indicates that A is predictive of B, with edge weight equal to the iRF-LOOP importance of A in predicting B. Using the raw iRF-LOOP output produced a dense network with all phenotypes connected to all others; however, we induced sparsity and highlighted only strong predictive relationships by retaining only edges whose weight falls in the top 5%. For interpretation, we also visualized the subset of retained edges that connect traits measured from different pod tissues. PPN outputs allowed us to perform an exploratory analysis of relationships between phenotypes before downstream analysis.

2.2.7 Genome-Wide Association Study

After measuring seed pod phenotypes and their relationships on a population scale, we performed a GWAS to find significant associations between SNP loci and variation in phenotypic traits.

2.2.7.1 Phenotype Curation and Heritability Estimation

Before running association tests, we assessed whether each phenotype had sufficient genetic signal to warrant GWAS by estimating broad-sense heritability in the dryland environment, in which the pod-morphology phenotypes analyzed here were measured. Because most accessions in our panel are unreplicated, H^2 was estimated on the subset of $n = 25$ replicated genotypes. For each phenotype, we fit a linear mixed model with genotype as a random effect and computed

$$H^2 = \sigma_G^2/(\sigma_G^2 + \sigma_E^2),$$

where σ_G^2 and σ_E^2 are genotypic and residual variance components. Phenotypes with $H^2 < 0.05$ were removed from downstream association analyses due to lacking sufficient heritable signal. We additionally restricted our analysis to phenotypes derived from the U-Net segmentation pipeline described in Section 2.2.3, excluding manually measured traits to maintain methodological consistency. Per-trait H^2 for the screened traits is reported in Table 2.1.

2.2.7.2 Genotypic Data

SNP genotypes were drawn from the imputed, lightly filtered whole-genome re-sequencing dataset of *Thlaspi arvense* natural accessions generated by Wu et al. [219]. We applied additional quality-control filtering: SNPs with severe departures from Hardy–Weinberg equilibrium ($p < 1 \times 10^{-150}$) were removed, genotypes with more than 10% missing calls were dropped, and the highly differentiated Armenian

Table 2.1: Broad-sense heritability (H^2) of the 34 U-Net-derived pod traits in the dryland environment that passed the $H^2 \geq 0.05$ screen and were carried into GWAS. H^2 was estimated from a linear mixed model with genotype as a random effect, fit on 25 replicated genotypes (2–3 replicate images per genotype; ∼15 pods per replicate image). Significance p is from a restricted likelihood-ratio test on the genotype variance component ($H_0 : \sigma^2_G = 0$). Traits marked † yielded significant GWAS associations (Table 2.4); note that heritability was necessary but not sufficient for a GWAS hit.

Trait	H^2	Significance (p)
envelope-to-wing area ratio	0.461	< 0.0001
envelope-to-total area ratio	0.423	0.0002
wing-to-envelope area ratio	0.409	0.0003
envelope perimeter	0.390	0.0016
envelope area	0.389	0.0017
wing-to-total area ratio	0.387	0.0003
wing area†	0.366	0.0010
wing HSV saturation	0.321	0.0022
wing perimeter	0.302	0.0060
seed area	0.297	0.0214
envelope-to-wing perimeter ratio	0.292	0.0006
wing-to-envelope perimeter ratio	0.287	0.0009
wing-to-seed area ratio†	0.285	0.0077
seed perimeter	0.263	0.0334
seed-to-wing area ratio	0.247	0.0117
envelope-to-total perimeter ratio	0.230	0.0051
wing CIELAB b^*	0.227	0.0525
envelope HSV saturation	0.220	0.0418
seed count	0.218	0.0319
envelope CIELAB b^*†	0.217	0.0315
seed RGB red	0.204	0.0881
envelope-to-seed area ratio†	0.200	0.0501
seed CIELAB b^*	0.190	0.0627
seed-to-envelope area ratio	0.188	0.0681
seed-to-total area ratio	0.188	0.0453
seed HSV hue	0.185	0.0796
seed HSV saturation	0.155	0.1048
wing RGB blue	0.131	0.1770
seed HSV brightness	0.120	0.1518
seed CIELAB lightness	0.097	0.1672
envelope RGB green	0.089	0.2169
envelope CIELAB lightness	0.086	0.2213
seed RGB blue	0.083	0.1234
envelope HSV brightness	0.068	0.2530

subpopulation was excluded. The resulting dataset comprised 423 genotypes and 1,975,204 SNPs. Association testing was restricted to the 189 accessions with matched U-Net-derived pod phenotypes.

2.2.7.3 Association Testing

We fit four single- and multi-locus association models implemented in GAPIT (Genome Association and Prediction Integrated Tool) [210]: MLM [225], MLMM [177], FarmCPU [119], and BLINK [77]. We further restricted the marker set to the 1,759,624 SNPs with a minor allele frequency (MAF) ≥ 0.03. We ran GWAS independently per phenotype and retained the union of SNPs reaching $p < 1 \times 10^{-6}$ under any of the four models. Because extensive linkage disequilibrium makes the per-SNP tests non-independent, a Bonferroni correction is overly stringent; we therefore treated raw $p < 1 \times 10^{-6}$ as a suggestive association cutoff and deferred false-positive control to GRIN (Section 2.2.8.1).

2.2.7.4 SNP-to-Gene Assignment

After finding significant loci, we mapped SNPs to candidate genes using a two-pronged approach. For each significant SNP, we first fit a local linkage disequilibrium (LD) decay curve and defined a data-driven window around the SNP based on fitted decay distance. We then identified all genes whose coordinates fall within this window on the pennycress TAV2 reference genome. When LD decay could not be reliably fit for a SNP, we instead assigned the two nearest flanking genes (one upstream and one downstream) by physical distance. Because functional annotation of the pennycress genome is limited, we annotated each assigned pennycress gene by its best BLAST-hit Arabidopsis ortholog from TAIR [53] to enable downstream biological interpretation.

2.2.7.5 Candidate Gene Set Construction

We aggregated candidate genes by taking the union of genes mapped from significant SNPs, regardless of which GWAS method produced the original association. We restricted our downstream analysis to traits with considerable heritability ($H^2 \geq 0.05$) and to phenotypes derived from the U-Net segmentation pipeline. Under these restrictions, four pod-related phenotypes (envelope CIELAB b^*, envelope-to-seed area ratio, wing area, and wing-to-seed area ratio) yielded significant GWAS hits, mapping to 6 unique SNPs and 69 pennycress genes, 64 of which had annotated Arabidopsis TAIR orthologs. We note that these four phenotypes are highly correlated, as they are geometric ratios computed from a common set of pod measurements. They therefore do not represent four biologically independent signals. These 64 orthologs constitute the candidate ortholog set, which we filtered with GRIN (Section 2.2.8.1) before MENTOR module derivation (Section 2.2.8).

2.2.8 MENTOR Module Derivation

2.2.8.1 GRIN Filtering

Before using our candidate genes as input to MENTOR for module construction, we applied GRIN [190] to reduce potential false-positive genes. Both GRIN and MENTOR operate over a nine-layer *Arabidopsis thaliana* multiplex network spanning a pool of 26,605 genes, in which each layer encodes a distinct line of biological evidence: protein–protein interaction, transcriptional regulation from two independent sources, gene co-expression, co-evolution, knockout phenotype similarity, methylation-derived predictive relationships, metabolic pathway membership, and a diversity-derived layer. All nine layers are undirected and unweighted, and each contributes equally to network propagation; per-layer sources and sizes are given in Table A.2. Given a candidate gene set, whose members serve as the seed nodes from which network propagation begins, and this multiplex network [93], GRIN operates in two stages. First, it runs random walk with restart (RWR) from the seed nodes, yielding a

rank vector of the genes in the network most closely related to them; in parallel, it runs RWR on 100 randomly sampled gene sets of the same size, creating a null distribution rank vector. Next, GRIN uses a sliding-window Mann-Whitney U test to evaluate the likelihood of observing the seed-node RWR ranks at random. With this sliding window, a threshold can be defined where the ranks become insignificant. The resulting significant genes are retained, while all others are dropped. GRIN retains genes that are more tightly connected in the multiplex network to the rest of the candidate set; this network proximity is taken as a proxy for shared biological function. In this paradigm, GRIN assumes that well-connected genes may jointly contribute to the associated phenotype. We refer to the genes GRIN retains as the network-supported gene set.

2.2.8.2 Module Construction

We applied MENTOR, a multi-step framework for deriving functionally related genetic modules, using the multiplex network and the network-supported gene set as input [191]. We used the *Arabidopsis thaliana* multiplex because this close relative has richer functional annotation and network resources than pennycress.

Starting from the network-supported gene set (Section 2.2.8.1), MENTOR applies RWR as a diffusion method to measure the probability of traveling to each gene in the multiplex from all genes in that set. Each seed node produces its own score vector over all multiplex nodes, with indices representing the probability of reaching each node from that seed. The score vectors are then rank-ordered, truncated at the elbow of the mean score-versus-rank curve, and converted into a dissimilarity matrix by taking $(1 - \rho)$, with ρ representing Spearman correlation, between each pair of genes' rank-ordered score vectors. With the resulting dissimilarities, MENTOR creates a dendrogram using agglomerative clustering with complete linkage as our module distance measure. This dendrogram is then thresholded to examine modules of a certain size.

2.2.9 MENTOR-IA Interpretation

After deriving a set of modules with MENTOR, we used MENTOR-IA to explore the biological mechanisms associated with these modules [208]. MENTOR-IA is a multi-agent LLM framework for mechanistic interpretation, in which specialized agents each contribute to a distinct line of evidence. These agents include a mygene interpreter, which utilizes gene summaries, GO terms, and pathway annotations to explore each gene's role individually; a literature agent, which explores mechanistic connections in related literature via Europe PMC; an enrichment agent, which tests whether a module is enriched for pathways in GO, KEGG, or Reactome; a network agent, which uses subgraph evidence to refine the interpretation; and a meta aggregator, which synthesizes evidence to produce a coherent mechanistic summary.

Using the evidence provided by MENTOR-IA as a starting point, we manually examined pennycress and *Brassicaceae* literature to assess whether the proposed mechanisms were consistent with known biology.

2.2.10 Statistical Analysis

This section summarizes the statistical analyses applied at each stage of the pipeline. Models, sample sizes, and significance thresholds are specified in the referenced sections, and the tables cited below report the resulting statistics.

Segmentation accuracy was evaluated with per-class IoU and mIoU (Section 2.2.5), computed per test image and summarized as mean ± standard deviation in Table 2.2. We additionally profiled the computational efficiency of each benchmarked architecture—parameter count, mean per-image inference time, and peak allocated GPU memory on the test set—reported in Table 2.3 and Figure 2.6.

Predictive phenotype networks were constructed with iRF-LOOP, using Mean Decrease in Impurity as the edge-weight statistic (Section 2.2.6). Broad-sense heritability was estimated from a linear mixed model with genotype as a random effect (Section 2.2.7.1); per-trait H^2 and the significance of the genotype variance

component are reported in Table 2.1. Association testing used four models implemented in GAPIT (Section 2.2.7); retained associations, lead SNPs, and their unadjusted p-values are reported in Table 2.4. Candidate genes were then filtered on network connectivity using the sliding-window Mann-Whitney U test implemented in GRIN (Section 2.2.8.1).

2.3 Results

We present our results in two stages: the first converts raw pod scans into a per-accession trait matrix, and the second carries that matrix toward candidate genetic mechanisms. In the first stage, we benchmarked segmentation architectures and selected U-Net, extracted per-pod phenotypes from the predicted masks, and applied the trained model at population scale to assemble the trait matrix. We then examined the trait matrix for biological coherence using a predictive phenotype network; this analysis is exploratory and validates the extracted phenotypes rather than selecting which traits enter the downstream analysis. In the second stage, we filtered the trait matrix by heritability, combined the retained traits with genome-wide resequencing genotypes for GWAS, and mapped the associated SNPs to candidate genes. We then pruned those candidates with GRIN, resolved the survivors into modules with MENTOR, interpreted the modules with the assistance of MENTOR-IA, and assessed whether the prioritized candidates are expressed in pod tissue.

2.3.1 Segmentation Benchmark

We evaluated our models' segmentation performance on 5 raw images containing 65 seed pods in total. Per-class IoU and across-class mIoU are reported in Table 2.2. The boundary down-weighted U-Net achieved the highest mIoU at 85.58%, narrowly ahead of the baseline U-Net at 85.53% and Mask2Former at 85.03%. These top scores were close relative to per-image variation, so we interpret the leading architectures as

Table 2.2: Segmentation benchmark results. Per-class IoU (wing, envelope, seed) and mean IoU (mIoU), reported as mean ± standard deviation (%) across the five full test images; mIoU is the mean of the three tissue-class IoUs. Standard deviations are population values (ddof = 0) computed over the five test images; nnU-Net was scored on pod crops under its own adaptive pipeline (Section 2.2.5) and grouped back to the source full images before summary. Bold values denote the best mean score in each column.

Model	Wing	Envelope	Seed	mIoU
U-Net	95.60 ± 0.98	82.84 ± 2.31	78.14 ± 3.83	85.53 ± 1.45
U-Net-Down	95.53 ± 0.92	**82.94** ± 2.35	**78.28** ± 3.59	**85.58** ± 1.46
U-Net-Up	95.58 ± 0.95	82.24 ± 1.79	77.60 ± 3.81	85.14 ± 1.22
nnU-Net	**95.77** ± 0.93	81.78 ± 1.44	75.40 ± 4.65	84.32 ± 1.39
DeepLabV3+	95.19 ± 0.90	81.53 ± 2.80	75.80 ± 3.93	84.17 ± 1.72
DeepLabV3+-Up	95.28 ± 0.92	81.70 ± 2.72	75.64 ± 3.76	84.21 ± 1.64
DeepLabV3+-Down	95.14 ± 0.91	81.31 ± 2.69	75.89 ± 3.79	84.11 ± 1.52
SegFormer	95.03 ± 0.99	80.45 ± 2.24	75.07 ± 3.95	83.52 ± 1.24
SegFormer-Up	94.99 ± 0.99	80.22 ± 2.18	75.23 ± 4.07	83.48 ± 1.23
SegFormer-Down	95.31 ± 1.04	81.68 ± 2.13	76.36 ± 4.29	84.45 ± 1.15
Mask2Former	95.65 ± 1.15	82.80 ± 3.67	76.65 ± 4.41	85.03 ± 2.05
SAM 3	95.15 ± 1.54	76.42 ± 4.30	65.62 ± 9.19	79.07 ± 4.04
SAM 3-Up	95.35 ± 1.54	76.78 ± 4.06	68.15 ± 7.33	80.09 ± 3.10
SAM 3-Down	94.95 ± 1.82	73.49 ± 3.92	57.61 ± 9.39	75.35 ± 3.32

Table 2.3: Computational efficiency of the benchmarked segmentation architectures. Parameter count (millions), mean inference time per full test image ($n = 5$, ROCm), and peak allocated GPU memory are summarized once per architecture. For architectures with boundary-weighting variants, inference time is averaged across the measured variants because parameter counts and peak allocated memory are architecture-level quantities. For nnU-Net, inference time and peak memory were computed by grouping its pod-crop predictions back to their five source full images. SAM 3 is adapted with LoRA, so only 5.70 M of its parameters are trainable. Bold values denote the lowest cost in each column.

Model	Params (M)	Inference time (s/image)	Peak memory (GB)
U-Net	**12.57**	**43.1**	**0.06**
nnU-Net	65.21	202.2	0.65
DeepLabV3+	45.67	122.7	0.18
SegFormer	81.97	412.1	0.39
Mask2Former	215.45	377.0	0.90
SAM 3	459.74	400.7	1.81

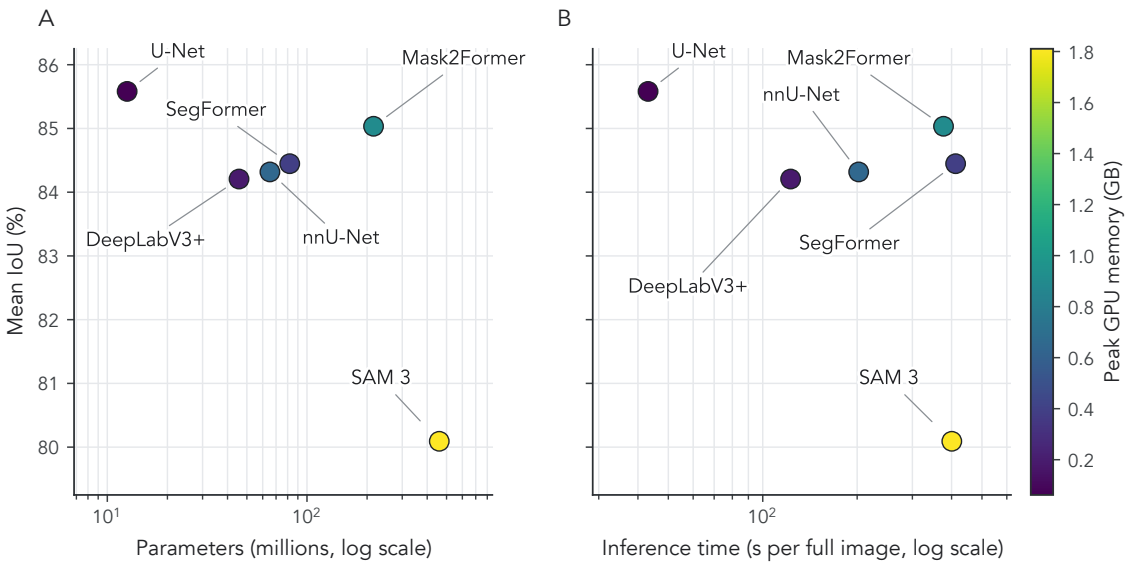

Figure 2.6: Accuracy and computational efficiency of benchmarked segmentation architectures. (A) Best mean IoU (mIoU) achieved by each architecture across its boundary-weighting variants plotted against total parameter count on a log scale. (B) Best mIoU plotted against mean full-image inference time on a log scale, where lower values indicate faster segmentation. Point color and the shared color bar encode peak allocated GPU memory during full-image inference; nnU-Net values were grouped from pod-crop predictions back to the source full images.

comparable in accuracy rather than as statistically separable. SAM 3 was the clearest laggard, especially for seed segmentation, where IoU fell as low as 57.61%.

Efficiency separated the leading architectures more strongly than accuracy (Table 2.3; Figure 2.6). U-Net used 12.57 million parameters, required 43.1 s per full test image, and peaked at 0.06 GB allocated GPU memory. nnU-Net, DeepLabV3+, SegFormer, Mask2Former, and SAM 3 required more parameters and longer inference times; Mask2Former had a similar mIoU but used 215.45 million parameters, required 377.0 s per full test image, and peaked at 0.90 GB allocated GPU memory. Thus, U-Net provided the most favorable accuracy-efficiency tradeoff for population-scale phenotyping in this small, specialized pod-segmentation task. We found no prior work measuring semantic segmentation of pennycress to compare with our model benchmark suite.

2.3.2 Population-Scale Phenotyping

Having selected U-Net as the most favorable benchmark on accuracy and efficiency, we applied it at population scale to characterize phenotypic structure across the pennycress accession panel. We measured phenotypes from 12,253 seed pods across 768 accessions. Each image yielded approximately 16 retained pod crops on average and took around 2 minutes of model inference time to segment, compared to approximately 1.5 hours of hand segmentation per image, yielding an estimated 30- to 45-fold throughput gain depending on annotator pace.

We used the feature extraction pipeline described in Section 2.2.4 to construct an open-source dataset of over 50 phenotypes for all 12,253 seed pods. For visualization and correlation summaries, we excluded detections for which the watershed step returned zero seeds. Figure 2.7 shows the distributions of three key phenotypes: wing area, envelope area, and seed area. These traits correspond directly to the three annotated tissue classes, making their distributions a useful biological plausibility check on segmentation output. Each trait was right-skewed and heavy-tailed,

consistent with the log-normal-like distributions commonly observed in quantitative plant traits [96].

Consistent with the wing-seed regulatory association described in Section 2.1, wing area was positively correlated with seed area and seed count (Pearson $r = 0.51$ and 0.30, respectively; Figure 2.8). The resulting per-accession phenotype matrix served two roles: as input to iRF-LOOP for PPN construction (Section 2.2.6), and as the candidate trait pool for heritability filtering and GWAS (Section 2.2.7).

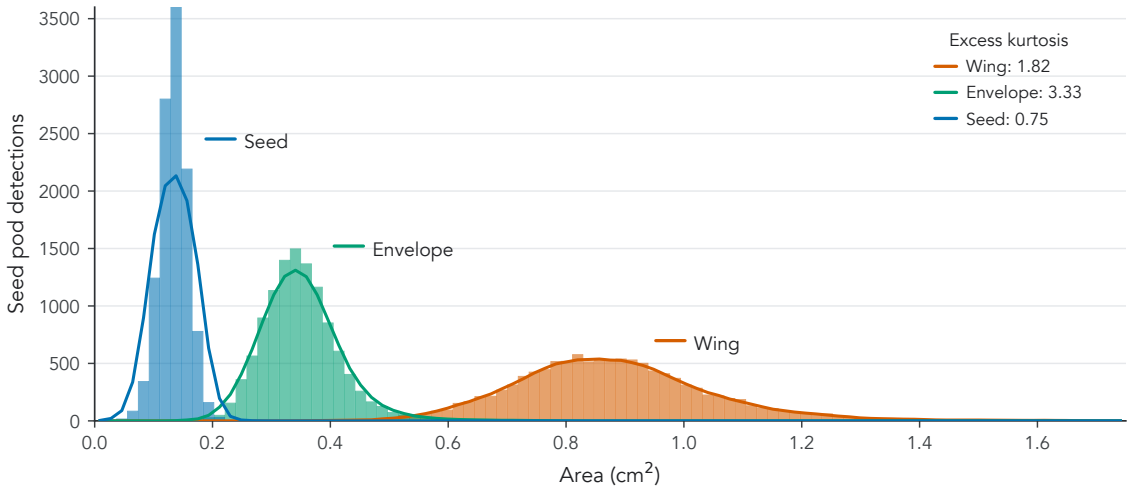

Figure 2.7: Distribution of seed-pod component areas across non-empty population-scale detections. Histograms show the measured areas (cm^2) of wing, envelope, and seed tissues after excluding detections with zero watershed seed markers. Colored lines show five-bin moving averages of histogram counts, and inset values report excess kurtosis, quantifying heavier-than-normal distribution tails. Seed area is concentrated at smaller values, envelope area at intermediate values, and wing area at larger values; all three traits are right-skewed and log-normal-like, consistent with natural variation in plant morphology.

2.3.3 Predictive Phenotype Network

After extracting phenotypes from our dataset, we used iRF-LOOP to construct PPNs to check the biological plausibility of extracted phenotypes. Running iRF-LOOP on the entire dataset yielded a PPN with 48 nodes and 108 directed edges after filtering the network to include only the top 5% of relationships (Figure 2.9). Despite being

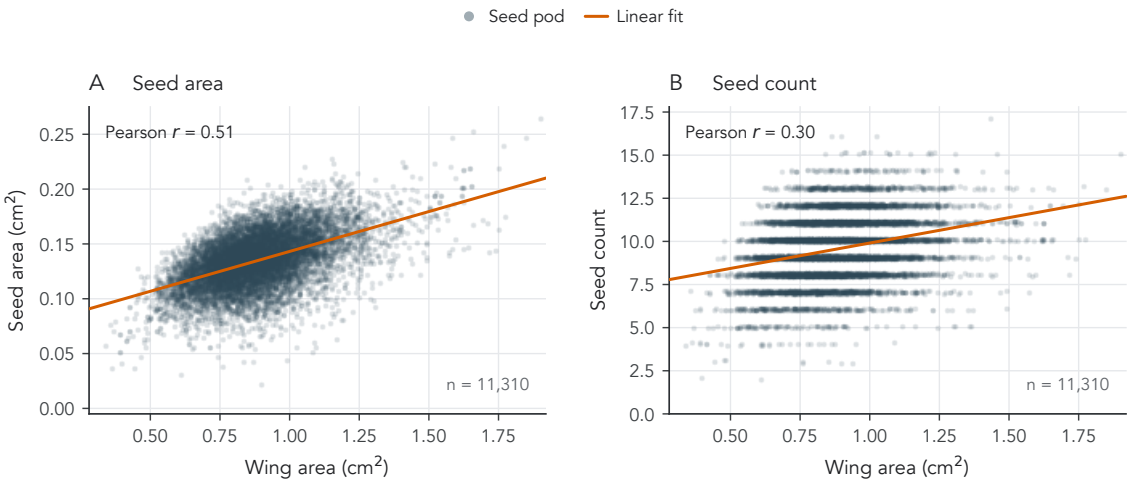

Figure 2.8: Relationship between wing area and seed traits across non-empty population-scale detections. Wing area is plotted against (A) seed area and (B) seed count, with each point representing one seed pod and the solid line showing the linear least-squares fit. Points in (B) are vertically jittered by less than 0.1 seed for visibility; correlations and fitted lines use the unjittered seed-count values. Wing area is positively correlated with both seed area (Pearson $r = 0.51$) and seed count ($r = 0.30$), consistent with wing–seed regulatory association in *Brassicaceae*.

a directed network, the PPN exhibited high reciprocity, indicating that predictive relationships tend to be mutual, which is expected for many geometrically derived traits.

The PPN separated into three connected components after filtering, each dominated by a single feature type: a 27-node component containing all color features across all three tissues, a 12-node component containing absolute geometry and area ratios, and a 9-node component containing all perimeter ratios. This separation by measurement type is expected given how tightly color and geometric features covary within each class.

To interpret the cross-tissue relationships more directly, we extracted the subset of PPN edges that connect traits across pod tissues (Figure 2.10). This focused subset separates two recurring patterns: pigment coupling among color traits and size coupling among envelope, seed, and wing geometry traits.

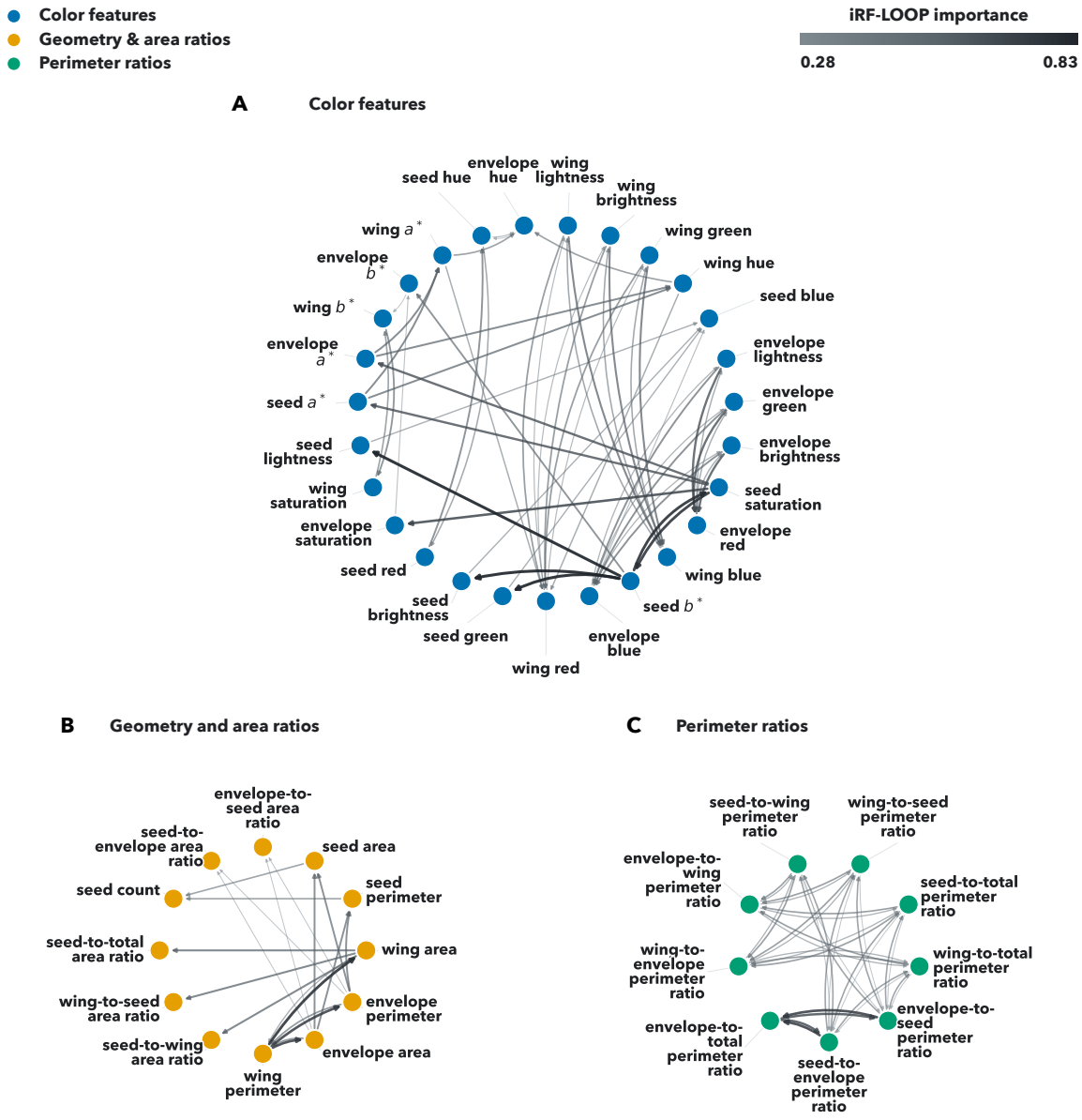

Figure 2.9: Predictive phenotype network (PPN) for pod-related traits after retaining the top 5% of iterative Random Forest Leave-One-Out Prediction (iRF-LOOP) relationships. Nodes are measured phenotypes; node color and panel membership denote feature classes: (A) color features, (B) geometry and area ratios, and (C) perimeter ratios. Directed arrows indicate predictor-to-response relationships, and edge width and darkness encode iRF-LOOP importance. The filtered network contains 48 nodes and 108 directed edges. In color-feature labels, a^* and b^* are CIELAB color channels.

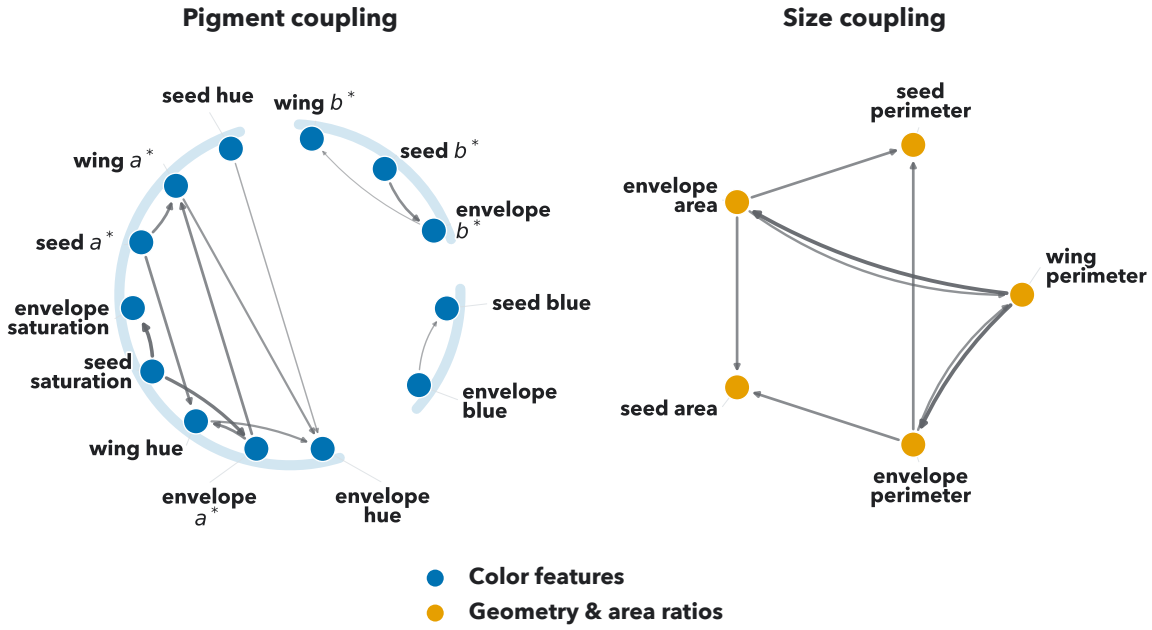

Figure 2.10: Focused subset of cross-tissue predictive phenotype network relationships. The subset highlights PPN edges connecting traits across pod tissues, separated into pigment coupling and size coupling groups. Nodes are measured phenotypes, with blue nodes denoting color features and orange nodes denoting geometry and area-ratio features. Directed arrows indicate predictor-to-response relationships, and edge width and darkness encode iRF-LOOP importance. Pale blue arcs visually group related pigment-feature relationships. In color-feature labels, a^* and b^* are CIELAB color channels.

Within this focused subset, several informative relationships emerged that could not be explained by geometric identities alone. Most notably, both envelope area and perimeter were predictive of seed area and perimeter (iRF-LOOP importance ≈ 0.56), consistent with the pod's active role in seed development and resource allocation in *Brassicaceae* [13]. This relationship mirrors the envelope-to-seed area ratio, which was the most recurrent of our significant GWAS signals (Section 2.2.7). Similarly, seed and envelope color features were predictive of each other in CIELAB b^*, RGB blue, and HSV hue spaces. This suggests that color features, which provide a proxy for pod maturity and pigmentation, may be co-regulated across tissues. These relationships are correlative and do not imply causality, but they indicate that extracted phenotypes capture biologically coherent structure beyond trivial geometric relationships.

2.3.4 Heritability of U-Net-Derived Pod Traits

Of the 149 phenotypes available for this panel, 52 were extracted by our U-Net pipeline. Screening these for heritable genetic signal retained 34 traits, with H^2 ranging from 0.068 to 0.461 (Table 2.1); four of the 34 ultimately yielded significant GWAS associations.

Heritability was strongly structured by trait class. The 19 retained geometry and ratio traits spanned H^2 0.188–0.461, whereas the 15 retained color traits spanned 0.068–0.321, and all seven of the most heritable traits were geometric. The most heritable color trait, wing HSV saturation ($H^2 = 0.321$), fell below every one of them. This separation anticipates the association results below, in which three of the four traits with significant hits are geometry or area ratios.

Heritability was necessary but not sufficient for a GWAS hit, and the retained set includes traits whose genotype variance components were weakly resolved. Fourteen of the 34 retained traits did not reach nominal significance on the restricted likelihood-ratio test ($p > 0.05$), and 12 of those 14 were color traits. Envelope-to-seed area ratio, which produced the most recurrent association signal, was itself borderline ($H^2 =$

0.200, $p = 0.0501$). We therefore treat the downstream candidates as exploratory rather than as well-powered genetic findings. Per-trait H^2 for all 52 U-Net-derived traits, including the 18 that fell below the screen, is reported in Table A.1.

2.3.5 Genome-Wide Association and Gene Mapping

We performed GWAS on the U-Net-derived pod traits on a panel of 189 accessions using four GAPIT models (MLM, MLMM, FarmCPU, and BLINK). We retained candidate SNPs with raw-$p < 1 \times 10^{-6}$ (a suggestive association cutoff; a Bonferroni threshold is overly stringent given linkage disequilibrium among SNPs) and MAF ≥ 0.03, deferring false-positive control to GRIN filtering in the module derivation stage.

GWAS yielded six unique SNPs across four phenotypes: envelope CIELAB b^*, envelope-to-seed area ratio, wing area, and wing-to-seed area ratio. The most recurrent signal came from envelope-to-seed area ratio. Our GWAS results are summarized in Table 2.4.

Table 2.4: Summary of retained GWAS associations for heritable, U-Net-derived pod traits (dryland environment), tested across 189 genotyped accessions. Associations are reported at a suggestive raw $p < 1 \times 10^{-6}$ cutoff; linkage disequilibrium among SNPs makes per-SNP tests non-independent, so a Bonferroni threshold is overly conservative. H^2, broad-sense heritability; SNPs, number of retained single nucleotide polymorphisms; Models, GAPIT models (MLM, MLMM, FarmCPU, BLINK) producing the associations; Lead SNP, most significant SNP (chromosome_position); Lead raw p, its unadjusted p-value.

Trait	H^2	SNPs	Models	Lead SNP	Lead raw p
envelope CIELAB b^*	0.217	1	MLMM	`4_8127381`	8.71×10^{-7}
envelope-to-seed area ratio	0.200	3	BLINK, FarmCPU, MLMM	`5_963442`	1.04×10^{-7}
wing area	0.366	1	MLMM	`7_63065229`	4.14×10^{-7}
wing-to-seed area ratio	0.285	1	MLMM	`2_2343404`	6.06×10^{-7}

We mapped the six retained SNPs from GWAS to pennycress genes using the mixed LD-decay and distance-based approach described in Section 2.2.7.4. For each

SNP, we defined a window centered on the SNP with a width set by local LD-decay distance and assigned the annotated genes falling in that window, retaining any gene whose body directly contained that SNP. Windows spanned approximately 24.6–49.6 kb, with strong LD-decay fits (R^2 = 0.944–0.989), reflecting the different recombination contexts of each chromosomal region. Because all six SNPs yielded reliable LD-decay fits, every candidate gene was assigned from an LD window, and the distance-based fallback was not used.

This procedure produced 69 candidate pennycress genes, with half of the loci tracing to the envelope-to-seed area trait and the remaining loci tracing to the other three GWAS traits. We mapped these genes to their nearest Arabidopsis orthologs in TAIR [53] to enable cross-species functional annotation and module derivation. This mapping yielded 64 unique orthologs, as several pennycress genes shared a single Arabidopsis ortholog. This ortholog set constitutes the candidate ortholog set carried forward to GRIN filtering and module derivation (Section 2.2.8).

2.3.6 MENTOR Module Derivation

GRIN filtering reduced the candidate ortholog set of 64 genes to 33, pruning 31 that lacked sufficient network support; these 33 constitute the network-supported gene set. We applied MENTOR to this set, using the process and standard settings described in Section 2.2.8, resolving it into three modules of 5, 11, and 17 genes (Figure 2.11). We then used these modules for mechanistic interpretation (Section 2.2.9).

2.3.7 MENTOR-IA Exploratory Interpretation

We used MENTOR-IA to conduct an exploratory interpretation of the three modules derived by MENTOR (Section 2.2.8). Module 1, which contained 17 Arabidopsis orthologs, was run with the enrichment agent, literature agent, mygene interpreter, and meta aggregator. Module 2, which contained 11 orthologs, was run with the same agents as Module 1. Module 3, which contained 5 orthologs, was run with only the

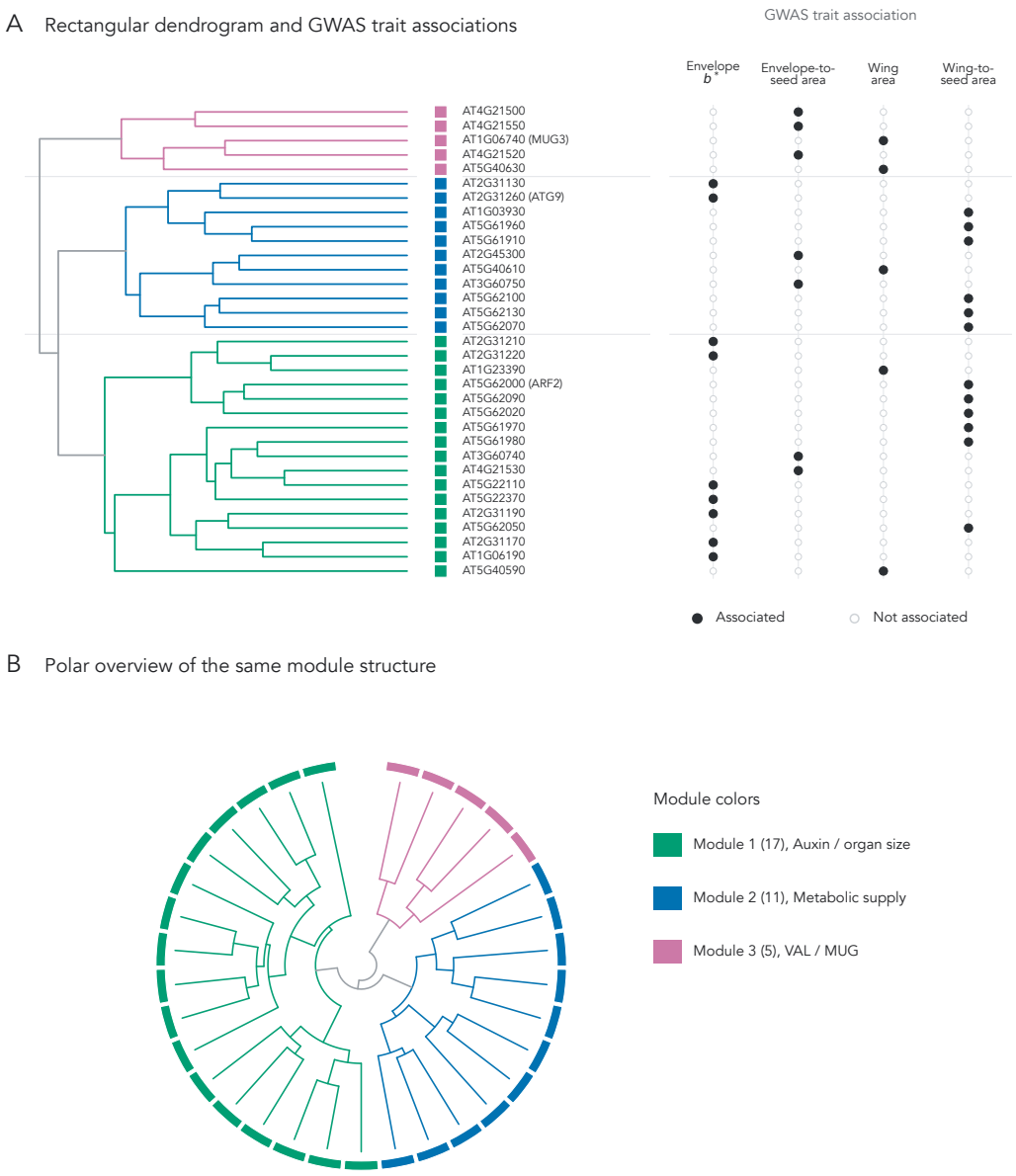

Figure 2.11: MENTOR dendrogram of the 33 GRIN-refined Arabidopsis orthologs retained from U-Net-derived pod-trait GWAS candidates. (A) Complete-linkage clustering of random-walk-with-restart (RWR) dissimilarities, with the adjacent trait matrix indicating which of the four GWAS traits—envelope CIELAB b^*, envelope-to-seed area ratio, wing area, and wing-to-seed area ratio—each gene was associated with. (B) Polar view of the same topology, emphasizing the three MENTOR modules. Branch and outer-ring colors indicate Module 1 (17 genes; auxin, organ-size, and reproductive development), Module 2 (11 genes; metabolic supply, autophagy, and nutrient remobilization), and Module 3 (5 genes; VAL/MUG transposon-derived regulators).

mygene interpreter and meta aggregator, as the small gene count limited the power of enrichment analyses. Because our modules did not have an associated network, we did not use the network agent for analysis.

MENTOR-IA produced per-agent summaries for each module, along with a final interpretation that synthesized evidence across agents. This interpretation included keyword themes, enrichment statistics from GO and KEGG, and a mechanistic summary with candidate supporting citations. Across modules, the proposed mechanisms recovered known *Brassicaceae* pod and seed biology, demonstrating our pipeline's ability to connect measured phenotypes to candidate mechanistic explanations. We manually cross-checked the agent output against pennycress and *Brassicaceae* literature, finding the mechanistic summaries to be consistent with reported biology.

Because functional interpretation used Arabidopsis orthologs, the mechanisms below should be read as candidate hypotheses rather than validated pennycress-specific effects. For clarity, gene-symbol mentions below refer to Arabidopsis orthologs; parenthetical IDs give the Arabidopsis locus followed by the corresponding pennycress TAV2 locus or loci.

2.3.7.1 Module 1 Interpretation

We confirmed that the MENTOR-IA module 1 analysis recovered mechanistic themes centered around auxin-mediated control of organ size and reproductive development, with supporting themes connecting to auxin and cell division more broadly. [173] demonstrated that Arabidopsis *ARF2* (`AT5G62000`; pennycress `TAV2_LOCUS7857`) represses auxin-responsive transcription, limiting integument cell proliferation, and loss of *ARF2* function leads to enlarged seeds and organs. [147] found that *ARF2* genetically interacts with *SEEDSTICK* and *GORDITA* during seed development, linking this auxin-response factor to integument growth and seed-coat differentiation. These findings provide the strongest candidate bridge to envelope-to-seed area ratio,

while any connection to wing area remains more indirect through broader auxin-mediated organ-growth biology.

Module 1's link to auxin and cell division is further supported by the presence of Arabidopsis *APC4* (`AT4G21530`; pennycress `TAV2_LOCUS24640`), which encodes a subunit of anaphase-promoting complex (APC/C). [212] demonstrated that Arabidopsis *apc4* mutants exhibit defects in female gametogenesis and embryogenesis, with aberrant embryo cell division and distorted auxin distribution. [120] also implicated *ARF2*, together with *ARF4* and *ARF5*, in gametophyte development, reinforcing the reproductive-development theme. Arabidopsis *RUS2*/*WXR1* (`AT2G31190`; pennycress `TAV2_LOCUS13944`) provides a second auxin-related but less direct line of evidence. [54] showed that *RUS2*/*WXR1* is required for normal polar auxin transport and PIN/AUX1 abundance in roots, while [108] placed *RUS2* with *RUS1* in a root UV-B-sensing pathway. Thus, the proposed *RUS2* connection to wing growth is an inference from auxin-distribution biology rather than a demonstrated pod-morphology effect. The module also surfaced a possible connection to color phenotypes such as envelope CIELAB b^*; *arf2* mutants show delayed senescence and altered chlorophyll retention in leaves [44]. Because this evidence comes from leaves and developmental timing instead of pod-envelope color directly, the connection to pod color remains tentative.

In addition to the literature-supported themes that we manually verified, the MENTOR-IA enrichment agent returned several significant GO and KEGG terms consistent with reproductive development, such as floral whorl development ($p = 9.8 \times 10^{-5}$), ovule development ($p = 3.6 \times 10^{-3}$), and embryo development ending in seed dormancy ($p = 3.6 \times 10^{-3}$), providing additional support for the proposed developmental theme.

Despite the strong theme and supporting evidence around auxin-mediated control of organ size and reproductive development, the remaining orthologs in module 1 had only gene-level annotation and no clear trait-linking literature in the MENTOR-IA output, so we leave their exploration for future work and specify them as additional

candidates. The module 1 interpretation is consistent with known biology of pod and seed development in *Brassicaceae*.

2.3.7.2 Module 2 Interpretation

Module 2 is centered around metabolic supply, autophagy, nutrient remobilization, and chloroplast carbon metabolism. Unlike module 1, which provides evidence of a direct developmental regulator in *ARF2*, module 2 provides an indirect hypothesis in which metabolic supply from the pod to the seed may influence pod growth. The strongest candidate connection is to envelope-to-seed area ratio and pod filling because the literature links these pathways to seed resource allocation rather than directly to pod morphology.

Arabidopsis *ATG9* (`AT2G31260`; pennycress `TAV2_LOCUS12330`) anchors the module's autophagy signal. [182] showed that *ATG9* contributes to autophagic flux, while [85] found that *atg9* mutants retain constitutive autophagy in root tips, supporting a modulatory role instead of a strictly essential one. [94] and [124] place *ATG9* in autophagosome formation and closure pathways, and [107] provides structural evidence for *ATG9*'s role in autophagy via cryo-EM, showing that it is the core integral membrane autophagy protein. [59] observed that *atg* mutants show reduced nitrogen remobilization to seeds under both low- and high-nitrate conditions, and [41] demonstrated that autophagy affects resource allocation and storage-protein accumulation during seed development. These findings suggest that variation in autophagy may affect pod filling or envelope-to-seed nutrient allocation, providing a candidate link to phenotypes like envelope-to-seed area ratio.

Arabidopsis *TKL1* (`AT3G60750`; pennycress `TAV2_LOCUS16016`) provides a second line of evidence for metabolic supply and chloroplast carbon metabolism. [163] found that *TKL1* is phosphorylated at Ser428 in a light- and chloroplast-dependent manner, suggesting that it may be a regulatory point for carbon flux in the Calvin-Benson-Bassham (CBB) cycle and oxidative pentose phosphate pathway (OPPP). Additionally, [98] demonstrated that altered plastid transketolase dosage disrupts

growth through thiamine- and TPP-related metabolic constraints. Because carbon supply is important to seed growth, we infer that *TKL1* may provide a second, indirect link to pod growth and envelope-to-seed area, supporting our mechanistic supply hypothesis.

Additionally, [181] adds a redox-homeostasis line of evidence, showing that Arabidopsis *GPDHc1* (`AT5G40610`; pennycress `TAV2_LOCUS25599`) controls plant NADH/NAD^+ ratios through the mitochondrial glycerol-3-phosphate shuttle. Although the OPPP is not directly implicated in this work, it supports a broader interpretation in which module 2 captures genes involved in carbon and redox balance during pod and seed development.

Outside of the autophagy and metabolic supply themes, the remaining module 2 genes had only gene-level annotation and no clear trait-linking literature, so we specify them as additional candidates for future exploration. Overall, the module 2 interpretation is consistent with known biology of seed filling and envelope-to-seed biomass allocation, providing a plausible but indirect mechanistic hypothesis connecting our phenotypes to metabolic supply and autophagy.

2.3.7.3 Module 3 Interpretation

Since module 3 yielded only sparse gene-level annotation and lacked enrichment and literature support, we did not advance a mechanistic interpretation for it; its genes remain candidates for future analysis.

2.3.7.4 Expression-Based Plausibility Check

We further evaluated the plausibility of our candidate mechanisms by examining gene expression in pennycress leaf, pod, and root tissues using the dataset from [32]. For the purpose of connecting candidate genes to pod phenotypes, we focused on expression in pod tissues. The strongest literature-supported candidate loci were expressed in pods. These included the module 1 candidates *ARF2* (`TAV2_LOCUS7857`),

APC4 (TAV2_LOCUS24640), and *RUS2* (TAV2_LOCUS13944), and the module 2 candidates *ATG9* (TAV2_LOCUS12330), *TKL1* (TAV2_LOCUS16016), and *GPDHc1* (TAV2_LOCUS25599). By contrast, several lower-priority candidate loci were not highly expressed in pod tissues. The *BAG2* orthologs were TAV2_LOCUS6658 and TAV2_LOCUS26197. The *bHLH010* ortholog was TAV2_LOCUS13948; the *bHLH091* orthologs were TAV2_LOCUS13945 and TAV2_LOCUS13946. These lower expression patterns support our focus on the literature-backed mechanisms. The sparse expression of many module 3 paralogs also supports treating that module as a future-work candidate set. Overall, expression results indicate that prioritized candidates are active in pod tissues, supporting their plausibility as candidate mechanisms and providing a basis for future experimental validation.

2.4 Discussion

We found that a custom U-Net provided the most favorable accuracy-efficiency trade-off among the benchmarked CNN, transformer, and foundation model architectures when training and testing on our open-source, hand-annotated ground truth dataset of 347 seed pod images. These results suggest that for small, specialized biological datasets like ours, a task-specific CNN with tailored inductive biases can match or modestly exceed more complex architectures while requiring less computation. However, we note that our small dataset size and tiling scheme may have limited the performance of transformer- and foundation model-based architectures, which rely on large receptive fields and pretraining priors. Future work could explore whether larger datasets and different preprocessing strategies can unlock the potential of these architectures for pennycress phenotyping.

Our pipeline produced 52 biologically plausible, tissue-resolved phenotypes at population scale (12,253 seed pods over 768 accessions). Deep learning methods provided a 30- to 45-fold speedup over manual annotation while achieving 85.58% mIoU. Extracted seed, wing, and envelope areas exhibited a log-normal distribution,

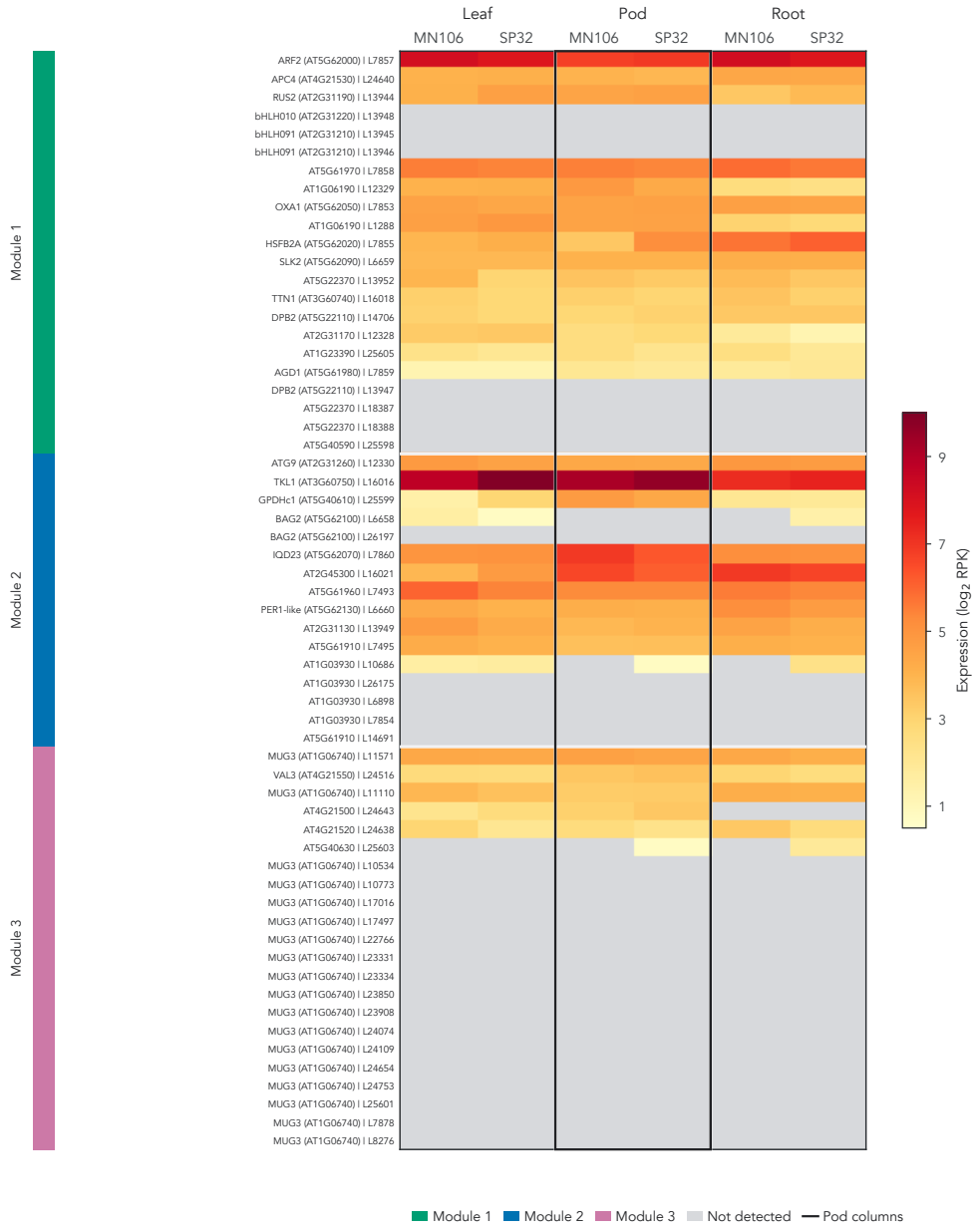

Figure 2.12: Expression of MENTOR-derived candidate loci across pennycress tissues. Rows are pennycress loci grouped by MENTOR module; repeated Arabidopsis orthologs reflect paralogous pennycress loci. Cell color encodes $\log_2$-transformed expression (reads per kilobase) in leaf, pod, and root tissue for MN106 and SP32, with gray cells denoting values below the detection threshold (≤ 0.5). Pod columns are grouped and outlined because the candidate-gene plausibility assessment focuses on pod expression, and bold row labels mark literature-supported candidates emphasized in the text.

wing area was positively correlated with seed area and count, and the resulting PPN separated into coherent color, geometry, and perimeter sub-networks while highlighting cross-tissue pigment and size coupling. These results indicate that our pipeline may enable new avenues for research on pennycress seed pod biology through accurate and fast high-throughput measurements.

Starting from our image-derived pod traits, our pipeline surfaced candidate genes and mechanisms that recover known *Brassicaceae* pod and seed biology. GWAS yielded significant associations for four pod traits, which we mapped to 69 pennycress genes and 64 Arabidopsis orthologs. GRIN narrowed these to 33 network-supported candidates, which MENTOR resolved into three modules; two were related to pod and seed biology. Module 1 was the strongest, centered on auxin-mediated organ growth, reproductive development, and cell division. Its key gene, *ARF2*, offers the clearest candidate bridge to envelope-to-seed area. Module 2 was more indirect: autophagy, nutrient remobilization, and redox balance via *ATG9*, *TKL1*, and *GPDHc1* support a plausible metabolic-supply hypothesis for pod filling and envelope-to-seed allocation. However, these candidates rest on Arabidopsis orthologs, as pennycress network and literature data remain limited.

We compared the MENTOR-IA results to pennycress gene expression data, using the expression heatmap to assess hypothesis plausibility in pod tissues. Prioritized Module 1 and 2 candidates were expressed in pods, while some lower-priority candidates were weakly expressed or not detected, supporting our focus on the literature-backed mechanisms. Furthermore, Module 3 expression is sparse, supporting our decision to treat it as a candidate set for future work. It is important to note that expression was measured across the whole pod and is correlative, so it does not establish causality or eQTL regulation. However, our expression evidence supports prioritization of our literature-backed candidates for future studies.

Despite this progress, our work has some limitations. Collecting and scanning seed pods to use for segmentation is still a labor-intensive process; this could become a bottleneck in future population-scale studies, and it only yielded a small training

set. Additionally, the model's performance is dependent on the quality of label data due to its supervised nature. Accurately annotating seed border pixels was difficult due to the low brightness and contrast levels in image scans, and this limitation was reflected in the lower accuracy for seeds than other surfaces. Furthermore, benchmark compatibility with nnU-Net is imperfect due to its built-in adaptive preprocessing and evaluation pipeline.

Additionally, GWAS results are limited as they used only 189 accessions and retained SNPs with a permissive threshold, although GRIN did help filter candidates further. Due to the limited availability of pennycress data, we relied on Arabidopsis orthologs for module derivation and interpretation, which may not fully capture pennycress biology. Finally, the MENTOR-IA interpretation is exploratory and based on literature evidence from Arabidopsis, so it must be experimentally validated in pennycress to draw strong conclusions.

We present several avenues for future work to extend this pipeline. Improving imaging quality and scanning throughput could yield higher-quality datasets, which can also be expanded through more hand annotation for ground truth. Additionally, performing GWAS on a larger panel of accessions can provide higher-powered association results. Finally, experimental validation of the candidate mechanisms, such as *ARF2*/*APC4*/*RUS2* and *ATG9*/*TKL1*/*GPDHc1*, can test their relevance to pennycress pod phenotypes and provide a deeper understanding of the underlying biology.

This work has several implications for future pennycress research. We found no prior work that extracts intensive phenotypes from pennycress seed pods due to the labor-intensive nature of this task. Therefore, this article provides a novel pipeline for extracting several biologically important phenotypes at the population scale for pennycress and linking them to genetic variants. The framework allows scientists to perform experiments on pennycress that were previously impossible; for example, researchers can examine how treatments and experimental conditions affect entire populations by measuring seed pod phenotypes across treatment groups.

Furthermore, scientists can test hypotheses about phenotypic relationships on a broad scale, leading to higher-impact research. More broadly, this work shows how deep learning-derived phenotypes can be carried into genetic association and module-level interpretation to prioritize table biological hypotheses.

Chapter 3

Deep Learning for Genotype-by-Environment Prediction of Maize Grain Yield

3.1 Introduction

Genotype-by-environment (G×E) interactions are a key mechanism underlying crop stability and performance [121, 164]. Accurately modeling these interactions has important practical implications, including increased food security and optimized farming practices [161]. Because the G×E contribution to grain yield variation is substantial, yield rankings are not stable across environments; a hybrid superior in one site or year may be inferior in another [164, 121]. A breeder who selects on cross-environment mean performance will systematically mis-rank hybrids for the environment of interest. Therefore, predicting within-environment hybrid ordering is critical for improved selection.

An increase in publicly available data has brought both renewed focus and challenge to G×E prediction. The Genomes to Fields (G2F) Initiative curated a public dataset of genotypes and environments, which spans multiple years, hybrids,

and field sites [121]. Each yield observation carries both the genotype and the environmental covariates of the plot it was grown in, so a model can represent G×E interactions directly. The 2024 G2F prediction competition turned these data into a benchmark, standardizing the task with a common training set, a held out target year, and a single evaluation metric [24]. This design makes independently developed methods directly comparable. The competition scored predictions by the Pearson correlation within each environment, averaged across environments, which gives relative ranks priority over absolute scale; the preceding G2F competition had scored on root mean squared error instead [10, 214]. Additionally, since the G2F dataset was collected in real-world field conditions, it contains heterogeneous data sources with varying quality and extensive missingness. By addressing these constraints, models developed on this dataset are more likely to generalize to real-world breeding scenarios.

A variety of methods have been developed for G×E prediction. Early genomic prediction approaches relied on statistical models such as genomic best linear unbiased prediction (GBLUP) [204] and Bayesian linear mixed models [130] to predict breeding values purely from genotypes. These methods were later extended to add genotype-by-environment covariance structure, either through explicit modeling of environmental covariates [34, 68] or kernel-based approaches [90]. Although these methods have proven effective for G×E prediction in some scenarios, they are limited by several inductive biases. The interaction structure must be specified a priori in the case of kernel methods, which prevents flexible modeling of complex relationships. Furthermore, these methods assume a linear relationship between inputs and outputs, which limits the depth of interactions that can be learned. In the case of G×E prediction, this is particularly limiting because the relationship between genotype and environment is often nonlinear [136].

By contrast, deep learning models are a natural extension for multimodal genomic prediction because they can learn nonlinear feature hierarchies from heterogeneous inputs [4, 135]. Prior studies have applied multilayer perceptrons (MLPs) to compute

structured interaction components [134], combined genotype and weather data to predict yield directly [185], and used sequence models, including state-space and transformer architectures, to model marker sequences [172, 46, 218].

However, when a wide range of modeling strategies were evaluated head-to-head in the first public G2F prediction competition, which used the 2022 season as its held out target, statistical and deep learning models both produced satisfactory yield estimates with no family separating decisively from the rest [214]. This result suggests that no single model class guarantees better accuracy on this task. Multimodal architectures for genomic prediction commonly encode each modality with its own encoder and combine the resulting representations late, through concatenation, a weighted sum, or a dedicated fusion layer [135, 185]. Under this paradigm, marker–environment interactions can only be formed after each modality has been processed independently, so the interactions available to the model are limited to those that survive compression. Whether direct interactions between individual marker and environmental tokens improve G×E prediction therefore remains untested.

We designed this study to test whether a unified marker and environment representation could outperform branch-based late fusion for G×E prediction. We trained a model we call MERGE (Marker–Environment Representation for G×E) with the 2024 G2F competition data and encoded marker and environmental tokens in one transformer sequence [24, 206]. A sparse mixture-of-experts (MoE) block supported token-level specialization, and an affine head corrected absolute scale while it preserved within-environment rank. We reviewed 427 run records and selected completed comparisons of fusion architecture, objective choice, and affine calibration. The selected model achieved a macro environment PCC of 0.423 and MSE of 55.40; this result corresponds to third place in the retrospective comparison. Future work will test whether new data and feature representations can close the gap to the competition-winning PCC of 0.437.

3.2 Methods

3.2.1 Data

Data for this model were obtained through the 2024 Genomes to Fields (G2F) Maize Prediction Competition [24]. The dataset included trait, metadata, soil, weather, genotype, and environmental covariate files spanning the 2014-2024 growing seasons, with yield labels for 2014-2023 released prior to the competition, and 2024 labels made publicly available after the competition's end. The training dataset contained 173,960 trait rows across 272 environments; 164,921 of those rows had finite yield. The 2024 test dataset contained 10,057 submission-template rows across 23 environments. Raw genotype data contained 5,899 hybrid records with 2,425 single nucleotide variant (SNV) marker loci. Environmental data contained 654 numeric environmental covariates (ECs), along with 16 daily weather variables and separate metadata and soil tables.

The data were collected across many sites, years, and management regimes, so they are heterogeneous and carry extensive missingness. The competition organizers deliberately released much of the data in raw form to capture the difficulty of modeling real-world data. Extensive preprocessing, including filtering and imputation, was necessary to address inconsistencies and ensure reliable performance. We detail the steps taken to clean our marker and environmental data in Sections 3.2.1.1 and 3.2.1.2 respectively.

3.2.1.1 Marker Data

The raw numerical genotype table contained 5,899 hybrid records and 2,425 candidate SNV loci. The raw marker files stored these states as 0, 0.5, and 1, so at each locus, we encoded alleles by dosage according to the scheme in Table 3.1 to make them compatible with tokenization. A value of 2 represents two major alleles, a value of 1

represents a heterozygous pair of alleles (major-minor), and a value of 0 represents two minor alleles.

Allele Combination	Model Token
Major-Major (AA)	2
Major-Minor (Aa)	1
Minor-Minor (aa)	0

Table 3.1: Genomic input marker encodings. Each diploid marker is encoded as a single token representing the dosage of the major allele by multiplying the raw dosages (0, 0.5, 1) by two.

Marker data preprocessing consisted of filtering and imputation, which we applied to the released all-hybrid genotype table. We removed loci with more than 10% missing values. This process removed 201 loci, leaving 2,224 locus inputs to our final model. Additionally, phenotype rows without a corresponding genotype were dropped, decreasing the training set from 164,921 to 163,026 rows.

For the reported model, missing marker calls were processed sequentially in genotype-file order. For each missing call, we calculated the locus-specific median among genotype records sharing either parent with the target hybrid. Missing loci were replaced with the corresponding median value. Because records were processed sequentially, values imputed for earlier records could contribute to the donor medians used for later records. In total, this procedure filled 104,820 missing calls. After imputation, dosage values were multiplied by two and rounded to the nearest integer to produce model tokens. This conversion affected 2,516 imputed calls whose values fell outside the raw $\{0, 0.5, 1\}$ dosage grid; ordinary heterozygous calls at 0.5 converted exactly to token 1.

3.2.1.2 Environmental Data

In addition to genotypic data, we cleaned environmental data in a multi-step process. First, we focused on the output column, which measured grain yield in megagrams

per hectare (Mg/ha) and served as the prediction target. We excluded all rows which lacked either yield measurements or complete environmental data. Of the 173,960 raw trait rows, 9,039 lacked a finite yield value and were dropped. From the 163,026 rows that retained both a finite yield and a matching genotype record (Section 3.2.1.1), we further removed rows without seasonal weather data, yielding 161,803 records, and rows without EC coverage, yielding 143,050 records. These plot records corresponded to 238 unique environments. Because 34 environments lacked complete weather or EC blocks, they were excluded from the final cohort.

After applying these filters, we performed imputation on the weather station location data included in the competition dataset. For weather stations with partially missing data, we replaced missing observations with the station's mean latitude/longitude coordinates over all existing years. One research station, location TXH4, had no location data present. We used the Texas Tech New Deal Research farm [198] as a proxy to retrieve the coordinates for this station, since both are located in Lubbock, Texas. We used the facility's official website to retrieve the New Deal Research Farm's latitude and longitude.

In addition to imputing weather-station locations, we compressed the daily weather data into 46 summary features. First, we indexed the 16 daily weather variables by environment and date, and dropped five variables to reduce redundant feature signal: three NASA POWER soil-wetness variables (`GWETTOP`, `GWETROOT`, `GWETPROF`) and two shortwave radiation components (`ALLSKY_SFC_PAR_TOT`, `ALLSKY_SFC_SW_DNI`). Dropping these features reduced the daily weather set to 11. We then aggregated variables starting on the planting date, and ending 14 days after harvest for historical environments and on the last date supplied in the seasonal-weather record for 2024 environments. For each retained weather variable, we calculated the minimum, maximum, mean, and accumulated sum over the aggregation window, yielding 44 summary features. We added the starting month and number of included days as additional features.

Following filtering and imputation, we integrated our location, weather, and environmental covariate datasets into a unified set of 705 variables, which consisted of 654 unpruned ECs, 46 weather summaries, two coordinates, and three categorical fields. In the best run, the three categorical fields (`Irrigated`, `Treatment`, and `Previous_Crop`) were excluded, leaving 702 numeric environmental covariates as model inputs. Continuous variables were standardized to zero mean and unit variance, and the scaler was fit only on the training set to prevent data leakage.

We applied the same coverage requirements to the 2024 benchmark rows. Of the 10,057 submission-template rows across 23 environments, 9,486 rows across 22 environments entered the benchmark cohort. The filters removed all 385 rows of environment `SCH1_2024`, so that environment does not appear in the cohort or in any reported benchmark result. The remaining 186 excluded rows came from 13 environments that were retained.

3.2.2 Model Architecture

3.2.2.1 Overview

We developed MERGE, a full-sequence transformer model that predicts maize yield as a function of genotype and environment. The model represents marker dosages and environmental covariates in one token sequence, allowing self-attention to model genotypic, environmental, and G×E structure in a shared representation space. A sparse mixture-of-experts (MoE) feed-forward sublayer [180] adds capacity through token-level expert routing, and a learned `[CLS]` summary token [40] provides the representation used for rank prediction. An environment-conditioned affine-calibration branch converts the rank prediction into the final yield estimate. The architecture is summarized in Figure 3.1.

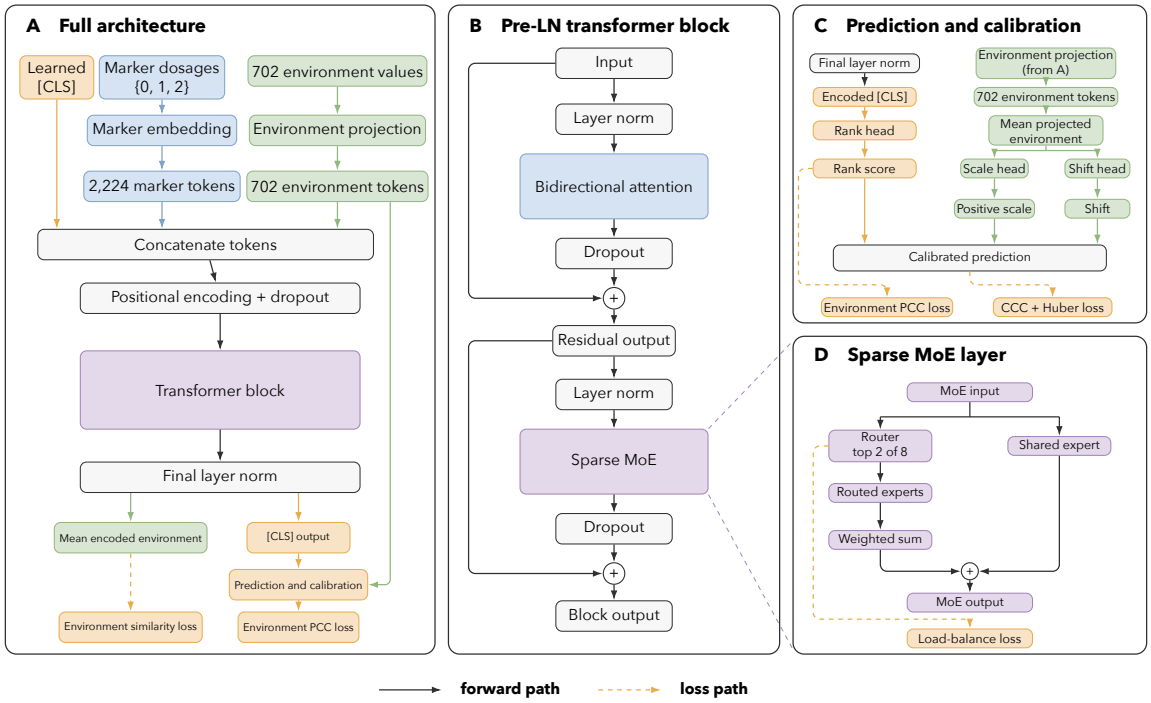

Figure 3.1: MERGE architecture for maize yield prediction. (A) The model embeds 2,224 marker dosages and projects 702 environmental values into one sequence with a learned `[CLS]` token. One transformer block produces the normalized `[CLS]` and environmental representations used by the primary and auxiliary losses. (B) The pre-layer-normalized block applies bidirectional attention and a sparse MoE layer with dropout and residual connections. (C) The rank head predicts relative yield, while the mean projected environmental representation supplies a positive scale and shift for affine calibration. The environment PCC loss acts on the rank score, and the CCC and Huber losses act on the calibrated prediction. (D) The MoE router selects two of eight routed experts per token and combines their weighted output with one shared expert. A load-balance loss discourages routing collapse. Blue blocks show marker and attention operations, green blocks show environment operations, purple blocks show MoE operations, and orange blocks show `[CLS]` and losses. Solid arrows show forward paths, and dashed arrows show loss paths.

3.2.2.2 Input Representation

For the reported model, each sample was represented by a sequence of length 2,927 (Figure 3.1A): one learned `[CLS]` summary token, 2,224 marker tokens, and 702 environmental tokens. Marker dosages in $\{0, 1, 2\}$ were embedded with a shared learned lookup table of vocabulary size three and width 256. The same learned `Linear(1,256)` projection was applied to every standardized environmental scalar, producing one token per environmental feature. The `[CLS]`, marker, and environmental tokens were concatenated in that order, and a static sinusoidal positional encoding [206] was added to the sequence. This produced a per-sample encoder input of shape $2{,}927 \times 256$.

Because yield prediction depends on the complete input instead of autoregressive next-token prediction, no causal attention mask was applied. Bidirectional attention [40] allowed every marker and environmental token to attend to every other position, enabling direct marker–environment interactions within the shared representation.

3.2.2.3 Transformer Encoder with Mixture-of-Experts

The reported MERGE model used one encoder block with hidden width 256 and four bidirectional attention heads (Figure 3.1B). The block followed the pre-layer-normalized residual organization used in GPT-2 [154], with layer normalization applied before each sublayer [5]. The normalized token representations were passed through multi-head self-attention, and dropout with rate 0.15 was applied to the attention output before it was added to the block input through a residual connection [62]. The updated token representations were normalized again and passed through the MoE sublayer, whose output received the same dropout before the second residual addition.

The reported encoder replaced the conventional dense feed-forward sublayer with a sparse MoE layer [180] (Figure 3.1D). A learned linear router produced eight logits, one for each routed expert, for every sequence token, including `[CLS]`, marker, and

environmental tokens. The two routed experts with the highest logits were selected independently for each token, and a softmax over those logits produced weights used to combine their outputs. Each routed expert was a two-layer FFN with hidden width 256, GELU activation, and output width 256. An additional shared expert with the same structure processed every token, and its output was added to the weighted sum of routed-expert outputs. During training, an auxiliary load-balancing loss with weight 0.01 discouraged routing collapse by encouraging routing mass to remain distributed across the eight routed experts. This design increased conditional capacity while activating only two routed experts, in addition to the shared expert, for each token.

3.2.2.4 Prediction and Calibration Heads

After the final transformer block, layer normalization was applied to the full sequence (Figure 3.1C). The normalized representation at the `[CLS]` position was passed through a `Linear(256,1)` head to produce a scalar rank prediction. This rank head was optimized directly against the environment PCC loss described in Section 3.2.3.

The selected model additionally applied an environment-conditioned affine calibration to transform the rank prediction into an absolute-yield prediction. For each sample, we took the mean of its 702 projected environmental input tokens, computed before positional encoding and transformer processing. Every plot record within one environment carried the same environmental covariates, so this mean was constant within an environment; we therefore write it as $\bar{\mathbf{z}}_E$ for environment E. Two learned `Linear(256,1)` heads, f_s and f_b, mapped $\bar{\mathbf{z}}_E$ to a scalar raw-scale value and a scalar shift, respectively. The calibrated total prediction was computed as

$$\begin{aligned} s_E^{\text{raw}} &= f_s(\bar{\mathbf{z}}_E), & b_E &= f_b(\bar{\mathbf{z}}_E), \quad f_s, f_b : \mathbb{R}^{256} \to \mathbb{R}, \\ \hat{y}_{\text{total}} &= \left[\text{softplus}(s_E^{\text{raw}}) + 10^{-4}\right] \hat{y}_{\text{rank}} + b_E. \end{aligned} \tag{3.1}$$

The weights of both calibration heads and the bias of the shift head were initialized to zero. The scale-head bias was initialized to the inverse softplus of one, so the initial scale was approximately 1.0001 after adding the 10^{-4} offset, while the initial shift was zero. The softplus activation and offset constrained the scale to be strictly positive. Every hybrid in the same environment shared the same positive scale and shift, so calibration preserved both the within-environment rank ordering and Pearson correlation, leaving the macro environment PCC unchanged while allowing adjustment of absolute yield scale.

The model also exposed the mean of the encoded environmental tokens after the transformer and final normalization for use in the auxiliary environment-similarity loss (Section 3.2.3.1). This pooled representation was not itself a prediction head; it shaped the learned environmental representation as described below.

3.2.3 Training

3.2.3.1 Training Objective

The training objective was built around an environment PCC loss designed to mirror the competition's macro environment PCC metric. Within each local per-GPU microbatch, we computed the Pearson correlation between observed yield and the model's rank prediction for every environment $e \in \mathcal{E}_B$ containing at least four observations. Here, $\hat{y}_i$ denotes the rank prediction for sample i. Each GPU computed correlations from its local microbatch of 256 samples, and the sufficient statistics were not pooled across GPUs. The raw environment correlations were uniformly averaged without Fisher z transformation or sample-count weighting. The environment PCC loss was defined as

$$r_e = \frac{\sum_{i \in e}(y_i - \bar{y}_e)(\hat{y}_i - \bar{\hat{y}}_e)}{\sqrt{\sum_{i \in e}(y_i - \bar{y}_e)^2}\sqrt{\sum_{i \in e}(\hat{y}_i - \bar{\hat{y}}_e)^2}}. \tag{3.2}$$

$$L_{\text{envpcc}} = 1 - \frac{1}{|\mathcal{E}_B|} \sum_{e \in \mathcal{E}_B} r_e. \tag{3.3}$$

In addition to optimizing within-environment ranking, we implemented an auxiliary environment-similarity objective to align internal environmental representations with the raw environmental data. We mean-pooled the encoded environmental tokens into an embedding $\mathbf{z}_i$ for each sample i. The cosine similarity between $\mathbf{z}_i$ and $\mathbf{z}_j$ defined the learned similarity S^z_{ij}, while the cosine similarity between their standardized raw environmental vectors defined the target similarity S^E_{ij}. We computed the loss as the mean-squared difference between these similarities over all off-diagonal pairs in a local microbatch of size B:

$$L_{\text{env-sim}} = \frac{1}{B(B-1)} \sum_{\substack{1 \le i,j \le B \\ i \ne j}} (S^z_{ij} - S^E_{ij})^2. \tag{3.4}$$

The maximum weight of this loss was 0.1. The term was inactive until epoch 50 and then ramped linearly to its maximum weight by epoch 100.

For absolute-yield calibration, the calibrated total prediction was optimized with an environment concordance correlation coefficient (CCC) loss [116] and a mean Huber loss [82]. For each environment e, the concordance correlation was

$$\begin{aligned} \rho_{c,e} &= \frac{2 \, \operatorname{cov}\left(y_e, \hat{y}^{\text{total}}_e\right)}{\sigma^2_{y,e} + \sigma^2_{\hat{y}^{\text{total}},e} + \left(\mu_{y,e} - \mu_{\hat{y}^{\text{total}},e}\right)^2}, \\ L_{\text{envCCC}} &= 1 - \frac{1}{|\mathcal{E}^{(2)}_B|} \sum_{e \in \mathcal{E}^{(2)}_B} \rho_{c,e}, \end{aligned} \tag{3.5}$$

where $\mathcal{E}^{(2)}_B$ contained environments in the local microbatch with at least two observations and nonzero yield and prediction variance. The environment CCC loss had maximum weight 0.05. The Huber loss used mean reduction, $\delta = 1.0$, and maximum weight 0.02. Both calibration terms were inactive until epoch 150 and ramped linearly to their full weights by epoch 250. Until epoch 250, the rank

prediction was detached from calibration gradients, allowing the calibration heads to learn without altering the rank head. Beginning at epoch 250, calibration gradients reached the rank prediction with a weight of 0.1, while the environment PCC loss on the rank prediction remained fully active throughout training.

Combining all of these objectives, the total loss function was defined as

$$L(t) = L_{\text{envpcc}} + 0.1\, r_E(t) L_{\text{env-sim}} + r_C(t)\,[0.05 L_{\text{envCCC}} + 0.02 L_{\text{Huber}}] + 0.01 L_{\text{MoE}}, \quad (3.6)$$

where $r_E(t)$ increased linearly from zero to one between epochs 50 and 100, and $r_C(t)$ increased linearly from zero to one between epochs 150 and 250. The final term was the MoE load-balancing loss described above.

3.2.3.2 Training Details

The reported model contained 1,552,259 trainable parameters. We trained it on Oak Ridge National Laboratory's Frontier supercomputer using distributed data parallel training across four compute nodes, with eight GPU devices per node. Training used automatic mixed precision with the `bfloat16` data type. Each GPU processed a microbatch of 256 samples, yielding a global batch size of $4 \times 8 \times 256 = 8192$. The reported run reached early stopping after 53 minutes and 47 seconds of wall-clock time, which corresponds to approximately 29 GPU-hours.

We used the AdamW optimizer [122] with weight decay 1×10^{-5}. The learning rate increased linearly to 1×10^{-4} during an approximately five-epoch warmup and then followed cosine decay to a floor of 1×10^{-5} over a 400-epoch horizon, defined as $\min(2 \times 200, 3000)$. Training was configured for at most 3000 epochs and stopped early after 200 epochs without improvement in the validation criterion. The model used dropout 0.15, four attention heads, embedding dimension 256, one transformer block, and base model and split seeds of 1.

Because the primary objective computed within-environment correlations, we used environment-stratified minibatches with at least 32 samples per represented environment when possible. As described above, correlations were computed within each 256-sample per-GPU microbatch rather than across the global batch. For leave-environment-out validation, we randomly held out 15% of entire pre-2024 environments from model fitting. Checkpoints and early stopping were determined by the macro environment PCC of the rank prediction on this validation subset, with validation loss used to break ties. The 2024 data were excluded from model fitting and checkpoint selection and were subsequently used as a retrospective competition benchmark.

3.2.4 Evaluation

We used the 2024 evaluation data from the G2F competition [24] as our retrospective competition benchmark. The competition metric was the macro environment PCC between predicted and observed yield across the 2024 benchmark environments [10]. We retained only rows with finite observed and predicted yields. An environment was eligible for PCC evaluation if it contained at least two observations and both its observed and predicted values were nonconstant. We computed the Pearson correlation within each eligible environment and averaged these correlations uniformly without weighting by environment size. Calibrated total predictions were used to compute the primary reported PCC; rank-head PCC was also calculated for comparison and was equal up to numerical precision because the calibration head applied a constant positive affine transformation within each environment. The evaluation procedure is summarized in Algorithm 1.

In addition to the macro environment PCC evaluation, we implemented a macro environment MSE evaluation to assess absolute-yield calibration. We defined the macro environment MSE as

Algorithm 1 Macro Environment PCC Evaluation

for each eligible environment $e \in \mathcal{E}_{2024}^{\text{valid}}$ in the retrospective benchmark **do**
  $y_e \leftarrow$ observed yields for e
  $\hat{y}_e^{\text{total}} \leftarrow$ calibrated total predictions for e
  $PCC_e \leftarrow \text{Pearson}(y_e, \hat{y}_e^{\text{total}})$
end for
$PCC_{\text{macro}} \leftarrow \frac{1}{|\mathcal{E}_{2024}^{\text{valid}}|} \sum_{e \in \mathcal{E}_{2024}^{\text{valid}}} PCC_e$

$$\begin{aligned} \text{MSE}_e &= \frac{1}{n_e} \sum_{i \in \mathcal{I}_e} \left(y_i - \hat{y}_i^{\text{total}}\right)^2, \\ \text{MSE}_{\text{macro}} &= \frac{1}{|\mathcal{E}_{2024}^{\text{MSE}}|} \sum_{e \in \mathcal{E}_{2024}^{\text{MSE}}} \text{MSE}_e, \end{aligned} \tag{3.7}$$

where $\mathcal{I}_e$ is the set of finite observed-prediction pairs in the environment e, n_e is the number of such pairs, $\mathcal{E}_{2024}^{\text{MSE}}$ is the set of environments containing at least one finite pair, and $\hat{y}_i^{\text{total}}$ is the calibrated total prediction for sample i. In our reported results, $\text{MSE}_{\text{macro}}$ is the macro environment MSE stated alongside the macro environment PCC.

3.2.5 Model Development Comparisons

To evaluate the effectiveness of critical architectural and training choices, we examined run records from the model-development process. We highlight three comparisons: unified versus branch-based processing of markers and environmental covariates, environment PCC versus global MSE training objectives, and uncalibrated versus affine-calibrated predictions. Each comparison was performed on the same 2024 retrospective benchmark and used the same evaluation procedure. Within each comparison, we held the principal settings constant. All six runs used one transformer block, four attention heads, embedding dimension 256, a mixture-of-experts feed-forward layer with eight routed experts, top-2 routing and one shared expert, dropout 0.15, global batch size 8192, microbatch size 256, environment-stratified batching, LEO validation, and the learning rate schedule described in Section 3.2.3. The three

comparisons were assembled at different stages of development, however, and both the auxiliary losses and the checkpoint-selection criterion changed between those stages. Comparisons therefore hold within a pair and not across pairs.

The first comparison evaluated the performance of a unified transformer block that processed marker and environmental tokens together (Section 3.2.2) against a branch-based late-fusion model that processed markers and environmental covariates separately before it combined their representations. The branch-based late-fusion model applied a transformer encoder and a parallel LD-oriented CNN to the marker sequence, and an MLP to the environmental covariates. The environment representation was appended to the marker token sequence as a single token, the LD representation was aligned to that sequence and added elementwise, and a layer normalization combined them. A final G×E transformer block processed the fused sequence, and a linear head read the rank prediction from the `[CLS]` position. Figure 3.2 shows this architecture. The branch-based late-fusion model contained 2,886,404 trainable parameters. These divided into 1,452,288 for the marker encoder, 246,784 for the environment encoder, 396,288 for the LD encoder, and 791,044 for the G×E encoder. The unified MERGE model contained about half as many parameters, so this comparison did not favor the unified model on capacity. At this stage in development, neither model used the environment-similarity auxiliary loss or the affine calibration branch. Both runs in this pair also selected checkpoints on a target-weighted validation score, which reweighted validation locations toward the 2024 site distribution, stabilized each per-location correlation with a Fisher z transformation, weighted it by sample count, and took a pessimistic bootstrap quantile across locations.

The remaining two comparisons came from a later stage of development. Both used the environment-similarity auxiliary loss at weight 0.1 and selected checkpoints on the macro environment PCC of the rank prediction. After comparing unified and branch-based architectures, we examined the effect of training objective choice. We trained two MERGE controls (Section 3.2.2) with identical architectures and principal

hyperparameters, but optimized one model with the environment PCC loss defined in Section 3.2.3.1 and the other with a global MSE loss.

Finally, we compared the effect of the affine calibration head on absolute yield prediction. The reported MERGE model used the environment PCC loss together with the affine calibration branch, while its control used the environment PCC loss alone. These two runs came from adjacent development commits, so this pair is near-matched, not exactly matched.

Two runs of the environment PCC configuration completed at the same commit under identical settings. We report both and treat their difference as an estimate of run-to-run variation. Every other comparison arm is a single run, so this study does not yet estimate variation across random initializations. The complete experiment export contained 427 run records, of which 374 finished successfully. The accompanying machine-readable manifest records every exported run, its provenance, completion status, reported metrics, and manuscript disposition. Only completed comparisons from the legacy data scheme entered the primary results; experiments that used the later data-generation or tokenization schemes are designated as future work.

3.3 Results

3.3.1 Performance on the 2024 Retrospective Benchmark

On the 2024 retrospective benchmark, the reported affine-calibrated MERGE model achieved a macro environment PCC of 0.423 and a macro environment MSE of 55.40. These metrics came from the calibrated total prediction of the checkpoint selected by pre-2024 LEO validation. The result came from one run using seed 1, so no across-seed uncertainty estimate was available.

3.3.2 Internal Model Comparisons

We evaluated architectural and training variants during model development to determine how performance varied across design choices. Section 3.2.5 defines the three comparisons and the settings that differ between them. In the architecture comparison, the unified MERGE control achieved a macro environment PCC of 0.411 and a macro environment MSE of 96.34, whereas the branch-based late-fusion model achieved a macro environment PCC of 0.308 and a macro environment MSE of 116.47. These results indicate that unified processing of markers and environmental covariates was associated with better within-environment correlation and lower absolute error in this comparison.

Next, we examined how matching the primary training objective to the competition's evaluation metric affected performance. Both models were evaluated with macro environment PCC and macro environment MSE, although one optimized global MSE during training. The model trained with the environment PCC loss achieved a macro environment PCC of 0.423 and a macro environment MSE of 100.57. The model trained with the global MSE loss achieved a macro environment PCC of 0.305 and a macro environment MSE of 14.54. These results indicate that optimizing within-environment correlation substantially improved macro environment PCC but increased macro environment MSE, suggesting that the resulting predictions were less well-calibrated for absolute yield.

Motivated by this rank–scale trade-off, we compared a near-matched uncalibrated rank model with the reported affine-calibrated model. Both models achieved a macro environment PCC of 0.423, but their macro environment MSE differed: 101.06 for the uncalibrated model and 55.40 for the affine-calibrated model. Calibration improved macro environment MSE by approximately 45.2%, while macro environment PCC changed by only 0.00004. This behavior was expected because an environment-specific affine transformation with a positive scale preserves within-environment Pearson

correlation while allowing the absolute scale and offset of the predictions to be adjusted.

The replicate pair described in Section 3.2.5 bounds how much of this variation is run-to-run noise. Its two runs agreed to three decimals on macro environment PCC at 0.423, and gave macro environment MSE values of 100.57 and 101.06. These differences, 0.0000039 in PCC and 0.50 in MSE, provide the study's only estimate of run-to-run variation. On that scale, the 45.66 MSE improvement from calibration is about 90 times the replicate difference. The corresponding PCC difference of 0.0000378 amounts to 0.009% of the score. Every other comparison arm is a single run. These results should therefore be interpreted as descriptive model-development comparisons rather than replicated estimates of individual-component effects. Table 3.2 reports the highlighted comparisons at full precision.

Comparison	Variant	Macro environment PCC	Macro environment MSE
Architecture	Branch-based late-fusion model	0.30819	116.46976
	Unified MERGE control	0.41066	96.33830
Training objective	Global MSE	0.30504	14.54242
	Environment PCC	0.42316	100.56502
Calibration	Uncalibrated rank model	0.42317	101.06046
	Affine-calibrated model	0.42321	55.40464

Table 3.2: Highlighted internal model-development comparisons on the retrospective 2024 benchmark. Comparisons should be made within the three labeled groups. Each row reports a single run. The architecture and objective pairs used matched principal settings; the calibration pair used near-matched runs from adjacent development commits. Values are given to five decimals so that the small differences discussed in the text remain visible. The complete run-level inventory is provided in `supplement/ablation_run_manifest.csv`.

3.3.3 Performance Across Benchmark Environments

To better understand the model's performance across environments, we examined variation in PCC among the 2024 benchmark environments (Figure 3.3). All 22 environments in the benchmark cohort met the eligibility criteria described in the

Evaluation section and yielded defined PCC values. Across these environments, the model achieved a mean PCC of 0.423 and a median PCC of 0.434. The interquartile range of environment-level PCC values was 0.364–0.480, and the full range was 0.247–0.567. The model achieved positive PCC in all 22 environments, with 16 of 22 (72.7%) having PCC greater than 0.40. Thus, performance was consistently positive but varied meaningfully among benchmark environments.

We also examined the upper and lower ends of the environment-level PCC distribution. The environment with the highest PCC was `IAH4_2024`, with a PCC of 0.567. The second-highest PCC occurred in `OHH1_2024`, with a PCC of 0.532, and the third-highest occurred in `MOH1_2024`, with a PCC of 0.522. In contrast, the environment with the lowest PCC was `NYH3_2024`, with a PCC of 0.247. The second-lowest was `IAH2_2024`, with a PCC of 0.304, and the third-lowest was `GAH1_2024`, with a PCC of 0.306. The difference between the highest and lowest environment-level PCC values was 0.319, demonstrating considerable heterogeneity in model performance across environments.

3.3.4 Competition Context

To contextualize the reported MERGE result, we compared it with the final competition rankings [10]. The competition ranked submissions by the average within-environment Pearson correlation, the same primary metric used in our retrospective benchmark. Entries that withdrew, were disqualified, or were entered as non-competing benchmarks are excluded from those rankings. MERGE achieved a PCC of 0.423, which would have placed third among the qualified entries shown in Table 3.3, 0.003 below the second-place score and 0.014 below the winning score. Because MERGE was evaluated after the competition and was not submitted, this placement is a retrospective comparison, not an official result.* Its proximity to the

*MERGE was developed after the competition closed and was not the model our group submitted. Our official entry was the ORNL CompBio team, which placed 7th using an ensemble model that included an earlier version of MERGE. Official entries are reported in [10].

second-place score nevertheless indicates that the architecture was competitive with the leading competition entries.

Rank	Entry	Competition PCC
1	PARaBra	0.437
2	The Cornquerors	0.426
(3)	MERGE (retrospective)	0.423
3	GxE4GoodY	0.414
4	LisbonBio	0.414
5	G2Amours(G_et_E)	0.405

Table 3.3: Comparison of the MERGE result with the five leading qualified entries in the 2024 G2F G×E Prediction Competition [10]. Entries that withdrew, were disqualified, or were entered as non-competing benchmarks are excluded. The MERGE score is a retrospective comparison and was not an official competition submission; its rank is given in parentheses to mark the position the score would have taken.

3.4 Discussion

This study tested whether a unified representation of marker and environmental tokens could support maize G×E yield prediction more effectively than branch-based late fusion. Our results support early cross-modal interaction as a useful design choice and show that prediction rank and absolute scale require distinct treatment. The final MERGE model combined these choices and achieved a competitive retrospective macro environment PCC of 0.423 on the 2024 competition benchmark.

Unlike a late-fusion architecture, MERGE places marker and environmental tokens in one sequence before either modality enters a separate encoder. This design gives each marker token a direct attention connection to every environmental token; thus, the model can form cross-modal representations before separate encoders can discard information needed for later interactions. Prior multimodal G×E methods often use separate encoders and late fusion [135, 185], so the comparison addresses a common design choice. The higher PCC and lower MSE of the unified control were consistent with this rationale. The unified model also reached the higher score with about half

the trainable parameters of the branch-based late-fusion model, so capacity does not explain the difference. However, one seed and several component differences prevent a causal attribution to early fusion alone.

Objective choice exposed a practical trade-off between within-environment selection and absolute yield prediction. Macro environment PCC emphasizes whether the model preserves relative yield variation within each environment. By contrast, macro environment MSE penalizes errors in yield units, including errors in scale and offset. These metrics therefore support different uses: PCC supports hybrid selection within a trial, whereas MSE supports yield estimates on an absolute scale. The environment-conditioned affine head recovered part of the scale gap through a positive scale and shift that preserved within-environment PCC. In the near-matched comparison, this calibration reduced macro environment MSE by 45.2%, which supports separate correlation and scale targets. However, the calibrated model's macro environment MSE was still 3.8× higher than that of the global MSE-trained model. That model in turn reached a macro environment PCC of only 0.305, 0.118 below the calibrated model, so it would rank hybrids poorly within a trial. Calibration therefore narrowed the trade-off but did not completely remove it.

The mean benchmark PCC concealed substantial variation among the eligible 2024 environments. Although every environment had positive PCC, the model did not provide uniform reliability across environments. This variation could reflect differences in environmental complexity, data quality, sample composition, or other unmeasured factors, but this study did not test these explanations. These results show that model assessment requires both aggregate and environment-level metrics.

The retrospective placement below the two leading entries supports competitiveness, but it does not constitute an official competition result. The winning entry was a multivariate mixed linear model with a Gaussian genomic kernel and no environmental covariates [10], so the leading score came from a method that models neither environment nor nonlinearity in the way MERGE does. Both G2F competitions

found that diverse modeling strategies deliver comparable estimates [214, 10], so the MERGE score establishes method viability, not architecture superiority.

3.4.1 Limitations and Future Work

Each highlighted comparison used one seed, so the study could not estimate variation across random initialization. Future work will evaluate multiple seeds to quantify variation across model initializations.

The architecture comparison matched principal training settings, but its two designs differed in several components. The calibration comparison also used near-matched runs from adjacent commits. These comparisons therefore provide descriptive evidence, but not causal effects. Future work will use controlled single-component ablations across multiple seeds. Additionally, the current study reports no classical genomic prediction baseline. Future work will compare MERGE with linear mixed models, GBLUP, and other genomic prediction methods under the same standardized protocol.

The study used the 2024 labels for several model comparisons, so the benchmark was not an untouched external test set. The reported scores therefore describe retrospective benchmark performance, not a prospective competition submission. After method development, we will evaluate MERGE on a future G2F season or another untouched benchmark.

Leave-environment-out validation holds out whole environments from the pre-2024 years, so it measures spatial interpolation for hybrids that the model has already seen. The 2024 benchmark posed a different problem. All 22 of its environments were new to the model, and 87.6% of its hybrids were crosses absent from the training data. We therefore selected the reported checkpoint with a criterion that does not completely track benchmark performance. An improvement measured under leave-environment-out validation need not transfer to novel crosses in a novel year. Future work will

build a validation design that reproduces the novel-cross and novel-year structure of the benchmark.

The tester composition of the benchmark was also highly asymmetric. PHP02 accounted for 5,119 of the 9,486 rows and LH287 for 3,716, which left 651 rows for all other testers. The reported scores therefore describe performance on crosses to two tester backgrounds. They provide little evidence about hybrids with other testers.

Access to compute constrained the design of the study. Training the reported model required 32 GPUs across four nodes of a shared supercomputer, and every run depended on an allocation award and on queue scheduling. This constraint is the practical reason the highlighted comparisons use single runs rather than replicated seeds. The reported result also rests on one checkpoint and its captured run configuration. We will archive that checkpoint, its configuration, the benchmark predictions, and the metric output with the manuscript.

In addition to the future work that is necessary to address the study's limitations, we outline several directions for further model improvement. First, we plan to evaluate the new reproducible data pipeline and feature tokenization options. The pipeline creates traceable and repeatable data from the raw competition files, including fixed imputation, filters, feature order, and data splits. This structure will also support controlled tests of alternative preprocessing choices. The tokenization options will support feature-aware representations for temperature, precipitation, soil, management, and other environmental covariates.

Furthermore, we plan to add temporal detail to the weather representation. The current model uses seasonal weather summaries, which can hide the date and crop stage of a weather event. Instead, we plan to test a weather encoder that can process daily weather sequences from planting to harvest. This approach will align weather features by days after planting instead of calendar dates. Development measures, such as growing degree days, could provide a closer link to crop stage. A transformer, RNN, or LSTM will convert the daily sequence into one or more weather tokens.

The model can then use attention to combine these tokens with soil, management, and marker tokens. We will compare seasonal weather summaries, temporal weather tokens, and their combination for G×E prediction.

In addition to improving data and environmental representations, we plan to test whether external parental representations improve predictions for unseen hybrids. Currently, the model uses a shared learned dosage embedding across markers, with positional codes that identify loci. We plan to evaluate external parental information, such as parent genotypes, pedigree or relationship data, and known tester identity. Parent genotype data may overlap with hybrid marker data, but direct parental representations could expose inheritance and relationships more clearly.

We will compare hybrid markers alone with parent identity tokens, parent genotype representations, pedigree data, and combining-ability features. We will calculate any combining-ability features from the fit partition only to prevent target leakage. We will report performance on novel crosses, known crosses, and major testers.

Two further directions follow from the results reported above. The affine calibration head was trained as a low-weight auxiliary objective, so we will test whether a stronger calibration objective reduces macro environment MSE further without harming macro environment PCC. We will also test whether environmental complexity, data quality, or sample composition explain the variation in environment-level PCC, and whether environment-specific adaptation or calibrated uncertainty estimates improve reliability across environments.

All future comparisons will use multiple paired seeds and report mean scores, standard deviations, and paired differences to measure effects beyond seed variation.

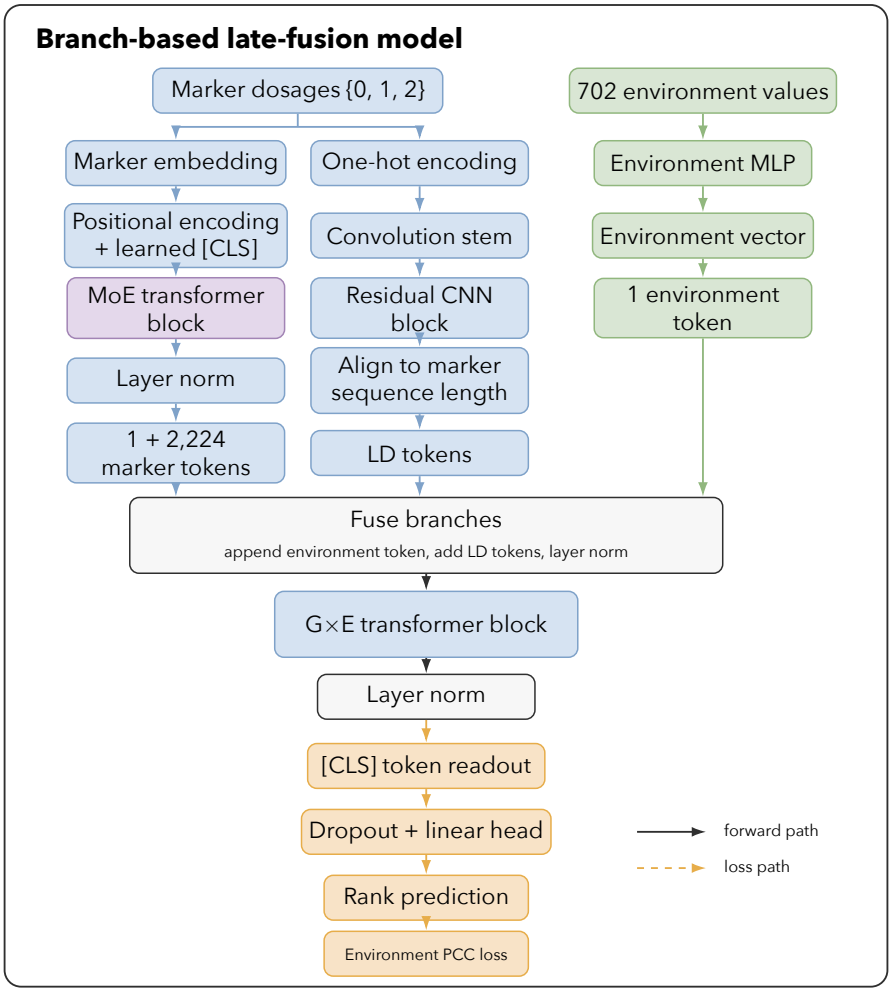

Figure 3.2: Branch-based late-fusion model used in the architecture comparison. Marker dosages enter a transformer encoder with a mixture-of-experts feed-forward layer and, in parallel, a one-hot convolutional encoder that captures local linkage-disequilibrium structure. The 702 environmental values enter a multilayer perceptron that produces a single environment token. The environment token is appended to the marker token sequence, the aligned LD representation is added elementwise, and a layer normalization combines them. One G×E transformer block then processes the fused sequence, and a linear head reads the rank prediction from the `[CLS]` position. Blue blocks show marker operations, green blocks show environment operations, purple blocks show mixture-of-experts operations, and orange blocks show `[CLS]` and loss operations.

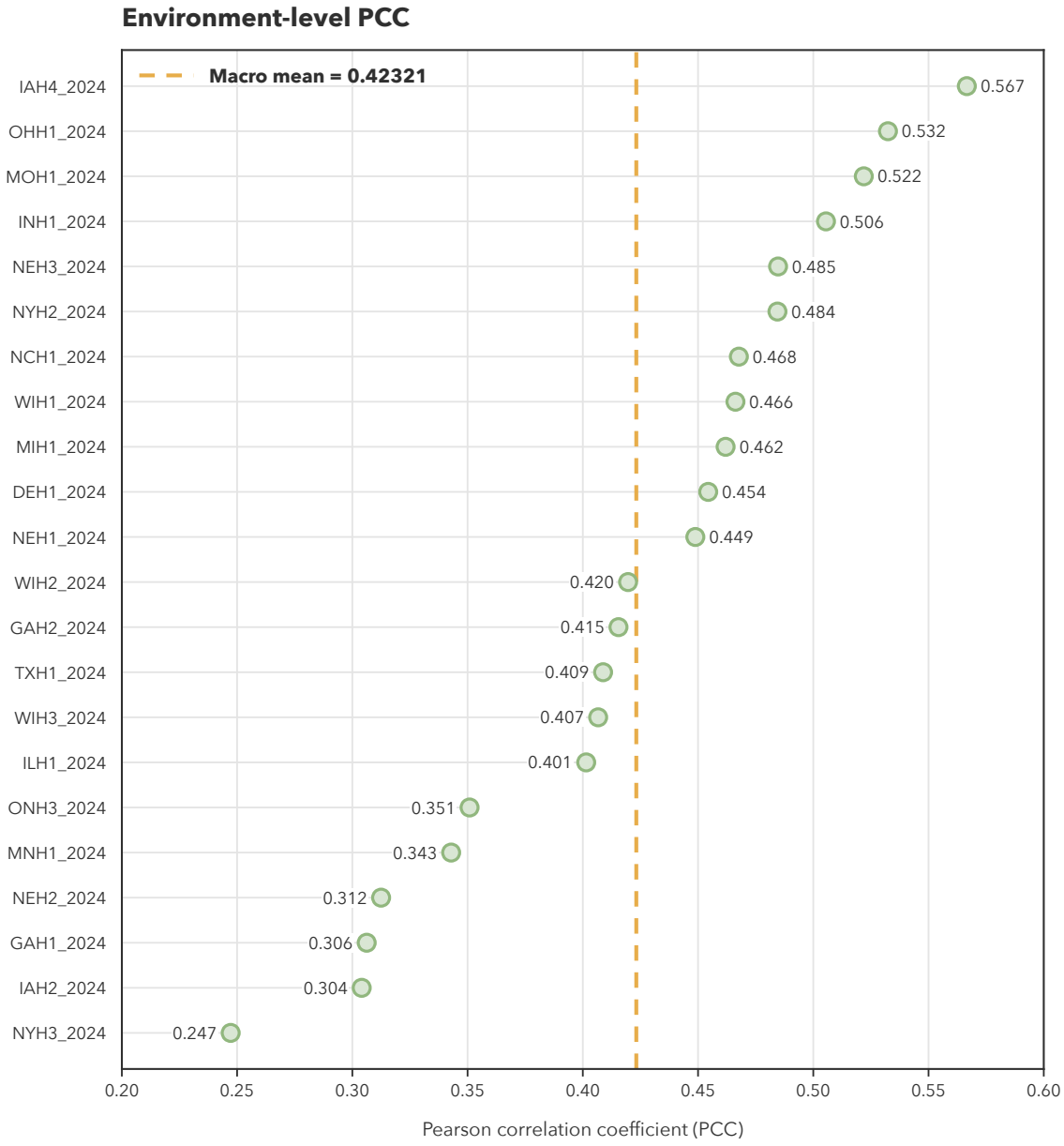

Figure 3.3: Environment-level Pearson correlation coefficients (PCCs) for the selected MERGE checkpoint on the 2024 retrospective benchmark. Green points show PCC for each of the 22 environments in the benchmark cohort, ordered from highest to lowest. The dashed orange line marks the macro environment PCC of 0.423. Point labels show values rounded to three decimals.

Chapter 4

MENTOR-RL: An Open-Weight Model for Mechanistic Interpretation

4.1 Introduction

Mechanistic interpretation identifies relationships among molecular entities such as genes, proteins, and RNA. Systems biology studies their interactions to find *modules*, or groups of functionally related entities, and characterize the mechanisms underlying their behavior [61, 8]. Because these interactions span scales and regulatory layers, modern analysis uses multiplex networks with the same genes connected by different edge types [7, 138, 38] and propagation methods such as random walk with restart (RWR) to identify functional neighborhoods [103, 33]. Such analyses support disease-mechanism discovery, drug-target prioritization, and polygenic-phenotype interpretation [128]. These methods build on one another: iterative random forest constructs predictive networks from data [9, 30], diffusion identifies functional neighborhoods within networks, and MENTOR [191] organizes the resulting distances into a module hierarchy. Despite providing a structured view of the interactome, these methods still produce modules and candidate lists that are too large to explore systematically.

Large language models (LLMs) can synthesize large amounts of information, and several techniques make them suitable for mechanistic exploration. Intermediate reasoning, such as chain-of-thought and tree-of-thoughts, improves reliability [215, 104], while external tool use provides deterministic verification and provenance [171, 221]. Applying explicit self-verification to these steps has been shown to further improve factuality [125, 183]. Applied to biology, such techniques have equipped models with gene-function embeddings [26], database-supported self-verifying pipelines [213], and multi-agent systems for mechanistic synthesis [208, 123]. However, these systems reach that behavior by using frontier-model inference, so their cost grows with exploration length and reasoning depth. Furthermore, none are trained end-to-end as a policy over verifiable rewards.

Reinforcement learning supplies the machinery to close the gap between frontier and open-weight models. Direct preference optimization (DPO) aligns a policy with pairwise judgments [29, 146, 156], while policy-gradient methods such as proximal policy optimization and group relative policy optimization (GRPO) optimize scalar rewards over complete rollouts [174, 179]. Verifiable rewards can elicit long-horizon reasoning [60], and related methods train tool-using agents for browsing, APIs, and multistep web tasks [140, 152, 151]. Although these methods have shown promise for a variety of use cases, they have not yet been applied to a policy grounded in network-derived biological targets.

Current systems also treat networks as static objects to query and encode rather than learning the rules they obey, limiting generalization to new cell types, tissues, or diseases. Sequence models trained on synthetic tasks can instead form internal, causal models of a generating process and generalize to unseen states [113]. These compact, recurring rules can be parameterized by model weights, even though the large multiplex structures themselves are too large. Thus, it is possible for a language model to learn network structure while keeping biological edges, walk scores, and distances in a deterministic runtime. Learning such a world model also requires

dense sampling of the generation process, since fixed templates over a single network can be satisfied through memorization.

We therefore propose MENTOR-RL, an open-weight model trained on a structured lattice of multiplex networks. In every context, relevant predictive expression network (PEN) layers define the gene universe, the remaining layers are induced on that universe, and all layers are stacked. Varying the cell type, tissue, or brain region changes the network while holding this construction rule fixed. This *multiplex lattice* supplies the dense sampling that a single network cannot. Across contexts, the operator that maps layer composition to structure stays fixed. We expect hierarchies to differ across contexts; this divergence is biological signal, and recovering it tests whether the model learned the operator.

However, a hierarchy alone cannot serve as the world model, because it is a lossy abstraction of the multiplex. Two clades far apart in a tree may still be joined by short, biologically supported paths through bridge nodes, which is why a signal such as drug-target enrichment can appear in distant branches but act through a single connected system. We therefore train the model on network structure itself, so it can reason about the cross-clade connectivity that hierarchy suppresses. The hierarchy is used to prioritize candidates, while the network is used to confirm them.

This proposal pursues four objectives. First, we define an ontology-grounded multiplex lattice, serving as a training substrate for supervised fine-tuning (SFT). Next, we propose to train a world model on three structural views using typed, runtime-validated targets. After world model training, we plan to post-train an exploration agent via reinforcement learning with deterministic rewards. Finally, we will test the world model claim through two flagships: hierarchy generation for an unseen context, and projection of a seed neighborhood onto the hierarchy followed by network-connectivity recovery. We employ activation probes to test whether the recovered structure is causal. Together these objectives aim to make mechanistic discovery broader, faster, and more context-aware.

4.2 Proposed Methods

4.2.1 Multiplex Lattice

In this section, we define the multiplex lattice that serves as the substrate for our mechanistic world model. The lattice comprises multiplex networks in which entities such as genes and proteins are nodes and their context-specific interactions are edges, with each interaction type being represented in a different layer. For each context, we take every available PEN layer and define its *gene universe* as the union of genes contained in those layers. The remaining non-PEN layers are then *induced* on that universe, restricting them to genes present in the PEN layers and the edges among them. Finally, all layers in the context are *stacked* to form a multiplex network that captures multilayer interactions among genes. Thus, there is no single shared backbone across the lattice; even layers drawn from a common source become context-specific once induced. Two sampled points in the lattice may contain different nodes, edges, and therefore hierarchies. For the scope of this proposal, we build the lattice on the brain, for which we hold a pre-existing whole-brain multiplex assembled by the same rule. The brain multiplex combines PEN layers over the brain gene universe with a HumanNet V3 backbone [99] induced on it.

The lattice comprises a tiered structure that captures the relationships between different contexts. We visualize an example brain multiplex lattice in Figure 4.1. Its most granular component is a single cell type, such as an excitatory neuron, inhibitory neuron, or glial cell, within one tissue of one region. Combining cell types yields a tissue-level multiplex; combining tissues yields a region-level multiplex such as the prefrontal cortex, hippocampus, or cerebellum; and combining regions builds up to the whole-brain multiplex described above. Every tier is therefore a multiplex in its own right, built by the same rule. These tiers do not form a strict tree, since a context can belong to more than one grouping at the tier above it. For example, an astrocyte in CA1 sits beneath both the all-cell-types node for CA1 and the astrocyte node for

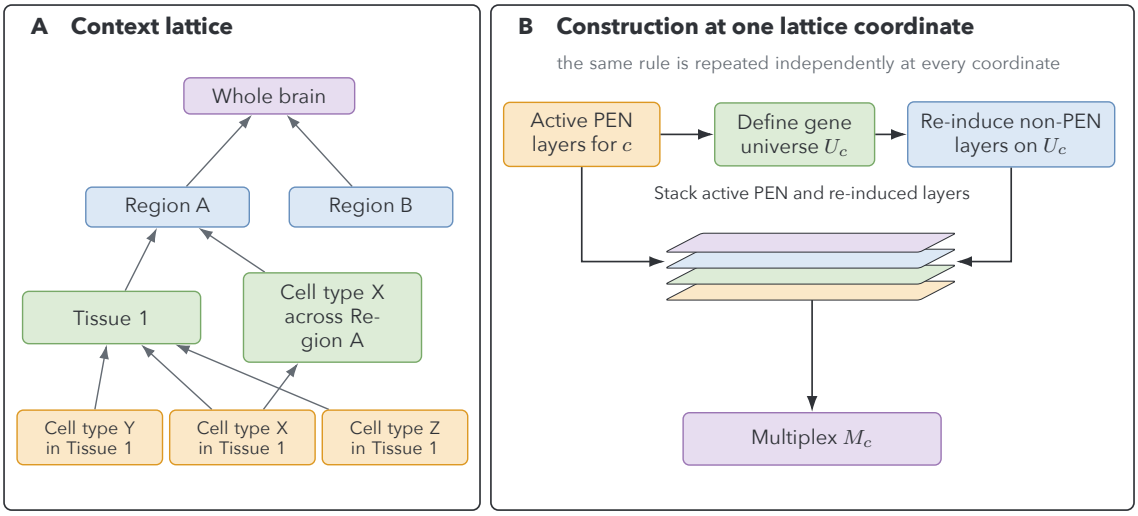

Figure 4.1: The multiplex lattice and its shared construction rule. Panel A gives a generic partial-order example. A cell-type-within-tissue context belongs to both its tissue context and its cell-type-across-region context. Panel B shows the construction at one coordinate. Active PEN layers define the gene universe, non-PEN layers are re-induced on that universe, and both layer sets are stacked into a distinct multiplex.

the hippocampus as a whole. Section 4.2.2 formalizes the resulting structure. Disease networks for anxiety, suicide, and pan-substance-use-disorder are points in this same lattice, which makes transfer meaningful. A model that has learned the construction operator can be applied to diseases directly.

Higher tiers are built by applying the same rule to a union of contexts. We take the union of the parts' gene universes, re-induce the non-PEN layers on it, and stack the parts' PEN layers on top. This process makes lattice sampling highly combinatorial, as changing a single PEN layer changes two things at once. The gene universe grows or shrinks, and the backbone is re-induced to match. A pair of multiplexes that differ by one layer therefore also differ by the backbone that the layer induces. Measuring the effect of a layer on its own requires holding the gene universe fixed, which we will test via controlled ablation. The lattice also holds far more combinations than we can build, so we note that tier counts and sampling rates are provisional until divergence validation shows how much contexts actually differ.

4.2.2 Context Ontology

To train MENTOR-RL reproducibly, we define a *context ontology* that specifies relationships among contexts. This ontology is supplied, versioned metadata that lets us name, version, and align the lattice with external sources. We use several grounded ontologies to define the context hierarchy. Anatomy is defined by UBERON [139], while cell types are defined by the Cell Ontology [42] and extended by the Provisional Cell Ontology [195] for transcriptomic brain types not yet represented in the Cell Ontology. Developmental stages are defined by HsapDv [14], anatomy-to-cell-type mappings by the Human Reference Atlas [17], and node metadata by the CELLxGENE schema [1]. Because the Human Reference Atlas covers the whole body, this paradigm can later extend beyond the brain.

Contexts form a partial order instead of a tree because some cell types appear in multiple tissues and some tissues appear in multiple regions. We therefore define a *join* operation, where the join of two contexts is the smallest context containing both. Joining two contexts produces the union multiplex described in Section 4.2.1, which links the ontology to the construction rule.

We supply the ontology as input and do not store it in the model weights through training. Instead, the model is trained to navigate the ontology by returning a context's parents, children, siblings, joins, and distances. Because only terms change while the operators remain static across the lattice, we expect the model to generalize to unseen contexts.

4.2.3 Three Structural Views

To represent this ontology-based lattice structure as a training target, we define three equally weighted structural views of a multiplex network: bottom-up random-walk-with-restart lines-of-evidence (RWR-LOE) neighborhoods, top-down MENTOR-EV-derived hierarchies, and cross-module network connectivity. Each view represents the same underlying multiplex with different biases.

The RWR-LOE view is a bottom-up representation of a multiplex that captures the local topology around each node [93]. For each node in a given multiplex, we perform a random walk with restart over all layers of the multiplex, treating each layer as an independent line of evidence. The walk returns a probability distribution over all nodes in the network, with higher probabilities representing closer proximity to the seed. We then threshold the distribution using a geometric elbow, which cuts the score-versus-rank curve at its bend. The genes above the cutoff form a module of entities that are proximal to the seed over multiple lines of evidence. From RWR-LOE modules, we can learn several operator types, including rank and score lookup, rank comparison, and relations between modules.

MENTOR-EV-derived modules represent an alternative, top-down view of the multiplex. MENTOR-EV applies MENTOR [191] using every gene in the multiplex as a seed, so the resulting hierarchy spans the full gene universe, not a supplied gene set. To create MENTOR-EV modules, we first use RWR to compute the probability distribution for each node in the multiplex. We then compute pairwise similarity between RWR vectors using Spearman correlation and subtract those correlations from one to construct a dissimilarity matrix over all nodes. Finally, hierarchical clustering produces a dendrogram in which leaves are genes and each internal node defines a *clade*, the set of genes beneath it. Cutting the dendrogram at a given merge height produces a family of nested modules and controls their granularity. From MENTOR-EV modules, we can learn several operator types, including module membership, module comparison, lowest common ancestors, and distance between modules.

Every one of these targets is read from a single dendrogram. The structural targets are alignment-free and do not require two hierarchies to agree, although we compare derived divergence summaries across contexts. This is what makes the divergence described in Section 4.2.1 trainable instead of an obstacle. Hierarchies differ across contexts, but each one supplies its own targets.

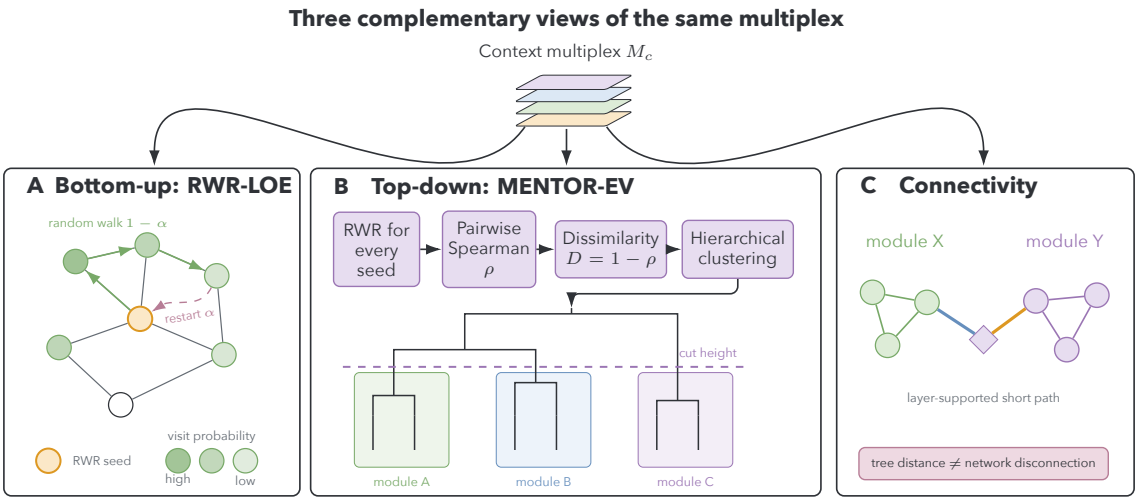

Figure 4.2: Three structural views derived from the same context multiplex. RWR-LOE supplies bottom-up neighborhoods, MENTOR-EV supplies top-down nested clades, and direct network operators recover paths and bridge nodes suppressed by either abstraction. The views are complementary targets, not interchangeable descriptions.

However, RWR-LOE and MENTOR-EV modules are both lossy abstractions of the underlying multiplex structure. Two clades on distant branches of the dendrogram may still be connected by short paths through a small number of bridge nodes, and a neighborhood cut at the elbow drops genes that remain reachable in a few hops. As such, we also include learnable network connectivity operators as a third view, which include cross-clade path recovery, connector and bridge node identification, path decomposition by supporting layer, and cohesion tested against a null. These operators allow the model to reason about the underlying multiplex structure and recover connectivity that is not captured by the other two views.

4.2.4 Stage 1: Multiplex World Model

In Stage 1, we train a world model to internalize the multiplex lattice by learning the operators that generate its structure. The model learns using a deterministic runtime that answers graph queries, so numeric results come from the runtime,

while the model learns to reason about the operators that produce them. A ten-step curriculum introduces the operators in order, and every training example is generated and validated by the runtime. Stage 1 will produce the checkpoint that initializes agent training in Section 4.2.5.

4.2.4.1 Deterministic Runtime and Tool Contract

To make the lattice environment compatible with the training pipeline, we use a compiled native multiplex library exposed to Python through `ctypes` bindings [88]. Corpus construction, training, and optimization code are implemented in Python [167], while graph storage and graph operations are handled by the runtime for efficiency on large multiplexes. Multiplex networks exist as raw binaries described by JSON manifests; this format is confined to native storage and execution. Model-facing tools instead accept structured biological requests and return compact, structured observations from the queried multiplex. The manifest carries the layer names, node metadata, and build metadata needed to load the graph.

The runtime uses a set of optimized multiplex functions developed as part of the MENTOR [191] and MENTOR-IA [208] projects, known as RWR++. Core operators include neighborhood retrieval, shortest-path search, random walk with restart on both monoplex and multiplex views, and subgraph induction. These operators provide the primary graph-facing actions available to the model.

We structure graph tools as learnable, interpretable operators according to three rules. First, each tool accepts only biologically meaningful arguments, such as gene identifiers, layer names, and module identifiers. Thus, when learning to produce a tool call, the model reasons about the biology instead of the file system or command-line interface. Second, we defer numeric computation to the runtime; because tokenization makes precise floating-point values difficult for the model to produce, the model calls a tool to compute any required statistic. Third, we enforce determinism, so the same arguments, graph version, layer scope, and parameter version always return identical output. This enables tool-call caching and reproducible scoring.

Runtime operators expose compact graph summaries and provenance metadata to the model instead of raw vectors or matrices. These observations record the relevant layer identity, operator parameters, and source information needed to trace the final interpretation back to the graph evidence used to construct it. To ensure consistency across serialized multiplex files, C++ runtime objects, Python wrappers, and logged training artifacts, all nodes are represented in a single canonical identifier space and all edges are referenced using deterministic, layer-aware identifiers. Table 4.1 lists the model-facing tool vocabulary used in the curriculum.

4.2.4.2 Supervised Curriculum

To train the model to internalize the multiplex lattice, we design a supervised curriculum that teaches it to navigate the lattice and learn the operators that generate its structure. We divide training between closed-book and open-book examples. Closed-book examples are finite and stable rules that should be memorized by the model, such as layer tags, layer families, module identifiers, and direction conventions, while open-book examples cover unbounded items like continuous values and statistics, which should be read from the runtime. The same distinction extends to tool-call training, where the model learns to parse returned objects instead of restating them from memory. We intend for the model to learn the operators that generate structure, not the specific values they produce.

Using these principles, we define a ten-step supervised training curriculum that gradually introduces more complex operators and tasks. Each stage builds on the previous one, starting with basic graph navigation and moving toward module comparison, path recovery, and enrichment analysis. We group similar stages in the prose; Table 4.2 presents the full curriculum.

Curriculum stages S0–S3 focus on foundational graph-navigation operators. S0 covers Ensembl-to-symbol and symbol-to-Ensembl conversions and resolution of ambiguous symbols, which are necessary for all downstream tasks. S1 introduces the lattice coordinate system, teaching the model to navigate the multiplex structure,

Table 4.1: Model-facing RWR++ tool vocabulary. Related calls are grouped by function while preserving the tool names the model emits. The final column gives the curriculum step at which each group is introduced (Section 4.2.4.2).

Tools	Purpose	Primary inputs	Step
`get_neighbors`; `induce_subgraph`	Direct neighbors of a gene and the edges induced within a gene set	gene or gene set; layer scope	S2
`get_gene_layers`; `get_nodes_by_layer`; `get_layer_stats`; `get_component_summary`	Layer membership, presence of a set in a layer, per-layer statistics, and connected components	genes; layer identifiers	S2
`shortest_paths`; `get_path_layer_counts`	Shortest paths within and across layers, and the per-layer edge counts supporting a path	source and target genes; path	S3
`rwr`; `rwr_loe`	Random-walk expansion from seeds, and leave-one-out relatedness with its elbow cutoff	seed genes; query genes; `top_k`	S4
`get_rank`; `get_distance`; `get_spearman`; `get_pearson`; `get_dot_similarity`	Rank of a target gene from a seed, and pairwise distance or similarity under a stated metric	source and target genes; metric	S4
`get_rank_vector_summary`; `get_encoding_summary`	Compact summaries of rank-vector and encoding outputs without exposing dense vectors	seed or query genes; layer scope	S4
`get_seed_essentiality`; `get_layer_ablation`; `get_node_perturbation`	Effect of removing a seed, a layer, or a node on the resulting neighborhood	seed genes; layers; genes	S4, S6
`get_clade`; `get_lca`; `get_subtree`; `get_clade_relations`	Clade membership, lowest common ancestor of a gene pair, the subtree induced by a cut, and parent, child, and sibling relations	clade or module identifier; gene pair; cut height	S5
`rwr_netstats`	Module and cohesion statistics, with an empirical p-value where applicable	module identifiers; statistic; null size	S5, S6
`query_mygene`; `enrich_gene_set`	Gene annotation, and functional or module enrichment of a gene set	query or gene set; `against`; thresholds	S7

retrieve layer information, and interpret rankings and metrics. S2 covers atomic graph facts and negative examples, such as edge existence, neighbors, layer membership, and

degree. S3 introduces path-based and cross-layer operators, including connector and bridge nodes.

After foundational training, stages S4–S6 focus on the three structural views of the multiplex. S4 teaches the model to operate on bottom-up RWR-LOE neighborhoods, S5 trains it on top-down hierarchical modules, and S6 unifies these views through network connectivity. S5 also includes examples that teach correspondences between the RWR-LOE and MENTOR-EV views, preventing them from being learned as unrelated skills.

Stages S7 and S8 integrate the lattice structure by adding projection and enrichment tasks. S7 teaches the projection workflow, which combines the top-down and bottom-up views of the lattice. A projection neighborhood is selected, which can be a GWAS-derived candidate list, an RWR-LOE neighborhood around a single target, or any other relevant set of genes. The runtime then enriches that neighborhood against every level of the MENTOR-EV hierarchy, yielding an enrichment score and significance value for each clade. High enrichment across several distant clades is evidence of a single connected system, not unrelated biological processes. This reinforces the idea that tree distance does not mean that clades are unrelated (Section 4.2.3). Therefore, recovering bridge nodes, short cross-clade paths, and their supporting layers is essential. S8 teaches composition and context-ontology navigation in open-book mode. Given a supplied partial order, the model returns a context's parents, children, and siblings, computes joins and ontology distances, and uses each join to name the corresponding union multiplex. It also learns to predict how neighborhoods, clades, and connectivity change when a layer is added or removed, which is crucial for understanding differences across sampled lattice points.

Finally, S9 consolidates the curriculum by combining earlier stages and preparing the checkpoint for the multistep reasoning required in Stage 2. The model trains on tool-call construction, selection, parsing, provenance, and task refusal, all of which are necessary for the mechanistic agent (Section 4.2.5.1) to operate effectively in the multiplex lattice.

The curriculum scales difficulty along several axes. We scale the lattice tier from single cell types to tissues, regions, and eventually the entire brain. Dendrogram cut height controls module size, progressing from coarse to fine-grained views of clades. We also scale operator complexity and book mode explicitly. To measure progress, we define task-specific gates that the model must pass before advancing to the next stage. All gates use held out question phrasings and graph regions for each family, as well as negative examples that measure abstention.

4.2.4.3 Corpus Generation and Validation

To build the training corpora for each curriculum stage, we generate questions procedurally from various points on the lattice. By sampling many contexts and layers, we allow the model to see each operator applied to many different networks, which encourages generalization. All targets have their ground truth produced by the deterministic runtime defined in Section 4.2.4.1, and examples are cached with their layer descriptor, metric, seed set, and graph version to ensure reproducibility. We freeze context, clade, and tuple assignments before generating training examples, then materialize each curriculum corpus on demand. Because assignments are frozen first, we can modify the sampling strategy or corpus volume without leaking information from the validation and test sets.

Every answer field is checked against a deterministic runtime, which ensures that both training examples and model outputs are valid. The runtime checks items such as path existence, module membership, and mechanistic claims, composing these checks based on the requirements of a task type. Using deterministic validation for our training examples and inference outputs allows cheap and scalable evaluation of reasoning throughout the process.

To provide sufficient training signal, our provisional lattice corpus includes 24 contexts and a full-brain multiplex. We expect to generate approximately 3 million supervised examples: roughly 10% closed-book, 75% open-book, and 15% tool-call examples, totaling about 1.2 billion training tokens. This corpus can be expanded to

Table 4.2: The ten-stage supervised curriculum. Book mode indicates whether targets are memorized (closed), read from supplied context (open), or produced by constructing and parsing a tool call. Data shares are provisional and will be reset once divergence validation establishes how strongly contexts differ across the lattice.

	Focus	Book mode	Share	Gate
S0	Entity normalization between symbols and Ensembl identifiers, including ambiguous symbols	closed	4%	Near-perfect accuracy on held out identifiers, and correct deferral on ambiguous symbols
S1	Lattice coordinate system: layer tags and families, module identifiers, and direction conventions	closed	4%	Near-perfect accuracy on held out tags and module identifiers
S2	Atomic graph facts: edge existence, neighbors, layer membership, degree and hubness, components, and calibration negatives	open	14%	Per-family accuracy on held out graph regions and phrasings, and correct refusal on calibration negatives
S3	Paths and cross-layer routes, path decomposition by layer, and connector and bridge nodes	open	8%	Path and bridge recovery on held out graph regions
S4	Bottom-up RWR-LOE neighborhoods: rank and score lookup, comparison, the elbow, leave-one-out support, and stability	open, tool	16%	Recovery and comparison accuracy stratified by distractor band
S5	Top-down MENTOR-EV hierarchy: clade membership, lowest common ancestor, subtree at a cut, and correspondence with RWR-LOE	open	16%	Hierarchy-navigation accuracy across cut heights, and correct cross-source containment
S6	Cross-clade connectivity: short paths between distant clades, bridge nodes, layer support, and cohesion against a null	open	12%	Cross-clade path recovery, and correct rejection of the false-separation reading
S7	The projection operation: select a seed neighborhood, enrich it against every hierarchy level, and return to the network	open, tool	12%	Correct projection, connectivity recovery, and interpretation, scored separately
S8	Composition and lattice scaling: open-book context-ontology navigation, joins, distances, and structural changes when layers are added or removed	open, tool	8%	Parent, child, and sibling set accuracy; join and ontology-distance accuracy; and correct structure on unions assembled from parts seen in training
S9	Tool-call consolidation: selection, argument construction, result parsing, provenance, and refusal of raw path arguments	tool	6%	Schema-valid tool-call rate and correct parsing on held out tools and phrasings

additional context as lattices for them become available, which we discuss further in Section 4.4.3. Cached ground truth, not text, dominates storage, with an estimated 2–4 terabytes across the 24 contexts. This provides a second reason to materialize corpora in curriculum order instead of all at once. These estimates are provisional and will be adjusted based on results at each curriculum stage.

4.2.4.4 Splits and Leakage Control

Because contexts share genes, they also share edges, which creates a risk of leakage between training, validation, and test sets. To mitigate this risk, we hold out whole contexts, whole modules, and whole seed genes. Two to four of the 24 contexts will be held out entirely, so their hierarchies are never seen in training. Modules are held out by cutting the maximal subtrees at a chosen anchor height, since genes appear in every module above that height. All tuples sharing a seed gene go to the same split, and we limit the overlap between held out and training tuples. Additionally, each split is audited prior to training, allowing us to identify and remove any remaining leakage.

Using this protocol, we can reduce the risk of leakage while keeping biological structure intact. Contexts still share backbone edges, which are by definition present in every context. Instead of removing these edges, we bound and report them.

4.2.5 Stage 2: Mechanistic Agent

After training MENTOR-RL on all curriculum stages, we use the resulting checkpoint to initialize a mechanistic agent that explores the multiplex lattice and generates mechanistic interpretations. The agent is post-trained through preference optimization and reinforcement learning on long-horizon reasoning tasks. We formalize the agent loop and structured state in Section 4.2.5.1 and describe the task cases, corpora, and reward design in Sections 4.2.5.2–4.2.5.5. Section 4.2.6 details the full training pipeline and curriculum.

4.2.5.1 Agent Loop and Structured State

We model mechanistic exploration as a sequential decision process executed by a single policy with an internal self-verification step. The loop begins when the user provides a query q and optional evidence e, which may include free text, seed genes,

or a multiplex. These initialize the working interpretation I_0 and the structured state S_0.

Each step t includes multiple actions. First, given the current decision context, the agent emits a textual reasoning step r_t, naming its current subgoal or thinking trace, together with an optional tool call a_t. The tool call is executed by the deterministic runtime ε of Section 4.2.4.1 to produce an observation o_t; when no call is made, the observation is empty. The same policy is then used for self-verification. Conditioned on the decision context and on its own reasoning, action, and observation, it updates the interpretation and state and emits a continuation decision z_t of `continue`, `revise`, or `stop`. A decision of `revise` means the next iteration corrects the interpretation and state instead of extending them. One iteration is represented as

$$r_t, a_t \sim \pi_\theta^{\text{act}}(\cdot \mid D_t), \qquad o_t = \varepsilon(a_t), \qquad I_{t+1}, S_{t+1}, z_t \sim \pi_\theta^{\text{verify}}(\cdot \mid r_t, a_t, o_t, D_t),$$

where $D_t = (q, e, I_t, S_t)$ is the decision context and $\varepsilon(\varnothing) = \varnothing$ by convention. The actor and verifier are two prompting modes with the same weights, so biological vocabulary and tool-use competence learned in one mode transfer to the other. We acknowledge the risk that the verifier collapses into agreement with the actor, and will actively monitor this behavior during training. The loop repeats until the policy emits `stop` or the decision budget $T_{\max}$ is exhausted, and returns the pair (I_T, S_T): a human-readable answer grounded in the accumulated evidence, and the machine-readable record used for scoring and provenance.

At each step, the working interpretation holds the current mechanistic claim, the main evidence supporting that claim, any uncertainty, conflict, or alternative explanation, and the next subgoal the model is trying to resolve in a human-readable markdown format. The structured state stores the same hypothesis in a machine-readable form suitable for tool execution, reward computation, and provenance tracking. It contains:

1. the user-provided anchors (q, e),

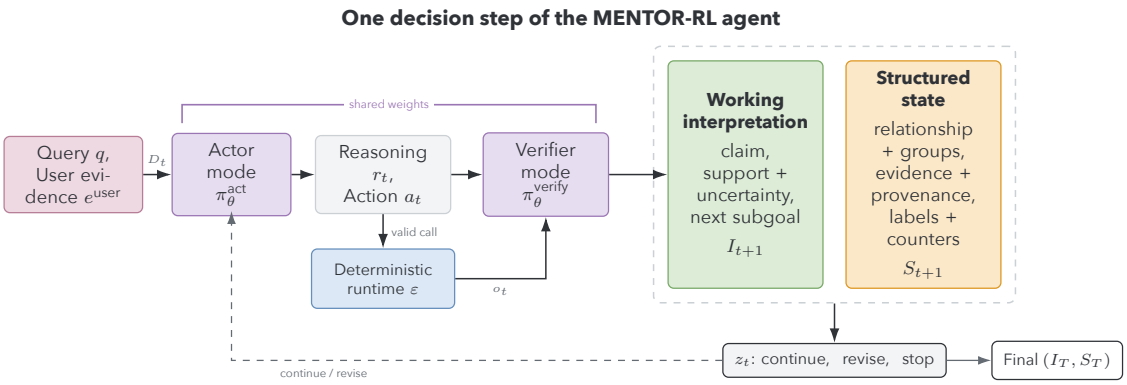

Figure 4.3: One iteration of the act–tool–verify loop. Actor and verifier modes share model weights, while the graph runtime remains deterministic. Verification produces synchronized human-readable and machine-readable views of the current hypothesis and a decision to continue, revise, or stop.

2. the current status of the working hypothesis: unknown, partial, validated, multiple, or insufficient support, where insufficient support indicates abstention,

3. the current findings: predicted groups, enriched clades, paths with their supporting layers, and bridge nodes, depending on the task requirements,

4. the evidence and provenance collected so far,

5. the current predicted mechanistic labels, each tied to the evidence that supports it,

6. the tool-use counters recording total and invalid calls, and

7. the remaining budget and continuation state, with the termination signal recorded at the end of the trajectory.

The findings that a task populates depend on the task type. Module tasks populate groups, while connectivity tasks populate paths, supporting layers, and bridge nodes, and projection tasks populate enriched clades together with the connectivity fields. Scoring depends only on the fields that are required by the metadata.

To ground the agent's reasoning in evidence, we train three behaviors. The agent supports every claim with retrieved evidence, states its uncertainty explicitly, and abstains when support is weak. It uses the MENTOR-EV dendrogram modules as an index and confirms its hypotheses on the network, following the multiview structure described in Section 4.2.3. It returns both a human-readable interpretation and a structured state for record-keeping. The reward function and training curriculum reinforce these behaviors.

4.2.5.2 Task Cases and Targets

To train the agent, we define three categories of agent-training tasks that build on the lattice representation learned in Stage 1: module, connectivity, and projection tasks. Each has targets that provide deterministic rewards and support evaluation. These tasks combine to answer two kinds of questions. First, whether a set of genes forms a complete and coherent module, and second, how the modules connect to one another in the multiplex lattice. Each task type has targets that provide deterministic rewards and support evaluation.

Module tasks fall into four cases: partial-group completion, mechanism explanation, group refinement, and recognition of no shared mechanism. In partial-group completion, the agent receives a module with some genes held out and must recover the complete module. In mechanism explanation, the agent receives a complete module and must identify its supporting mechanistic evidence without adding or removing genes. In group refinement, the agent receives a module with distractor genes and must remove them to recover a mechanistically coherent group. In the no-shared-mechanism case, the agent receives a conflict-free set that is not mechanistically coherent and must report that the evidence provides insufficient support for a shared mechanism.

Connectivity tasks join the two module views through the network that underlies both. The agent is given two clades that are distant in the MENTOR-EV hierarchy and identifies a short path between them, along with the layers that support that

path. It also identifies any bridge nodes that connect the two modules and provides mechanistic evidence for their role. This task tests the agent's ability to reason about the underlying multiplex structure and recover connectivity that is not captured by the bottom-up or top-down views alone.

Finally, we train the model on projection tasks, which require the agent to project a seed neighborhood onto the MENTOR-EV hierarchy and identify the clades that are enriched for that neighborhood. Because the neighborhood was built by propagation over the network, enrichment landing in several distant clades indicates one connected system, not unrelated processes, and the agent reads it that way. It then recovers the bridge nodes and short paths that connect those clades and provides mechanistic evidence for their role. Projection tasks are scored on three targets separately: the projection itself, the connectivity recovered to support it, and the interpretation placed on that connectivity. A candidate passes only when all three clear, since a correct projection paired with a failed connectivity recovery is precisely the error that the network view exists to prevent.

Each task family is used to organize training and scoring, but not the agent's behavior. The agent never receives a task-type label, but during training, the scorer knows which task is sampled and reads the corresponding structured state fields to compute rewards. At deployment, the agent answers the query with whichever operators the question requires.

The agent curriculum culminates in the applied case, in which the projected seed is a drug target or a disease signal. Here the agent additionally applies directional and translational filtering. Directional filtering asks whether the candidate intervention opposes the disease-associated direction within a specific module instead of merely acting near it. Directionality comes from differential expression with respect to disease context, which is returned by the runtime. Translational filtering ranks surviving candidates based on multiple metrics. Evidence strength is computed from retrieved provenance, while adverse-event risk, brain penetration, feasibility, and testability are reported for expert review and do not enter the training reward.

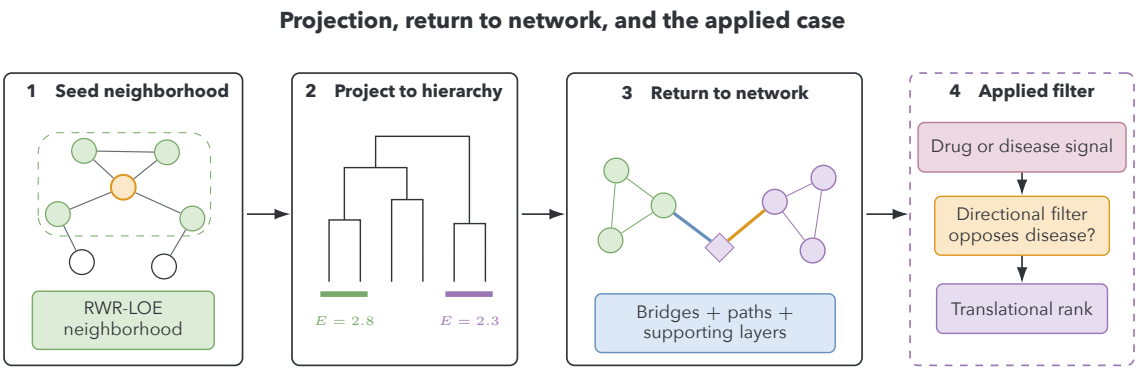

Figure 4.4: Projection and return to network, with the applied drug-target or disease-signal extension. A seed neighborhood is enriched against the hierarchy; significant, potentially distant clades are then connected and interpreted on the multiplex. Directional and translational filters are applied only after this network confirmation.

Module tasks use scorer-only membership targets drawn from RWR-LOE neighborhoods and MENTOR-EV modules. Recovery and refinement target the complete module, explanation preserves the supplied module, and `none` has an empty target. Because these computational modules usually lack per-entry experimental annotations, explanations are scored against evidence retrieved during exploration, not a hidden biological label.

The runtime supplies family-specific targets for connectivity and projection: optimal paths, connector and bridge nodes, and supporting layers for connectivity; and per-clade enrichment, cross-clade connectivity, and allowed or disallowed interpretations for projection. Any optimal path is accepted. All targets remain hidden from the agent, and evidence grounding is scored consistently across task families.

4.2.5.3 Task Corpora

We generate module tasks from top-down MENTOR-EV modules and bottom-up RWR-LOE neighborhoods sampled across the lattice. For each module of size n, we deterministically materialize up to four instances: `explanation` uses the full module, `recovery` drops k_{drop} genes, `refinement` adds k_{add} distractors, and `none` constructs

an unrelated pseudo-module with an empty target. Perturbation size increases with task difficulty and module size.

The two module sources differ only in where distractors come from. For MENTOR-EV modules, noise and `none` genes are drawn from difficulty-modulated *dendrogram distance bands*, defined by walking up the module's ancestry in the tree, so that difficulty controls how topologically close the distractors are to the module. For RWR-LOE modules we substitute rank bands, drawing from ranks just beyond, moderately beyond, or far beyond the seed's elbow cutoff. In both cases `none` genes are additionally constrained to be conflict-free, so that the sampled genes do not form a coherent module of their own, and all selection is deterministic given a fixed seed, making the corpus reproducible. Module-size bins are assigned after construction from the realized module sizes, and all splits follow the protocol of Section 4.2.4.4.

Connectivity instances pair sampled clades at specified tree distances; projection instances sample a seed neighborhood from the sources in Section 4.2.4.2, with difficulty set by how far apart the enriched clades fall in the hierarchy. The runtime then materializes the path, layer support, enrichment, and connectivity targets defined in Section 4.2.5.2.

4.2.5.4 Trajectory Generation

Trajectories for reinforcement learning and preference optimization are generated by shared-prefix branch exploration. At each decision point, the policy proposes several candidate continuations from the same state, the runtime executes any valid tool calls among them, and the resulting branches are scored against one another. One branch is retained as the trajectory, while eligible alternatives become material for preference pairs. The procedure is described below.

Given a sampled task and initialized state, we sample multiple action candidates $(r_{t,i}, a_{t,i})$ from the current decision context D_t and execute valid tool calls in the deterministic graph runtime. To ensure that candidates with the same prefix explore distinct regions of the action space, we adapt verbalized sampling to the visible task

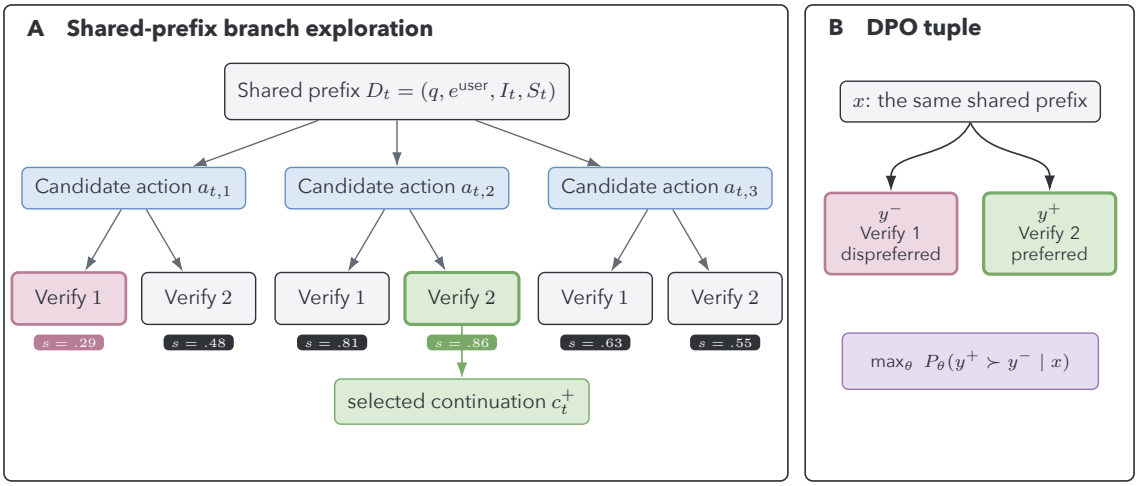

Figure 4.5: Shared-prefix branch exploration and DPO preference construction. Candidate actions and verifier updates are compared only after the same decision context. Task-quality selection chooses the retained continuation; eligible lower-scoring branches from that pool become dispreferred continuations when the normalized margin exceeds τ.

framing [226]. For example, a query asking the agent to recover missing members biases candidates toward random-walk expansion, neighborhood expansion, subgraph validation, or a direct decision. Framing affects which actions are explored while the underlying preference rewards do not vary. When a sampling round for recovery or refinement fails to produce valid tool calls, we issue a tool-coverage retry that requires a valid graph operator whenever one can be formed, preventing the branch pool from degenerating into tool-free continuations. Conditioned on each candidate action and observation, the same policy samples multiple verification candidates, yielding a tree of candidate next states rooted at a shared prefix. Figure 4.5 visualizes this branching scheme.

Each candidate branch is assigned a deterministic local score computed from the structured state, runtime outputs, and hidden targets for its task family. These local scores are used to rank same-prefix candidates and mine DPO preference pairs against that selection. Because several branches tie frequently or fall within a small margin of each other, continuation selection accepts any candidate within a margin ϵ of the best

normalized score. Ties between candidates are broken with task-specific heuristics, such as prompted seed-gene retention and tool-supported evidence, which favor well-grounded candidates over those that score highly by chance. We defer the formal score definitions to Section 4.2.5.5.

The preferred candidate is the branch selected as the trajectory continuation, not the strict score maximizer, since selection resolves near-ties using task-specific heuristics and therefore reflects the move judged best for the task. Candidates within a margin τ of the selected branch are discarded so that no pair is built from a near tie, and the remainder form the dispreferred pool. From that pool we draw examples across `easy`, `medium`, and `hard` difficulty bins, taking the lowest, median, and highest scoring remaining candidates respectively. Each pair is stored with its bin, task type, decision step, scores, and margin, and the corpus is rebalanced across task types and bins so that training is not dominated by the categories that happen to yield the most informative pairs.

Each retained trajectory is also logged as a sequence of turn records, from which we render short per-step findings and a final mechanistic summary. These become supervised examples and preference pairs for the training pipeline described in Section 4.2.6.

Model prompts carry a compact summary of the current state, so context does not grow as the trajectory lengthens. Each prompt supplies the current working interpretation, the relationship status, the predicted groups, the mechanistic labels tied to the evidence handles that support them, the tool and evidence counters, and the current observation. Prior observations are not appended, so prompt length stays bounded as trajectories grow, allowing for long-horizon exploration that is not limited by memory. The full evidence payloads are retained in the logged trajectory artifacts, where they remain available for scoring, audit, preference-pair mining, and expert review.

We provide several query modes and evidence types to represent the diversity of real-world inputs. During training, we sample queries from task-specific templates

paraphrased by an open-weight model to ensure linguistic diversity. The agent receives either minimal evidence, consisting only of the query and associated seed genes, or graph-supported evidence, which includes a relevant multiplex subgraph. At inference time, the agent receives a natural-language query with optional graph or seed evidence. However, it does not see the task-type label, the true relationship status, or the hidden targets, which are reserved for scoring and trajectory generation.

We define a three-stage curriculum for the mechanistic agent, which gradually increases in task complexity and trajectory length. The first stage focuses on module tasks, where the agent learns to recover and refine modules from partial or noisy inputs. The second stage introduces connectivity tasks, where the agent identifies paths and bridge nodes between distant clades. The final stage focuses on projection tasks, where the agent projects neighborhoods onto the hierarchy and recovers mechanistic evidence for the resulting connections, synthesizing the skills learned in the previous stages. Each stage builds on the previous one, ensuring that the agent develops a comprehensive understanding of the multiplex structure and the mechanistic relationships within it.

4.2.5.5 Reward Design

We define training-time scores over structured state transitions, deterministic runtime metadata, and hidden scorer-only targets specific to each task family; natural-language reasoning is not scored directly. This keeps scoring reproducible, machine-parseable, and scalable without relying on human judges during training. Every score has four shared components: schema compliance, task correctness, evidence-grounded mechanistic quality, and efficiency.

Task correctness is computed the same way for all task types, but the underlying targets differ. Each family declares which findings in the structured state are scored and against which hidden target, and each declared component is scored by overlap with that target. For families with several scored components, we compute the task score as a weighted mean, so $q^{\text{task}}(S) \in [0, 1]$ regardless of family.

Because DPO compares continuations after the same prefix, we score a candidate branch b by its change in task correctness and mechanistic quality:

$$s_{t,b}^{\text{local}} = w_s s_{t,b}^{\text{schema}} + w_q \Delta q_{t,b}^{\text{task}} + w_m \Delta m_{t,b} - w_e p_{t,b}^{\text{call}} - p_{t,b}^{\text{guard}}. \tag{4.1}$$

Here, $q^{\text{task}}(S) \in [0,1]$ is the correctness score associated with the sampled task family, $\Delta q_{t,b}^{\text{task}}$ is the difference between q^{task} at t and $t+1$, and $\Delta m_{t,b}$ is the difference between the mechanistic quality at t and $t+1$. The schema term is equal to one only when the candidate state and any tool request satisfy their typed validators. The guard penalty covers task-incompatible behavior and varies by task family. For module tasks, an `explanation` branch is penalized for removing members from a module that is already complete. Across all families, mechanistic improvement is clamped to be non-positive whenever a branch degrades task correctness.

For recovery, refinement, and explanation tasks, task correctness is the best match between any predicted group and the hidden module target C:

$$q^{\text{module}}(S; C, C_{\text{type}}) = \max_{G \in \mathcal{G}(S)} \left[\alpha J(G,C) + \beta P(G,C) + \gamma R(G,C)\right],$$

where $\mathcal{G}(S)$ is the set of predicted groups in structured state S, with an empty group used when no prediction is present, and J, P, and R representing the Jaccard similarity, precision, and recall of a predicted group against the target. The weights α, β, and γ sum to one and are set per task case: recovery upweights recall, refinement upweights precision, and explanation weights the three equally. Final values will be set during development. This formulation supports a multiple-groups state while retaining a single-module hidden target. For module tasks, q^{module} takes the role of q^{task} in Equation 4.1.

For a `none` task, the correctness score instead rewards calibrated abstention:

$$q^{\text{none}}(S) = 0.6\,\mathbf{1}[\text{status}(S) = \texttt{insufficient\_support}] + 0.4\,\mathbf{1}[\mathcal{G}(S) = \varnothing], \tag{4.2}$$

where status(S) is the hypothesis status of the structured state (Section 4.2.5.1) and $\mathbf{1}[\cdot]$ equals one when its condition holds and zero otherwise.

Thus, `none` is an instantiation of the shared task-correctness term, not a separate reward pathway. The multiple-groups status remains a supported state, but it receives no separate score unless the corpus later materializes a corresponding task family.

For connectivity tasks, q^{conn} combines path validity and optimality, connector and bridge-node recovery, and supporting-layer recovery. If v_{path} denotes the fraction of returned paths that are valid and optimally short, B the returned connector and bridge nodes, and L the returned supporting layers, then

$$q^{\text{conn}}(S) = \eta_p v_{\text{path}}(S) + \eta_b J(B(S), B^*) + \eta_l J(L(S), L^*), \qquad \eta_p + \eta_b + \eta_l = 1, \quad (4.3)$$

with component weights η_p, η_b, and η_l summing to one and set during development.

Any optimal path is accepted. For projection tasks, the correctness score preserves the operation's three distinct targets:

$$q^{\text{proj}}(S) = \eta_e q^{\text{enrich}}(S) + \eta_c q^{\text{conn}}(S) + \eta_i q^{\text{interp}}(S), \qquad \eta_e + \eta_c + \eta_i = 1. \quad (4.4)$$

The enrichment component scores significant clade recovery, where the runtime computes significance with a one-tailed Fisher exact test, and copied runtime values. The connectivity component scores the return to the network, and the interpretation component scores the structured allowed and disallowed readings. These components provide dense local feedback, but a completed projection passes only when all three clear their individual thresholds, which are set during development, so projection accuracy cannot compensate for failed connectivity recovery. Directional and translational fields in the applied case are scored within the structured interpretation component when present.

We define mechanistic quality $m(S) \in [0, 1]$ from the evidence the agent has retrieved. Each component measures the alignment between the mechanistic labels in the structured state and the evidence that supports them. Let g denote grounding, or the fraction of asserted entities and labels that appear in retrieved observations. Let n denote network support, or the fraction of asserted relationships confirmed in the multiplex in stated layers. Finally, let c denote corroboration, or the fraction of claims supported by at least two independent sources among enrichment, annotation, and network. For a structured state that asserts mechanistic labels, we define

$$m^{\text{claim}} = w_g\, g + w_n\, n + w_c\, c,$$

with weights summing to one and set during development. For an abstaining state, or one whose status is `insufficient_support` (Section 4.2.5.1), we set $m^{\text{abs}} = w_u\, u + w'_g\, g$, where u is one minus the best corroboration available for any candidate claim, so withholding is rewarded in proportion to the weakness of the evidence. On a non-`none` task, this score is forced to zero when a target is recoverable, preventing unsupported abstention from earning mechanistic reward.

The local efficiency term penalizes the fraction of a branch's tool calls that are invalid, scaled by a hyperparameter λ_{inv}. A branch that makes no tool calls receives no penalty.

Schema-invalid requests, exact duplicates, missing observations, and error observations count as invalid; a branch that makes no tool call receives no call penalty. Trajectory length is omitted locally because it is constant across same-prefix candidates and therefore cannot affect their DPO ranking.

4.2.5.6 Terminal Reward Decomposition for GRPO

Unlike DPO, which compares local same-prefix branches, policy gradient methods require a scalar reward for a completed rollout. We therefore define a terminal reward over the final structured state and trajectory log:

$$R_T = w_s^T s_{\text{schema}}^T + w_q^T q_T^{\text{task}} + w_{\Delta q}^T \left(q_T^{\text{task}} - q_0^{\text{task}}\right) + w_m^T m_T + w_{\Delta m}^T (m_T - m_0) - w_e^T p_{\text{eff}}^T.$$

Here, q_T^{task} uses the same family-specific definition as local scoring, so absolute and cumulative correctness apply equally to module, `none`, connectivity, and projection tasks. The terminal mechanistic terms use the final evidence-grounded score and its change from initialization. Mechanistic credit is capped when the task-specific correctness gate fails, preventing a fluent interpretation from compensating for an incorrect module, path, or projection. The terminal schema score additionally requires a valid final state and a recorded termination mode, whether the agent stops voluntarily or exhausts its budget.

The terminal efficiency penalty aggregates trajectory length and invalid tool use across the rollout:

$$p_{\text{eff}}^T = \lambda_1 \frac{T}{T_{\max}} + \lambda_2 \frac{\sum_{t=0}^{T-1} n_{\text{inv}}^t}{\max\left(1, \sum_{t=0}^{T-1} n_{\text{call}}^t\right)}. \tag{4.5}$$

The denominator convention makes the invalid-call term zero for a trajectory that makes no tool calls. The local score omits trajectory length because the same-prefix candidates have all reached the same point; complete rollouts, however, may vary in length, so the terminal reward penalizes verbosity through $T/T_{\max}$.

The local scores above supply the preference signal for DPO, while the terminal reward supplies the scalar return optimized during GRPO.

4.2.6 Training Pipeline and Curriculum

We first train the policy with SFT using the curriculum in Section 4.2.4.2. Training will run on OLCF's Frontier supercomputer [3], with Slurm [224] for job scheduling, Hugging Face and TRL [217] for optimization, DeepSpeed ZeRO-3 [159] for model parallelism, Low-Rank Adaptation for parameter-efficient fine-tuning [75], and

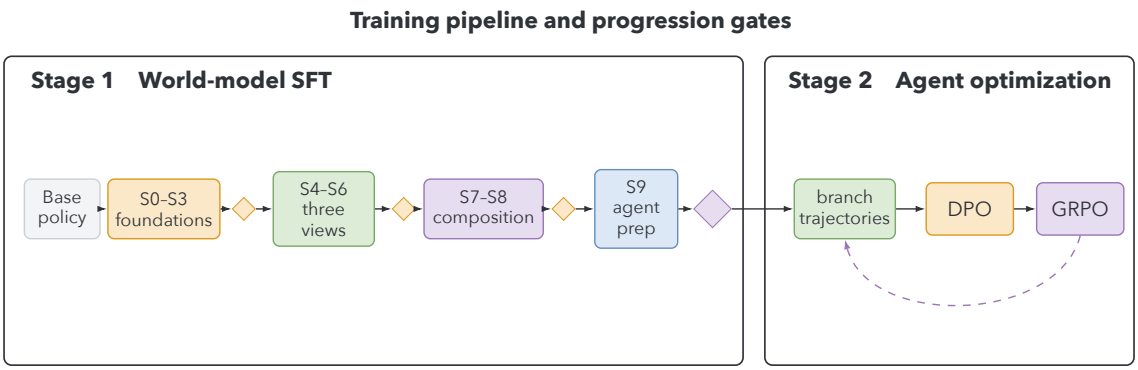

Figure 4.6: The gated two-stage training pipeline. Stage 1 advances through supervised curriculum steps only after held out gates clear. Its checkpoint initializes trajectory generation and preference optimization in Stage 2, followed by online GRPO. Checkpoint evidence accumulates from structural evaluations through agent behavior and expert review.

Weights & Biases [15] for convergence tracking. We use gpt-oss-120b [145] as the base policy and early-stopping to prevent overtraining.

After SFT on all curriculum stages, the trajectory method in Section 4.2.5.4 generates preference pairs scored by Section 4.2.5.5 to supply data for the next stage. The selected continuation is c_t^+, and eligible alternatives across the `easy`, `medium`, and `hard` bins supply c_t^-. DPO increases the likelihood of c_t^+ relative to c_t^- under a frozen reference policy without a separate reward model [156]. We regenerate trajectories from updated checkpoints as training progresses.

The DPO checkpoint then initializes GRPO for long-horizon exploration. Each rollout receives an advantage equal to its terminal reward standardized within its rollout group, removing the need for annotated trajectories or a learned value model [179]. The DPO checkpoint remains the fixed reference policy, anchoring online exploration while GRPO promotes trajectories beyond the logged preference data.

We monitor tool-use entropy, trajectory diversity, and the single-pass RWR shortcut, in which one expansion recovers much of a module without meaningful exploration. We monitor single-pass shortcuts because they can lead to task correctness without the multi-step reasoning that rewards are meant to encourage.

If shortcuts emerge, we introduce adversarial modules or a diversity bonus; these monitors must clear before advancement.

Each transition is gated. Stage 1 uses the per-step gates in Table 4.2; DPO must improve module recovery and mechanistic grounding over SFT on held out splits; and GRPO must improve the flagship evaluations. Deployment additionally requires expert preference to rise across the base, SFT, DPO, and GRPO checkpoints under Section 4.2.7.2.

Difficulty scales throughout optimization. Stage 1 independently advances book mode, operator complexity, lattice tier, cut-height granularity, and distractor band over curriculum stages. DPO moves from `easy` to `hard` pairs and from module recovery and refinement to connectivity and projection. GRPO increases the horizon $T_{\max}$, realized module size, perturbation size, distractor proximity, and lattice tier up to higher-order brain combinations. Batches remain balanced across task types, difficulty bins, and both module sources.

4.2.7 Evaluation

4.2.7.1 Stage 1 Evaluation: World Model SFT

Stage 1 advances through the gates in Table 4.2, which are evaluated on held out data at each step. Two flagship evaluations then measure mechanistic reasoning. The first generates a hierarchy for a context held out from training, and the second projects a seed set onto that hierarchy, returning it to the network. For both flagships, we use probes and interventions to test whether the recovered structure is causal.

In the first flagship, the model receives the active layers of an unseen context along with a query gene set, and it returns pairwise co-membership, lowest common ancestor depth, and a partition of the query gene set into clades. We score these results against a ground-truth dendrogram from the runtime. Because hierarchies diverge across contexts, this flagship tests whether the model has internalized the lattice structure.

The second flagship follows the projection described in Section 4.2.5.2. A seed neighborhood is selected, then enriched against the hierarchy, and the model recovers the paths and bridge nodes connecting significantly enriched clades. This flagship tests whether the model can unify multiplex views into a single interpretation.

To test whether lattice operators are internalized, we use multilayer-perceptron *probes* [2], which are classifiers trained on hidden layer activations, to recover edge presence, layer membership, clade co-membership, and connector status. We then intervene on those activations and measure whether answers change, distinguishing causal structure from mere decodability. This is a gate rather than a diagnostic and is repeated under the lean-context condition without the full rank-vector legend. Table 4.3 summarizes both flagships and the probe gate.

4.2.7.2 Stage 2 Evaluation: Mechanistic Agent DPO and GRPO

We will calculate evaluation metrics for Stage 2 by parsing the model's terminal structured state S_T, enabling scalable checkpoint evaluation. All evaluations will use a held out set of MENTOR-EV- and RWR-LOE-derived modules from the brain lattice. We will measure module, enriched-clade, and cross-clade path recovery with Jaccard, precision, recall, and F1. Jaccard penalizes both missing and extra genes, precision and recall separate those error directions, and F1 provides a balanced summary. We will also report mechanistic-grounding quality to test whether claims align with the evidence retrieved by the model.

We will report the weighted composite reward R_T, defined in Section 4.2.5.5, to compare reward magnitude on validation and holdout splits. For the `none` task type, where $C = \varnothing$, we will parse S_T for the terminal hypothesis status, status(S_T). In addition to global accuracy and reward, we will distinguish the two error directions: claiming that no mechanism exists for a related gene set (false negative) and fabricating a mechanism for unrelated entities (false positive). We will also report mean trajectory length, budget utilization, and invalid-tool-call rate, enabling comparison of holdout and validation efficiency under Equation 4.5.

Table 4.3: Scoring for the Stage 1 flagships and the cross-cutting probe gate. Components are reported separately; projection passes only when projection, connectivity recovery, and interpretation all clear their thresholds.

Evaluation or gate	**Component**	**Metric**
Held-out context hierarchy generation	Partition agreement	Adjusted Rand index [83], which measures agreement between two partitions corrected for chance occurrence
	Clade co-membership	Same-clade accuracy over query gene pairs
	Ancestry	Lowest common ancestor depth accuracy per gene pair
Projection and return to network	Projection	Jaccard of the predicted significant clades against the runtime enrichment, with per-clade score and q-value copy accuracy
	Connectivity recovery	Jaccard of recovered connector and bridge nodes, path validity rate, and the fraction of recovered paths at optimal length
	Interpretation	Accuracy of the connected-system reading, and the rate at which the disallowed reading that tree distance implies separation is asserted
Probing and intervention gate	Decodability	Probe classification accuracy for edge presence, layer membership, clade co-membership, and connector status
	Causality	Change in the model's answer when the probed activation is modulated to flip its predicted value
	Scaffold independence	Probe and interpretation accuracy retained under the lean context condition, without the full rank-vector legend

We will stratify module-task results by module size, noise, task type, and module source to examine the effect of curriculum staging. We will also stratify by lattice tier and cut-height granularity to measure performance across regions of the lattice. Each axis will be divided into three discrete bins. We will also assess stability by re-running the runtime's enrichment and propagation on leave-one-out subsets of each seed set and reporting the mean Jaccard between the resulting supporting-evidence sets. These metrics will show how MENTOR-RL performs as difficulty increases

and task type changes; comparison by module source will reveal whether the model generalizes across construction methods or favors one module type.

Alongside empirical validation, we will conduct blinded comparative evaluations of domain experts' model preferences. First, we will use an inter-model preference test to compare interpretation outputs from a panel of frontier and open-weight baselines. At present, these models would be MENTOR-RL, GPT-5 [187], LLaMA 4 [129], Kimi K2 [197], and Nemotron 3 Super [144], but they are subject to change with new releases. Between 5 and 15 domain experts will receive paired samples containing a MENTOR-RL output and an output from another randomly selected model. We will report the proportion of pairs in which experts prefer MENTOR-RL. A subset of pairs will be shared across raters to measure inter-expert reliability.

We will then use the same protocol for intra-model preference testing across the base, SFT, DPO, and GRPO-trained MENTOR-RL checkpoints. These blinded comparisons will assess characteristics such as style, tone, and concision that empirical metrics do not capture easily.

4.2.7.3 Ablation Studies

To ensure that each component of our methodology contributes to an improvement in mechanistic interpretation, we will ablate both training stages and design elements. All ablations will be evaluated using the holdout metrics defined in Section 4.2.7.2.

We ablate phases of the training pipeline to test whether each stage improves on the preceding checkpoint. First, we compare the SFT-only checkpoint with the SFT-and-DPO checkpoint to test whether preference optimization improves interpretation. Next, we compare the DPO checkpoint with the full pipeline to determine whether long-horizon RL improves on local preference training. Finally, we ablate Stage 1 entirely, training the agent from the base policy without the world-model curriculum and comparing it with the complete pipeline on module-recovery, connectivity, and projection tasks. This ablation tests the proposal's central claim that the world model is necessary rather than merely helpful. Preliminary trajectory diagnostics

in Section 4.4.1 indicate what to expect. An untrained policy prunes distractors successfully but fails to recover held out members or ground its claims in retrieved evidence.

We also ablate architectural and reward decisions. First, we remove the policy's verifier mode to test whether self-verification improves results. Second, we remove the efficiency penalty p^{eff} to test whether MENTOR-RL learns reasonable trajectory lengths without regularization. Finally, we remove progressive difficulty bins and train on all bins simultaneously to test whether the model can learn from hard examples without curriculum staging.

4.3 Expected Results

4.3.1 Expected Empirical Results

We expect MENTOR-RL to recover modules and cross-module paths accurately in held out lattice contexts, measured by Jaccard, precision, recall, and F1; project seed neighborhoods to enriched clades and connecting paths; and ground interpretations in retrieved evidence. Probes should decode lattice structure, and interventions should show that it is causal to reasoning. Improvement across SFT, DPO, and GRPO checkpoints on unseen contexts would indicate out-of-distribution generalization.

4.3.2 Expected Behavioral Results

We expect the act–verify loop to reduce invalid calls and unsupported claims, the efficiency penalty to shorten trajectories without reducing recovery, and expert preference to rise across the base, SFT, DPO, and GRPO checkpoints. The corresponding verifier, efficiency, and checkpoint ablations in Section 4.2.7.2 test each claim.

4.3.3 Deliverables

We will release, under licenses compatible with the underlying data sources, the following artifacts:

1. the lattice construction code and context ontology,
2. the lattice-grounded training environment, including the `ctypes` multiplex runtime, tool vocabulary, and deterministic scoring functions,
3. train, validation, and test split indices at the context, subtree, and tuple level, a leakage audit, and the code required to reconstruct the raw training corpus and cached API responses,
4. trained MENTOR-RL checkpoints for each stage of the pipeline (SFT stages S0-S9, DPO, and GRPO),
5. the flagship harnesses, including held out contexts and reserved target genes, and
6. training and generation code, curriculum configurations, and logged trajectories to support reproducible re-training and ablation.

By releasing a deterministically-scored, open-source mechanistic exploration world model and agent together with its training environment, we aim to lower the cost of biological interpretation research and establish a reproducible benchmark that does not require proprietary model access. The scoring framework is domain-specific, but the training process to learn operators over networks in a lattice structure is transferable to other scientific settings with curated ground truth.

4.4 Discussion

4.4.1 Preliminary Results

Several preliminary results support the feasibility of the proposed approach. First, we demonstrated full fine-tuning of the 120-billion-parameter open-weight gpt-oss-120b model [145] using data, tensor, and expert parallelism on OLCF's Frontier supercomputer. We have trained on up to 128 nodes. This infrastructure, together with the ability to test smaller models such as gpt-oss-20b under the same training and evaluation pipeline, meets the capacity requirements for the multiplex-lattice dataset.

We also implemented the Stage 2 trajectory-generation loop (Section 4.2.5.1) using the same gpt-oss-120b model. The loop runs end-to-end, provides a dashboard for run inspection, and produced no parsing errors in the preliminary runs. However, initializing trajectory generation from a base model without Stage 1 training yields poor performance. The protocol succeeds on refinement tasks but fails on recovery tasks and provides weak mechanistic grounding. These results support the need for Stage 1 to provide domain knowledge and tool-calling capabilities before Stage 2. Because they are based on a small number of runs and do not include the full evaluation suite, we will evaluate the loop more thoroughly after Stage 1 training.

Finally, we generated the corpora for the Stage 1 S0 and S1 curriculum steps. Our current focus is S0 training, which covers Ensembl-to-symbol and symbol-to-Ensembl mapping, as well as ambiguous-symbol resolution.

To begin Stage 1 training, we tested four tokenization approaches for mapping tasks. First, we used the gpt-oss-120b base tokenizer, fitted to its pretraining corpus with byte-pair encoding (BPE), as a control. Second, we fit a BPE tokenizer only to Ensembl identifiers and gene symbols, then concatenated the resulting vocabulary with the base vocabulary; we call this **domain-BPE** tokenization. Third, we encoded the Ensembl prefix (ENSG) as an atomic token, fit BPE to the remaining

identifier suffixes and gene symbols, and concatenated that vocabulary with the base vocabulary; we call this **atomic+domain-BPE** tokenization. Finally, we represented each Ensembl identifier using atomic tokens for its prefix and accession body, represented each gene symbol with one atomic token, and tokenized module identifiers compositionally; we call this **atomic** tokenization.

Our preliminary results show that base tokenization performs poorly on mapping tasks. We trained gpt-oss-120b on approximately 2,000 questions from a small S0 subset under each tokenization approach and evaluated it on 1,000 hard-subset questions. Overall accuracy increased from 45.6% with the base tokenizer to 85.9% with atomic tokenization. The mapping-specific gains were larger. Symbol-to-Ensembl accuracy increased from 8.3% to 100%, and Ensembl-to-symbol accuracy increased from 5.6% to 100%. These results select atomic tokenization for S0, pending confirmation at full vocabulary scale with held out and low-frequency entities.

4.4.2 Plan of Work

We will complete S0–S9 sequentially, materializing each corpus after the preceding gate clears; S0 is underway. The resulting Stage 1 checkpoint will undergo the two flagship evaluations, the cross-cutting probe gate, and curriculum ablations before generating Stage 2 trajectories. DPO on those trajectories will initialize GRPO, followed by the final evaluations, ablations, and deliverables in Section 4.3.3.

4.4.3 Limitations and Future Work

We identify several limitations and possible extensions. First, the lattice structure depends on the assumption that different contexts generate meaningfully different hierarchies. We will validate this assumption by comparing hierarchies across contexts; insufficient divergence would limit the lattice's utility. The approach also requires substantial compute to construct the RWR-LOE and MENTOR-EV corpora and train the model on the multiplex-lattice dataset. Although we have access to

OLCF's Frontier supercomputer, these requirements may limit accessibility for other researchers.

Future work can extend the lattice to other organ systems and disease states, yielding a more comprehensive model checkpoint. Extending the lattice across species could support cross-species mechanistic reasoning. Additional evidence modes and curriculum stages could also teach more operators and support more complex reasoning across modalities.

Bibliography

[1] Shibla Abdulla et al. "CZ CELLxGENE Discover: a single-cell data platform for scalable exploration, analysis and modeling of aggregated data". In: *Nucleic Acids Research* 53.D1 (2025), pp. D886–D900.

[2] Guillaume Alain and Yoshua Bengio. "Understanding intermediate layers using linear classifier probes". In: *arXiv preprint arXiv:1610.01644* (2016).

[3] Scott Atchley et al. "Frontier: Exploring Exascale". In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. SC '23. Denver, CO, USA: Association for Computing Machinery, 2023. ISBN: 9798400701092. DOI: 10.1145/3581784.3607089. URL: https://doi.org/10.1145/3581784.3607089.

[4] Christina B. Azodi et al. "Benchmarking Parametric and Machine Learning Models for Genomic Prediction of Complex Traits". In: *G3: Genes, Genomes, Genetics* 9.11 (2019), pp. 3691–3702. DOI: 10.1534/g3.119.400498.

[5] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. "Layer normalization". In: *arXiv preprint arXiv:1607.06450* (2016).

[6] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. "Neural machine translation by jointly learning to align and translate". In: *3rd International Conference on Learning Representations (ICLR)*. 2015. arXiv: 1409.0473 [cs.CL].

[7] Albert-László Barabási, Natali Gulbahce, and Joseph Loscalzo. "Network medicine: a network-based approach to human disease". In: *Nature reviews genetics* 12.1 (2011), pp. 56–68.

[8] Albert-Laszlo Barabasi and Zoltan N Oltvai. "Network biology: understanding the cell's functional organization". In: *Nature reviews genetics* 5.2 (2004), pp. 101–113.

[9] Sumanta Basu et al. "Iterative random forests to discover predictive and stable high-order interactions". In: *Proceedings of the National Academy of Sciences* 115.8 (2018), pp. 1943–1948.

[10] Lucas Alexandre Batista et al. "Large scale genotype by environment model development competition explores new ways of predicting maize yield from genomic data". Manuscript in preparation. 2026.

[11] Pau Bellot, Gustavo de los Campos, and Miguel Pérez-Enciso. "Can deep learning improve genomic prediction of complex human traits?" In: *Genetics* 210.3 (2018), pp. 809–819. DOI: 10.1534/genetics.118.301298.

[12] Yoshua Bengio et al. "A neural probabilistic language model". In: *Journal of Machine Learning Research* 3 (2003), pp. 1137–1155. URL: https://jmlr.org/papers/v3/bengio03a.html.

[13] Emma J. Bennett, Jeremy A. Roberts, and Carol Wagstaff. "The role of the pod in seed development: strategies for manipulating yield". In: *New Phytologist* 190.4 (2011), pp. 838–853. DOI: https://doi.org/10.1111/j.1469-8137.2011.03714.x. eprint: https://nph.onlinelibrary.wiley.com/doi/pdf/10.1111/j.1469-8137.2011.03714.x. URL: https://nph.onlinelibrary.wiley.com/doi/abs/10.1111/j.1469-8137.2011.03714.x.

[14] Bgee Group. *HsapDv: Human Developmental Stages Ontology*. OBO Foundry. http://purl.obolibrary.org/obo/hsapdv.owl. 2024.

[15] Lukas Biewald. *Experiment Tracking with Weights and Biases*. Software available from wandb.com. 2020. URL: https://www.wandb.com/.

[16] Kevin A. Bird et al. "Structure and sequence evolution in the pennycress (*Thlaspi arvense*) pangenome". In: *New Phytologist* 250.5 (2026), pp. 2723–2741. DOI: 10.1111/nph.71111.

[17] Katy Börner et al. "Anatomical structures, cell types and biomarkers of the Human Reference Atlas". In: *Nature Cell Biology* 23.11 (2021), pp. 1117–1128.

[18] James Bradbury et al. *JAX: composable transformations of Python+NumPy programs*. Version 0.3.13. 2018. URL: http://github.com/jax-ml/jax.

[19] G. Bradski. "The OpenCV Library". In: *Dr. Dobb's Journal of Software Tools* (2000).

[20] Tom Brown et al. "Language models are few-shot learners". In: *Advances in neural information processing systems* 33 (2020), pp. 1877–1901.

[21] Samuel Cahyawijaya et al. "SNP2Vec: Scalable self-supervised pre-training for genome-wide association study". In: *Proceedings of the 21st Workshop on Biomedical Language Processing*. Association for Computational Linguistics, 2022, pp. 140–154. DOI: 10.18653/v1/2022.bionlp-1.14.

[22] Nicolas Carion et al. "Sam 3: Segment anything with concepts". In: *arXiv preprint arXiv:2511.16719* (2025).

[23] Liang-Chieh Chen et al. "Encoder-decoder with atrous separable convolution for semantic image segmentation". In: *Proceedings of the European conference on computer vision (ECCV)*. 2018, pp. 801–818.

[24] Qiuyue Chen, Jacob D. Washburn, Dayane Cristina Lima, et al. "Genomes to fields 2024 maize genotype by environment prediction competition". In: *BMC Research Notes* 19.1 (2026), p. 113. DOI: 10.1186/s13104-026-07629-5.

[25] Stanley F Chen and Joshua Goodman. "An empirical study of smoothing techniques for language modeling". In: *Computer Speech & Language* 13.4 (1999), pp. 359–393. DOI: 10.1006/csla.1999.0128.

[26] Yiqun Chen and James Zou. "Simple and effective embedding model for single-cell biology built from ChatGPT". In: *Nature biomedical engineering* 9.4 (2025), pp. 483–493.

[27] Bowen Cheng, Alexander G Schwing, and Alexander Kirillov. "Per-pixel classification is not all you need for semantic segmentation". In: *Advances in neural information processing systems* 34 (2021).

[28] Bowen Cheng et al. "Masked-attention mask transformer for universal image segmentation". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022.

[29] Paul F Christiano et al. "Deep reinforcement learning from human preferences". In: *Advances in neural information processing systems* 30 (2017).

[30] Ashley Cliff et al. "A high-performance computing implementation of iterative random forest for the creation of predictive expression networks". In: *Genes* 10.12 (2019), p. 996.

[31] James W. Clohessy et al. "A Low-Cost Automated System for High-Throughput Phenotyping of Single Oat Seeds". In: *The Plant Phenome Journal* 1 (2018), pp. 1–13. DOI: 10.2135/tppj2018.07.0005.

[32] Rachel Combs-Giroir et al. "Morpho-physiological and transcriptomic responses of field pennycress to waterlogging". In: *Frontiers in Plant Science* 15 (2024), p. 1478507. DOI: 10.3389/fpls.2024.1478507.

[33] Lenore Cowen et al. "Network propagation: a universal amplifier of genetic associations". In: *Nature Reviews Genetics* 18.9 (2017), pp. 551–562.

[34] José Crossa et al. "Genomic selection in plant breeding: methods, models, and perspectives". In: *Trends in plant science* 22.11 (2017), pp. 961–975.

[35] Gabriela Csurka, Diane Larlus, and Florent Perronnin. "What is a good evaluation measure for semantic segmentation?" In: *Proceedings of the British Machine Vision Conference (BMVC)*. 2013. DOI: 10.5244/C.27.32.

[36] George Cybenko. "Approximation by superpositions of a sigmoidal function". In: *Mathematics of control, signals and systems* 2.4 (1989), pp. 303–314.

[37] Hugo Dalla-Torre et al. "Nucleotide Transformer: building and evaluating robust foundation models for human genomics". In: *Nature Methods* 22.2 (2025), pp. 287–297.

[38] Manlio De Domenico et al. "Mathematical formulation of multilayer networks". In: *Physical Review X* 3.4 (2013), p. 041022.

[39] Jia Deng et al. "Imagenet: A large-scale hierarchical image database". In: *2009 IEEE conference on computer vision and pattern recognition*. Ieee. 2009, pp. 248–255.

[40] Jacob Devlin et al. "Bert: Pre-training of deep bidirectional transformers for language understanding". In: *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 2019, pp. 4171–4186.

[41] Julien Di Berardino et al. "Autophagy controls resource allocation and protein storage accumulation in Arabidopsis seeds". In: *Journal of Experimental Botany* 69.6 (2018), pp. 1403–1414. DOI: 10.1093/jxb/ery012.

[42] Alexander D Diehl et al. "The Cell Ontology 2016: enhanced content, modularization, and ontology interoperability". In: *Journal of Biomedical Semantics* 7 (2016), p. 44.

[43] Alexey Dosovitskiy. "An image is worth 16x16 words: Transformers for image recognition at scale". In: *arXiv preprint arXiv:2010.11929* (2020).

[44] Christine M. Ellis et al. “AUXIN RESPONSE FACTOR1 and AUXIN RESPONSE FACTOR2 regulate senescence and floral organ abscission in Arabidopsis thaliana”. In: *Development* 132.20 (2005), pp. 4563–4574. DOI: 10.1242/dev.02012.

[45] Jeffrey L Elman. “Finding structure in time”. In: *Cognitive Science* 14.2 (1990), pp. 179–211. DOI: 10.1207/s15516709cog1402_1.

[46] Kieran Elmes et al. “SNVformer: An Attention-based Deep Neural Network for GWAS Data”. In: *bioRxiv* (2022), pp. 2022–07.

[47] Mark Everingham et al. “The pascal visual object classes (voc) challenge”. In: *International journal of computer vision* 88 (2010), pp. 303–338.

[48] Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. “Talk like a graph: Encoding graphs for large language models”. In: *International Conference on Learning Representations (ICLR)*. 2024. arXiv: 2310.04560 [cs.LG].

[49] Li Fei-Fei, Rob Fergus, and Pietro Perona. “Learning generative visual models from few training examples: An incremental bayesian approach tested on 101 object categories”. In: *2004 conference on computer vision and pattern recognition workshop*. IEEE. 2004, pp. 178–178.

[50] Christiane Fellbaum. *WordNet: An electronic lexical database*. MIT press, 1998.

[51] Kunihiko Fukushima. “Cognitron: A self-organizing multilayered neural network”. In: *Biological cybernetics* 20.3 (1975), pp. 121–136.

[52] Kunihiko Fukushima. “Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position”. In: *Biological cybernetics* 36.4 (1980), pp. 193–202.

[53] Margarita Garcia-Hernandez et al. “TAIR: a resource for integrated Arabidopsis data”. In: *Functional & integrative genomics* 2.6 (2002), pp. 239–253.

[54] Lei Ge et al. "Arabidopsis ROOT UVB SENSITIVE2/WEAK AUXIN RESPONSE1 is required for polar auxin transport". In: *The Plant Cell* 22.6 (2010), pp. 1749–1761. DOI: 10.1105/tpc.110.074195.

[55] Ross Girshick. "Fast r-cnn". In: *Proceedings of the IEEE international conference on computer vision*. 2015, pp. 1440–1448.

[56] Xavier Glorot and Yoshua Bengio. "Understanding the difficulty of training deep feedforward neural networks". In: *Proceedings of the thirteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings. 2010, pp. 249–256.

[57] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. "Deep sparse rectifier neural networks". In: *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings. 2011, pp. 315–323.

[58] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. http://www.deeplearningbook.org. MIT Press, 2016.

[59] Anne Guiboileau et al. "Autophagy machinery controls nitrogen remobilization at the whole-plant level under both limiting and ample nitrate conditions in Arabidopsis". In: *New Phytologist* 194.3 (2012), pp. 732–740. DOI: 10.1111/j.1469-8137.2012.04084.x.

[60] Daya Guo et al. "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning". In: *arXiv preprint arXiv:2501.12948* (2025).

[61] Leland H Hartwell et al. "From molecular to modular cell biology". In: *Nature* 402.Suppl 6761 (1999), pp. C47–C52.

[62] Kaiming He et al. "Deep residual learning for image recognition". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.

[63] Kaiming He et al. “Delving deep into rectifiers: Surpassing human-level performance on imagenet classification”. In: *Proceedings of the IEEE international conference on computer vision*. 2015, pp. 1026–1034.

[64] Kaiming He et al. “Mask r-cnn”. In: *Proceedings of the IEEE international conference on computer vision*. 2017, pp. 2961–2969.

[65] Kaiming He et al. “Masked autoencoders are scalable vision learners”. In: *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022, pp. 15979–15988.

[66] Xiaoxin He et al. “G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2024. arXiv: 2402.07630 [cs.LG].

[67] Donald O Hebb. *The Organization of Behavior: A Neuropsychological Theory*. New York: John Wiley & Sons, 1949.

[68] Nicolas Heslot et al. “Integrating environmental covariates and crop modeling into the genomic selection framework to predict genotype by environment interactions”. In: *Theoretical and Applied Genetics* 127.2 (2014), pp. 463–480. DOI: 10.1007/s00122-013-2231-5.

[69] William G Hill, Michael E Goddard, and Peter M Visscher. “Data and theory point to mainly additive genetic variance for complex traits”. In: *PLoS Genetics* 4.2 (2008), e1000008.

[70] Geoffrey E Hinton, Simon Osindero, and Yee-Whye Teh. “A fast learning algorithm for deep belief nets”. In: *Neural computation* 18.7 (2006), pp. 1527–1554.

[71] Geoffrey E. Hinton et al. “Improving neural networks by preventing co-adaptation of feature detectors”. In: *arXiv preprint arXiv:1207.0580* (2012).

[72] Sepp Hochreiter and Jürgen Schmidhuber. “Long short-term memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.

[73] Jordan Hoffmann et al. “Training compute-optimal large language models”. In: *arXiv preprint arXiv:2203.15556* 10 (2022).

[74] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. “Multilayer feedforward networks are universal approximators”. In: *Neural networks* 2.5 (1989), pp. 359–366.

[75] Edward J Hu et al. “Lora: Low-rank adaptation of large language models.” In: *Iclr* 1.2 (2022), p. 3.

[76] Wei Hua et al. “Maternal control of seed oil content in *Brassica napus*: the role of silique wall photosynthesis”. In: *The Plant Journal* 69.3 (2012), pp. 432–444. DOI: 10.1111/j.1365-313X.2011.04802.x.

[77] Meng Huang et al. “BLINK: a package for the next level of genome-wide association studies with both individuals and markers in the millions”. In: *GigaScience* 8.2 (2019), giy154. DOI: 10.1093/gigascience/giy154.

[78] Yanping Huang et al. *GPipe: Efficient training of giant neural networks using pipeline parallelism*. 2019. arXiv: 1811.06965 [cs.CV].

[79] David H Hubel and Torsten N Wiesel. “Receptive fields and functional architecture in two nonstriate visual areas (18 and 19) of the cat”. In: *Journal of Neurophysiology* 28.2 (1965), pp. 229–289.

[80] David H Hubel and Torsten N Wiesel. “Receptive fields of single neurones in the cat’s striate cortex”. In: *The Journal of Physiology* 148.3 (1959), pp. 574–591.

[81] David H Hubel and Torsten N Wiesel. “Receptive fields, binocular interaction and functional architecture in the cat’s visual cortex”. In: *The Journal of physiology* 160.1 (1962), pp. 106–154.

[82] Peter J. Huber. “Robust Estimation of a Location Parameter”. In: *The Annals of Mathematical Statistics* 35.1 (1964), pp. 73–101. DOI: 10.1214/aoms/1177703732.

[83] Lawrence Hubert and Phipps Arabie. “Comparing partitions”. In: *Journal of Classification* 2.1 (1985), pp. 193–218.

[84] Trey Ideker and Nevan J Krogan. “Differential network biology”. In: *Molecular systems biology* 8 (2012), p. 565.

[85] Yuko Inoue et al. “AtATG genes, homologs of yeast autophagy genes, are involved in constitutive autophagy in Arabidopsis root tip cells”. In: *Plant and Cell Physiology* 47.12 (2006), pp. 1641–1652. DOI: 10.1093/pcp/pcl031.

[86] Sergey Ioffe and Christian Szegedy. “Batch normalization: Accelerating deep network training by reducing internal covariate shift”. In: *International conference on machine learning*. pmlr. 2015, pp. 448–456.

[87] Fabian Isensee et al. “nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation”. In: *Nature methods* 18.2 (2021), pp. 203–211.

[88] ISO. *ISO/IEC 14882:2011 Information technology — Programming languages — C++*. Geneva, Switzerland: International Organization for Standardization, Feb. 2012. URL: http://www.iso.org.

[89] Sam Ade Jacobs et al. *DeepSpeed Ulysses: System optimizations for enabling training of extreme long sequence transformer models*. 2023. arXiv: 2309.14509 [cs.LG].

[90] Diego Jarquín et al. “A reaction norm model for genomic selection using high-dimensional genomic and environmental data”. In: *Theoretical and Applied Genetics* 127.3 (2014), pp. 595–607. DOI: 10.1007/s00122-013-2243-1.

[91] Yanrong Ji et al. “DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome”. In: *Bioinformatics* 37.15 (2021), pp. 2112–2120.

[92] Yangqing Jia et al. “Caffe: Convolutional Architecture for Fast Feature Embedding”. In: *arXiv preprint arXiv:1408.5093* (2014).

[93] David Kainer et al. "RWRtoolkit: multi-omic network analysis using random walks on multiplex networks in any species". In: *GigaScience* 14 (2025), giaf028.

[94] Sangwoo Kang et al. "Autophagy-related (ATG) 11, ATG9 and the phosphatidylinositol 3-kinase control ATG2-mediated formation of autophagosomes in Arabidopsis". In: *Plant Cell Reports* 37.4 (2018), pp. 653–664. DOI: 10.1007/s00299-018-2258-9.

[95] Jared Kaplan et al. "Scaling laws for neural language models". In: *arXiv preprint arXiv:2001.08361* (2020).

[96] Jens Kattge et al. "TRY–a global database of plant traits". In: *Global change biology* 17.9 (2011), pp. 2905–2935.

[97] Saeed Khaki and Lizhi Wang. "Crop yield prediction using deep neural networks". In: *Frontiers in Plant Science* 10 (2019), p. 621. DOI: 10.3389/fpls.2019.00621.

[98] Mahdi Khozaei et al. "Overexpression of plastid transketolase in tobacco results in a thiamine auxotrophic phenotype". In: *The Plant Cell* 27.2 (2015), pp. 432–447. DOI: 10.1105/tpc.114.131011.

[99] Chan Yeong Kim et al. "HumanNet v3: an improved database of human gene networks for disease research". In: *Nucleic Acids Research* 50.D1 (2022), pp. D632–D639.

[100] Diederik P Kingma. "Adam: A method for stochastic optimization". In: *arXiv preprint arXiv:1412.6980* (2014).

[101] Thomas N Kipf and Max Welling. "Semi-supervised classification with graph convolutional networks". In: *arXiv preprint arXiv:1609.02907* (2016).

[102] Alexander Kirillov et al. "Segment anything". In: *Proceedings of the IEEE/CVF international conference on computer vision*. 2023, pp. 4015–4026.

[103] Sebastian Köhler et al. "Walking the interactome for prioritization of candidate disease genes". In: *The American Journal of Human Genetics* 82.4 (2008), pp. 949–958.

[104] Takeshi Kojima et al. "Large language models are zero-shot reasoners". In: *Advances in neural information processing systems* 35 (2022), pp. 22199–22213.

[105] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. "Imagenet classification with deep convolutional neural networks". In: *Advances in neural information processing systems* 25 (2012).

[106] John Lagergren et al. *Few-Shot Learning Enables Population-Scale Analysis of Leaf Traits in Populus trichocarpa*. 2023. arXiv: 2301.10351 [cs.CV].

[107] Louis Tung Faat Lai et al. "Subnanometer resolution cryo-EM structure of Arabidopsis thaliana ATG9". In: *Autophagy* 16.3 (2020), pp. 575–583. DOI: 10.1080/15548627.2019.1639300.

[108] Colin D. Leasure et al. "ROOT UV-B SENSITIVE2 acts with ROOT UV-B SENSITIVE1 in a root ultraviolet B-sensing pathway". In: *Plant Physiology* 150.4 (2009), pp. 1902–1915. DOI: 10.1104/pp.109.139253.

[109] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. "Deep learning". In: *nature* 521.7553 (2015), pp. 436–444.

[110] Yann LeCun et al. "Gradient-based learning applied to document recognition". In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.

[111] Yann LeCun et al. "Handwritten digit recognition with a back-propagation network". In: *Advances in neural information processing systems* 2 (1989).

[112] Dmitry Lepikhin et al. *GShard: Scaling giant models with conditional computation and automatic sharding*. 2020. arXiv: 2006.16668 [cs.CL].

[113] Kenneth Li et al. "Emergent world representations: Exploring a sequence model trained on a synthetic task". In: *International Conference on Learning Representations (ICLR)*. 2023. arXiv: 2210.13382 [cs.LG].

[114] Lei Li, Qin Zhang, and Danfeng Huang. "A review of imaging techniques for plant phenotyping". In: *Sensors* 14.11 (2014), pp. 20078–20111.

[115] Shen Li et al. *PyTorch distributed: Experiences on accelerating data parallel training*. 2020. arXiv: 2006.15704 [cs.DC].

[116] Lawrence I-Kuei Lin. "A Concordance Correlation Coefficient to Evaluate Reproducibility". In: *Biometrics* 45.1 (1989), pp. 255–268. DOI: 10.2307/2532051.

[117] Tsung-Yi Lin et al. "Microsoft coco: Common objects in context". In: *European conference on computer vision*. Springer. 2014, pp. 740–755.

[118] Seppo Linnainmaa. "The representation of the cumulative rounding error of an algorithm as a Taylor expansion of the local rounding errors". PhD thesis. Master's Thesis (in Finnish), Univ. Helsinki, 1970.

[119] Xiaolei Liu et al. "Iterative usage of fixed and random effect models for powerful and efficient genome-wide association studies". In: *PLoS Genetics* 12.2 (2016), e1005767. DOI: 10.1371/journal.pgen.1005767.

[120] Zhenning Liu et al. "ARF2–ARF4 and ARF5 are essential for female and male gametophyte development in Arabidopsis". In: *Plant and Cell Physiology* 59.1 (2018), pp. 179–189. DOI: 10.1093/pcp/pcx174.

[121] Marco Lopez-Cruz et al. "Leveraging data from the Genomes-to-Fields Initiative to investigate genotype-by-environment interactions in maize in North America". In: *Nature communications* 14.1 (2023), p. 6904.

[122] Ilya Loshchilov and Frank Hutter. "Decoupled weight decay regularization". In: *arXiv preprint arXiv:1711.05101* (2017).

[123] Pan Lu et al. "Eubiota: Modular Agentic AI for Autonomous Discovery in the Gut Microbiome". In: *bioRxiv* (2026). DOI: 10.64898/2026.02.27.708412. eprint: https://www.biorxiv.org/content/early/2026/02/28/2026.02.

27.708412.full.pdf. URL: https://www.biorxiv.org/content/early/2026/02/28/2026.02.27.708412.

[124] Mengqian Luo et al. "Arabidopsis AUTOPHAGY-RELATED2 is essential for ATG18a and ATG9 trafficking during autophagosome closure". In: *Plant Physiology* 193.1 (2023), pp. 304–321. DOI: 10.1093/plphys/kiad287.

[125] Aman Madaan et al. "Self-refine: Iterative refinement with self-feedback". In: *Advances in neural information processing systems* 36 (2023), pp. 46534–46594.

[126] Martín Abadi et al. *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. Software available from tensorflow.org. 2015. URL: https://www.tensorflow.org/.

[127] Warren S McCulloch and Walter Pitts. "A logical calculus of the ideas immanent in nervous activity". In: *The bulletin of mathematical biophysics* 5.4 (1943), pp. 115–133.

[128] Jörg Menche et al. "Uncovering disease-disease relationships through the incomplete interactome". In: *Science* 347.6224 (2015), p. 1257601.

[129] Meta AI. *The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation*. Meta AI Blog. 2025. URL: https://ai.meta.com/blog/llama-4-multimodal-intelligence/ (visited on 09/01/2026).

[130] Theo HE Meuwissen, Ben J Hayes, and ME1461589 Goddard. "Prediction of total genetic value using genome-wide dense marker maps". In: *genetics* 157.4 (2001), pp. 1819–1829.

[131] Tomas Mikolov et al. *Efficient estimation of word representations in vector space*. 2013. arXiv: 1301.3781 [cs.CL].

[132] Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA, USA: MIT Press, 1969.

[133] Sharada P. Mohanty, David P. Hughes, and Marcel Salathé. "Using Deep Learning for Image-Based Plant Disease Detection". In: *Frontiers in Plant Science* 7 (2016), p. 1419. DOI: 10.3389/fpls.2016.01419.

[134] Abelardo Montesinos-López et al. "Multi-environment Genomic Prediction of Plant Traits Using Deep Learners With Dense Architecture". In: *G3: Genes, Genomes, Genetics* 8.12 (2018), pp. 3813–3828. DOI: 10.1534/g3.118.200740.

[135] Osval A. Montesinos-López et al. "A review of multimodal deep learning methods for genomic-enabled prediction in plant breeding". In: *Genetics* 228.4 (2024), iyae161. DOI: 10.1093/genetics/iyae161.

[136] Osval Antonio Montesinos-López et al. "A review of deep learning applications for genomic selection". In: *BMC genomics* 22.1 (2021), p. 19. DOI: 10.1186/s12864-020-07319-x. URL: https://doi.org/10.1186/s12864-020-07319-x.

[137] Bryan R Moser et al. "Production and evaluation of biodiesel from field pennycress (Thlaspi arvense L.) oil". In: *Energy & Fuels* 23.8 (2009), pp. 4149–4155.

[138] Peter J Mucha et al. "Community structure in time-dependent, multiscale, and multiplex networks". In: *science* 328.5980 (2010), pp. 876–878.

[139] Christopher J Mungall et al. "Uberon, an integrative multi-species anatomy ontology". In: *Genome Biology* 13.1 (2012), R5.

[140] Reiichiro Nakano et al. "Webgpt: Browser-assisted question-answering with human feedback". In: *arXiv preprint arXiv:2112.09332* (2021).

[141] Deepak Narayanan et al. *Efficient large-scale language model training on GPU clusters using Megatron-LM*. 2021. arXiv: 2104.04473 [cs.CL].

[142] Justin B. Nichol, Shakshi A. Dutt, and Marcus A. Samuel. "The Shock of Shatter: Understanding Silique and Silicle Dehiscence for Improving Oilseed Crops in Brassicaceae". In: *Plant Direct* 9.4 (2025), e70058. DOI: 10.1002/pld3.70058.

[143] Adam Nunn et al. "Chromosome-level *Thlaspi arvense* genome provides new tools for translational research and for a newly domesticated cash cover crop of the cooler climates". In: *Plant Biotechnology Journal* 20.5 (2022), pp. 944–963. DOI: 10.1111/pbi.13775.

[144] NVIDIA. *NVIDIA Nemotron 3: Efficient and Open Intelligence*. White Paper. 2025. URL: https://arxiv.org/abs/2512.20856.

[145] OpenAI. *gpt-oss-120b & gpt-oss-20b Model Card*. 2025. arXiv: 2508.10925 [cs.CL]. URL: https://arxiv.org/abs/2508.10925.

[146] Long Ouyang et al. "Training language models to follow instructions with human feedback". In: *Advances in neural information processing systems* 35 (2022), pp. 27730–27744.

[147] Dario Paolo et al. "Genetic interaction of SEEDSTICK, GORDITA and AUXIN RESPONSE FACTOR 2 during seed development". In: *Genes* 12.8 (2021), p. 1189. DOI: 10.3390/genes12081189.

[148] Adam Paszke et al. "Pytorch: An imperative style, high-performance deep learning library". In: *Advances in neural information processing systems* 32 (2019).

[149] Bowen Peng et al. *YaRN: Efficient context window extension of large language models*. 2023. arXiv: 2309.00071 [cs.CL].

[150] Xinyang Pu et al. "ClassWise-SAM-adapter: Parameter-efficient fine-tuning adapts segment anything to SAR domain for semantic segmentation". In: *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 18 (2025), pp. 4791–4804.

[151] Pranav Putta et al. "Agent q: Advanced reasoning and learning for autonomous ai agents". In: *arXiv preprint arXiv:2408.07199* (2024).

[152] Yujia Qin et al. "Toolllm: Facilitating large language models to master 16000+ real-world apis". In: *arXiv preprint arXiv:2307.16789* (2023).

[153] Alec Radford et al. *Improving language understanding by generative pre-training*. Tech. rep. OpenAI, 2018.

[154] Alec Radford et al. "Language models are unsupervised multitask learners". In: *OpenAI blog* 1.8 (2019), p. 9.

[155] Alec Radford et al. "Learning transferable visual models from natural language supervision". In: *Proceedings of the 38th International Conference on Machine Learning*. 2021, pp. 8748–8763.

[156] Rafael Rafailov et al. "Direct preference optimization: Your language model is secretly a reward model". In: *Advances in neural information processing systems* 36 (2023), pp. 53728–53741.

[157] Rajat Raina, Anand Madhavan, and Andrew Y Ng. "Large-scale deep unsupervised learning using graphics processors". In: *Proceedings of the 26th annual international conference on machine learning*. 2009, pp. 873–880.

[158] Samyam Rajbhandari et al. *ZeRO: Memory optimizations toward training trillion parameter models*. 2020. arXiv: 1910.02054 [cs.LG].

[159] Jeff Rasley et al. "Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters". In: *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 2020, pp. 3505–3506.

[160] Nikhila Ravi et al. "Sam 2: Segment anything in images and videos". In: *arXiv preprint arXiv:2408.00714* (2024).

[161] Deepak K. Ray et al. "Yield Trends Are Insufficient to Double Global Crop Production by 2050". In: *PLoS ONE* 8.6 (2013), e66428. DOI: 10.1371/journal.pone.0066428.

[162] Shaoqing Ren et al. "Faster R-CNN: Towards real-time object detection with region proposal networks". In: *IEEE transactions on pattern analysis and machine intelligence* 39.6 (2016), pp. 1137–1149.

[163] Agostinho G. Rocha et al. "Phosphorylation of Arabidopsis transketolase at Ser428 provides a potential paradigm for the metabolic control of chloroplast carbon metabolism". In: *Biochemical Journal* 458.2 (2014), pp. 313–322. DOI: 10.1042/BJ20130631.

[164] Anna R Rogers et al. "The importance of dominance and genotype-by-environment interactions on grain yield variation in a large-scale public cooperative maize experiment". In: *G3* 11.2 (2021), jkaa050.

[165] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. "U-net: Convolutional networks for biomedical image segmentation". In: *International Conference on Medical image computing and computer-assisted intervention*. Springer. 2015, pp. 234–241.

[166] Frank Rosenblatt. "The perceptron: a probabilistic model for information storage and organization in the brain." In: *Psychological review* 65.6 (1958), p. 386.

[167] G. van Rossum. *Python tutorial*. Tech. rep. CS-R9526. Amsterdam: Centrum voor Wiskunde en Informatica (CWI), May 1995.

[168] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. "Learning representations by back-propagating errors". In: *Nature* 323.6088 (1986), pp. 533–536. DOI: 10.1038/323533a0. URL: https://www.nature.com/articles/323533a0.

[169] Olga Russakovsky et al. "ImageNet Large Scale Visual Recognition Challenge". In: *International Journal of Computer Vision (IJCV)* 115.3 (2015), pp. 211–252. DOI: 10.1007/s11263-015-0816-y.

[170] Shibani Santurkar et al. "How Does Batch Normalization Help Optimization?" In: *Advances in Neural Information Processing Systems*. Vol. 31. 2018.

[171] Timo Schick et al. "Toolformer: Language models can teach themselves to use tools". In: *Advances in neural information processing systems* 36 (2023), pp. 68539–68551.

[172] Yair Schiff et al. "Caduceus: Bi-directional equivariant long-range dna sequence modeling". In: *Proceedings of machine learning research* 235 (2024), p. 43632.

[173] Marie C. Schruff et al. "The AUXIN RESPONSE FACTOR 2 gene of Arabidopsis links auxin signalling, cell division, and the size of seeds and other organs". In: *Development* 133.2 (2006), pp. 251–261. DOI: 10.1242/dev.02194.

[174] John Schulman et al. "Proximal policy optimization algorithms". In: *arXiv preprint arXiv:1707.06347* (2017).

[175] John Schulman et al. "Trust region policy optimization". In: *International conference on machine learning*. PMLR. 2015, pp. 1889–1897.

[176] John C Sedbrook, Winthrop B Phippen, and M David Marks. "New approaches to facilitate rapid domestication of a wild plant to an oilseed crop: example pennycress (Thlaspi arvense L.)" In: *Plant Science* 227 (2014), pp. 122–132.

[177] Vincent Segura et al. "An efficient multi-locus mixed-model approach for genome-wide association studies in structured populations". In: *Nature Genetics* 44.7 (2012), pp. 825–830. DOI: 10.1038/ng.2314.

[178] Claude E Shannon. "A mathematical theory of communication". In: *The Bell System Technical Journal* 27.3 (1948), pp. 379–423. DOI: 10.1002/j.1538-7305.1948.tb01338.x.

[179] Zhihong Shao et al. "Deepseekmath: Pushing the limits of mathematical reasoning in open language models". In: *arXiv preprint arXiv:2402.03300* (2024).

[180] Noam Shazeer et al. "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer". In: *International Conference on Learning Representations*. 2017. URL: https://arxiv.org/abs/1701.06538.

[181] Wenyun Shen et al. "Involvement of a glycerol-3-phosphate dehydrogenase in modulating the NADH/NAD+ ratio provides evidence of a mitochondrial glycerol-3-phosphate shuttle in Arabidopsis". In: *The Plant Cell* 18.2 (2006), pp. 422–441. DOI: 10.1105/tpc.105.039750.

[182] Kwang Deok Shin, Han Nim Lee, and Taijoon Chung. "A revised assay for monitoring autophagic flux in Arabidopsis thaliana reveals involvement of AUTOPHAGY-RELATED9 in autophagy". In: *Molecules and Cells* 37.5 (2014), pp. 399–405. DOI: 10.14348/molcells.2014.0042.

[183] Noah Shinn et al. "Reflexion: Language agents with verbal reinforcement learning". In: *Advances in neural information processing systems* 36 (2023), pp. 8634–8652.

[184] Mohammad Shoeybi et al. *Megatron-LM: Training multi-billion parameter language models using model parallelism*. 2019. arXiv: 1909.08053 [cs.CL].

[185] Johnathon Shook et al. "Crop yield prediction integrating genotype and weather variables using deep learning". In: *Plos one* 16.6 (2021), e0252402.

[186] Karen Simonyan and Andrew Zisserman. "Very deep convolutional networks for large-scale image recognition". In: *arXiv preprint arXiv:1409.1556* (2014).

[187] Aaditya Singh et al. *OpenAI GPT-5 System Card*. 2025. arXiv: 2601.03267 [cs.CL]. URL: https://arxiv.org/abs/2601.03267.

[188] Nitish Srivastava et al. “Dropout: a simple way to prevent neural networks from overfitting”. In: *The journal of machine learning research* 15.1 (2014), pp. 1929–1958.

[189] Jianlin Su et al. “RoFormer: Enhanced transformer with rotary position embedding”. In: *Neurocomputing* 568 (2024), p. 127063.

[190] Kyle A Sullivan et al. “Analyses of GWAS signal using GRIN identify additional genes contributing to suicidal behavior”. In: *Communications Biology* 7.1 (2024), p. 1360.

[191] Kyle A. Sullivan et al. “MENTOR: Multiplex Embedding of Networks for Team-Based Omics Research”. In: *bioarXiv* (2024). DOI: 10.1101/2024.07.17.603821.

[192] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. “Sequence to sequence learning with neural networks”. In: *Advances in Neural Information Processing Systems*. Vol. 27. 2014. arXiv: 1409.3215 [cs.CL].

[193] Richard S. Sutton. “Temporal Credit Assignment in Reinforcement Learning”. COINS Technical Report 84-05. PhD thesis. Amherst, MA 01003, USA: University of Massachusetts, Amherst, 1984.

[194] Christian Szegedy et al. “Going deeper with convolutions”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2015, pp. 1–9.

[195] Shawn Zheng Kai Tan et al. “Brain Data Standards – A method for building data-driven cell-type ontologies”. In: *Scientific Data* 10 (2023), p. 50.

[196] Takanari Tanabata et al. “SmartGrain: High-Throughput Phenotyping Software for Measuring Seed Shape through Image Analysis”. In: *Plant Physiology* 160.4 (2012), pp. 1871–1880. DOI: 10.1104/pp.112.205120.

[197] Kimi Team et al. *Kimi K2: Open Agentic Intelligence*. 2026. arXiv: 2507.20534 [cs.LG]. URL: https://arxiv.org/abs/2507.20534.

[198] Texas Tech Davis College of Agricultural Sciences & Natural Resources. *Research Farm at New Deal*. 2025. URL: https://www.depts.ttu.edu/agriculturalsciences/About/facilities/maps/resFarm.php (visited on 09/21/2025).

[199] Hanghang Tong, Christos Faloutsos, and Jia-Yu Pan. "Fast random walk with restart and its applications". In: *Sixth international conference on data mining (ICDM'06)*. IEEE. 2006, pp. 613–622.

[200] Mason Tuite et al. "Rapid and non-destructive screening of seed components in domesticated pennycress using near-infrared spectroscopy". In: *Industrial Crops and Products* 235 (2025), p. 121752. DOI: 10.1016/j.indcrop.2025.121752.

[201] Jordan Ubbens et al. "The use of plant models in deep learning: an application to leaf counting in rosette plants". In: *Plant Methods* 14 (2018), p. 6. DOI: 10.1186/s13007-018-0273-z.

[202] Emil Uffelmann et al. "Genome-wide association studies". In: *Nature Reviews Methods Primers* 1.1 (2021), p. 59.

[203] L.C. Uzal et al. "Seed-per-pod estimation for plant breeding using deep learning". In: *Computers and Electronics in Agriculture* 150 (2018), pp. 196–204. ISSN: 0168-1699. DOI: https://doi.org/10.1016/j.compag.2018.04.024. URL: https://www.sciencedirect.com/science/article/pii/S016816991731582X.

[204] P.M. VanRaden. "Efficient Methods to Compute Genomic Predictions". In: *Journal of Dairy Science* 91.11 (2008), pp. 4414–4423. ISSN: 0022-0302. DOI: https://doi.org/10.3168/jds.2007-0980. URL: https://www.sciencedirect.com/science/article/pii/S0022030208709901.

[205] Oron Vanunu et al. "Associating genes and protein complexes with disease via network propagation". In: *PLoS computational biology* 6.1 (2010), e1000641.

[206] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).

[207] Petar Veličković et al. “Graph attention networks”. In: *arXiv preprint arXiv:1710.10903* (2017).

[208] A. H. Vlot et al. “The MENTOR Interpretation Agent: From Network Embeddings to Mechanistic Narratives via Retrieval-Augmented LLMs”. Manuscript in preparation. 2026.

[209] Stéfan van der Walt et al. “scikit-image: image processing in Python”. In: *PeerJ* 2 (June 2014), e453. ISSN: 2167-8359. DOI: 10.7717/peerj.453. URL: https://doi.org/10.7717/peerj.453.

[210] Jiabo Wang and Zhiwu Zhang. “GAPIT version 3: boosting power and accuracy for genomic association and prediction”. In: *Genomics, proteomics & bioinformatics* 19.4 (2021), pp. 629–640.

[211] Xu Wang et al. “High-throughput phenotyping with deep learning gives insight into the genetic architecture of flowering time in wheat”. In: *GigaScience* 8.11 (2019), giz120.

[212] Yanbing Wang et al. “The Arabidopsis APC4 subunit of the anaphase-promoting complex/cyclosome (APC/C) is critical for both female gametogenesis and embryogenesis”. In: *The Plant Journal* 69.2 (2012), pp. 227–240. DOI: 10.1111/j.1365-313X.2011.04785.x.

[213] Zhizheng Wang et al. “GeneAgent: self-verification language agent for gene-set analysis using domain databases”. In: *Nature Methods* 22.8 (2025), pp. 1677–1685.

[214] Jacob D Washburn et al. “Global genotype by environment prediction competition reveals that diverse modeling strategies can deliver satisfactory maize yield estimates”. In: *Genetics* 229.2 (2025), iyae195.

[215] Jason Wei et al. "Chain-of-thought prompting elicits reasoning in large language models". In: *Advances in neural information processing systems* 35 (2022), pp. 24824–24837.

[216] Paul J Werbos. "Applications of advances in nonlinear sensitivity analysis". In: *System Modeling and Optimization: Proceedings of the 10th IFIP Conference New York City, USA, August 31–September 4, 1981*. Springer. 2005, pp. 762–770.

[217] Leandro von Werra et al. *TRL: Transformers Reinforcement Learning*. https://github.com/huggingface/trl. 2020.

[218] Cuiling Wu et al. "A transformer-based genomic prediction method fused with knowledge-guided module". In: *Briefings in Bioinformatics* 25.1 (2023).

[219] Xing Wu et al. "Population and adaptation history of 739 *Thlaspi arvense* natural accessions". In: *bioRxiv* (2025). preprint. DOI: 10.1101/2025.03.21.644658.

[220] Enze Xie et al. "SegFormer: Simple and efficient design for semantic segmentation with transformers". In: *Advances in neural information processing systems* 34 (2021), pp. 12077–12090.

[221] Shunyu Yao et al. "React: Synergizing reasoning and acting in language models". In: *The eleventh international conference on learning representations*. 2022.

[222] Shunyu Yao et al. "Tree of thoughts: Deliberate problem solving with large language models". In: *Advances in neural information processing systems* 36 (2023), pp. 11809–11822.

[223] Zhou Yao et al. "GEFormer: A genotype-environment interaction-based genomic prediction method that integrates the gene attention mechanism". In: *Molecular Plant* (2025). DOI: 10.1016/j.molp.2025.01.020.

[224] Andy B. Yoo, Morris A. Jette, and Mark Grondona. "SLURM: Simple Linux Utility for Resource Management". In: *Job Scheduling Strategies for Parallel Processing*. Springer Berlin Heidelberg, 2003, pp. 44–60. DOI: 10.1007/10968987_3.

[225] Jianming Yu et al. "A unified mixed-model method for association mapping that accounts for multiple levels of relatedness". In: *Nature Genetics* 38.2 (2006), pp. 203–208. DOI: 10.1038/ng1702.

[226] Jiayi Zhang et al. "Verbalized Sampling: How to Mitigate Mode Collapse and Unlock LLM Diversity". In: *arXiv preprint arXiv:2510.01171* (2025).

[227] Yanli Zhao et al. *PyTorch FSDP: Experiences on scaling fully sharded data parallel*. 2023. arXiv: 2304.11277 [cs.DC].

[228] Zhihan Zhou et al. "Dnabert-2: Efficient foundation model and benchmark for multi-species genome". In: *arXiv preprint arXiv:2306.15006* (2023).

[229] Xiaoyi Zhu et al. "Important photosynthetic contribution of silique wall to seed yield-related traits in *Arabidopsis thaliana*". In: *Photosynthesis Research* 137.3 (2018), pp. 493–501. DOI: 10.1007/s11120-018-0532-x.

[230] Yuanyuan Zhu et al. "Validation and characterization of a seed number per silique quantitative trait locus qSN.A7 in rapeseed (*Brassica napus* L.)" In: *Frontiers in Plant Science* 11 (2020), p. 68. DOI: 10.3389/fpls.2020.00068.

Appendix A

Supplementary Tables for Chapter 2

Table A.1: Broad-sense heritability (H^2) of all 52 U-Net-derived pod traits in the dryland environment, ordered by H^2. Estimates are from a linear mixed model with genotype as a random effect, fit on 25 replicated genotypes, and significance p is from a restricted likelihood-ratio test on the genotype variance component ($H_0 : \sigma^2_G = 0$). Traits marked * fell below the $H^2 \geq 0.05$ screen and were excluded from association testing; the remaining 34 were carried into GWAS and are reported with their GWAS status in Table 2.1.

Trait	H^2	Significance (p)
envelope-to-wing area ratio	0.461	< 0.0001
envelope-to-total area ratio	0.423	0.0002
wing-to-envelope area ratio	0.409	0.0003
envelope perimeter	0.390	0.0016
envelope area	0.389	0.0017
wing-to-total area ratio	0.387	0.0003
wing area	0.366	0.0010
wing HSV saturation	0.321	0.0022
wing perimeter	0.302	0.0060
seed area	0.297	0.0214
envelope-to-wing perimeter ratio	0.292	0.0006

Table A.1 (continued)

Trait	H^2	Significance (p)
wing-to-envelope perimeter ratio	0.287	0.0009
wing-to-seed area ratio	0.285	0.0077
seed perimeter	0.263	0.0334
seed-to-wing area ratio	0.247	0.0117
envelope-to-total perimeter ratio	0.230	0.0051
wing CIELAB b^*	0.227	0.0525
envelope HSV saturation	0.220	0.0418
seed count	0.218	0.0319
envelope CIELAB b^*	0.217	0.0315
seed RGB red	0.204	0.0881
envelope-to-seed area ratio	0.200	0.0501
seed CIELAB b^*	0.190	0.0627
seed-to-envelope area ratio	0.188	0.0681
seed-to-total area ratio	0.188	0.0453
seed HSV hue	0.185	0.0796
seed HSV saturation	0.155	0.1048
wing RGB blue	0.131	0.1770
seed HSV brightness	0.120	0.1518
seed CIELAB lightness	0.097	0.1672
envelope RGB green	0.089	0.2169
envelope CIELAB lightness	0.086	0.2213
seed RGB blue	0.083	0.1234
envelope HSV brightness	0.068	0.2530
seed RGB green*	0.050	0.2874
envelope HSV hue*	0.028	0.3328
wing-to-total perimeter ratio*	0.026	0.3427

Table A.1 (continued)

Trait	H^2	Significance (p)
envelope RGB red*	0.006	0.4167
envelope RGB blue*	0.000	0.4405
envelope CIELAB a^{**}	0.000	1.0000
envelope-to-seed perimeter ratio*	0.000	0.4513
seed CIELAB a^{**}	0.000	1.0000
seed-to-envelope perimeter ratio*	0.000	1.0000
seed-to-total perimeter ratio*	0.000	1.0000
seed-to-wing perimeter ratio*	0.000	0.4469
wing CIELAB a^{**}	0.000	1.0000
wing RGB green*	0.000	0.4450
wing HSV hue*	0.000	0.4492
wing CIELAB lightness*	0.000	0.4516
wing RGB red*	0.000	1.0000
wing-to-seed perimeter ratio*	0.000	1.0000
wing HSV brightness*	0.000	0.4486

Table A.2: Composition of the nine-layer *Arabidopsis thaliana* multiplex network used for GRIN filtering and MENTOR module derivation. Each layer encodes a distinct line of biological evidence over a shared pool of 26,605 genes; "Genes" is the number of genes with at least one edge in that layer, and layers are ordered by edge count. All layers are undirected, unweighted, and contribute equally to network propagation, giving 918,640 edges in total. Network identifiers follow the exascale-generated *A. thaliana* network collection.

Layer	Source network	Network ID	Genes	Edges
PP	PPI-6merged	`AT-UU-PP-00-AA-01`	19,191	317,787
RP	Regulation-Plantregmap	`AT-UU-RP-03-AA-01`	16,014	167,851
PX	PEN-Diversity	`AT-UU-PX-01-AA-01`	19,975	145,407
KS	Knockout Similarity	`AT-UU-KS-00-AA-01`	1,841	94,952
GA	Coex Gene-Atlas	`AT-UU-GA-01-AA-01`	7,683	84,959
PY	iRF-LOOP Methylation	`AT-UU-PY-01-LF-01`	13,314	71,287
RX	Metabolic-AraCyc	`AT-UU-RX-00-AA-01`	2,857	21,524
DU	CoEvolution-DUO 0.67 trans	`AT-UU-DU-67-AA-01`	2,283	13,514
RE	Regulation-ATRM	`AT-UU-RE-00-AA-01`	789	1,359